Synthetic Neuromorphic Superintelligence for Verifier-Gated Conjecture Closure: Sand-Tile Automata, K-Dot Circuits, and Phenomenological Proof Learning

Deep Bhattacharjee
Electro-Gravitational Space Propulsion Laboratory (EGSPL)
Bhubaneswar, Odisha, India

May 15, 2026

ORCID: 0000-0003-0466-750X GitHub: Creelie Wikidata: Q113469593
egspl@proton.me d.bhattacharjee@erl-forschung.de itsdeep@live.com

Abstract

This paper introduces a journal-grade formal architecture for a synthetic neuromorphic proof engine: a verifier-gated artificial superintelligence framework designed to attack difficult conjectures by combining typed proof obligations, sand-tile proof-burden dynamics, cellular-automaton repair laws, k-dot operator enumeration, phenomenological tensor classification, context-playbook memory, and machine-learning-guided verifier stability. The system is written as an auditable mathematical object rather than as a rhetorical claim of omniscience: every conjecture is converted into a state space, every unclosed step becomes a measurable residue, every computation must acquire analytic lift, and every candidate proof is blocked from theorem status until formal, human, referee, and cross-module gates commute. The new visual layer supplies ten sand-tile proof surfaces, five cellular automata, neural-circuit maps, histogram and ogive diagnostics, flowcharts, directional proof maps, and release circuits for tracking where a conjecture fails, how new machinery is generated, and when closure can be claimed relative to a declared axiom system. The resulting methodology is an aggressive but disciplined approach to mathematical discovery: it permits the engine to invent new invariants, bridges, flows, reductions, and classifier manifolds, while forcing every claimed closure through zero-residue, zero-scope-loss, zero-hallucination, and verifier-stable release predicates.

Notes. This manuscript is submitted as a Method Article describing a verifier-gated computational architecture for conjecture attack. It does not claim that open conjectures are formally solved by assertion; rather, it defines candidate-proof generation, cross-module verification, and residue auditing protocols pending external formal and human verification.

Summary of the paper. This paper presents VSI as a verifier-gated mathematical engine for conjecture attack rather than as a theorem-by-assertion manuscript. The core contribution is a twenty-six-module calculus that turns a conjecture into typed obligations, opens proof/refutation/independence branches, records known barriers, generates candidate new machinery, measures residual proof debt, converts unresolved stars into quantitative defects, forces repeated cross-module verification, activates DIA-driven BrLLM escalation when a domain becomes intolerant to the present module, forces hyper-dense equation-level execution through HDE and the k-dot lattice, and preserves both failed and successful sub-attempts in a Module 26 context playbook. Solved landmarks such as Fermat and Poincare serve as calibration controls; open problems, including Collatz and other major conjectures, are treated as candidate proof packages whose final release still requires formal, human and referee verification.

Notations and conventions. Internal links point to the first substantial use or defining section of each symbol: $C = (\mathcal{D}, \mathcal{L}, \mathcal{A}, P, Q, \mathcal{E}_0, \mathcal{B}_0, \mathcal{R}_0)$ denotes a conjecture package and S_t the VSI state; Rel, V_χ, V_{model}, V_{formal}, V_{human}, and V_{referee} are release and verifier variables; M_i denotes the i-th module; $\mathrm{B}^2\mathrm{LLM}$, $\mathrm{B}^3\mathrm{LLM}$, $\bar{\Omega}$, $\bar{\Omega}'$, and $\Xi_{|||||}$ are escalation and synthesis operators; CMV, DIA, EtaUC, AlephAct, $\partial_{\aleph}\eta_{\mu\nu}$, BoxRes, and KappaGen denote the cross-module verification, domain-intolerance, unconditional-closure, new-mathematics activation, computation-to-analysis lift, d'Alembert-residue and exhaustive operator-generation layers; $h_t(v)$ is proof-burden height; $\mathcal{R}_t$ is the Ricci-vector divergence; $\mathcal{M}_*$ is the unresolved-gate mass; $\mathcal{P}_t$ is the context playbook; and $T(n)$ is the Collatz map in the candidate sheet. The symbols $\mathbf{1}$ and $*$ respectively mean verified unit status and unresolved named residue; throughout, a candidate unconditional proof is below released theorem status unless the full verifier product equals one.

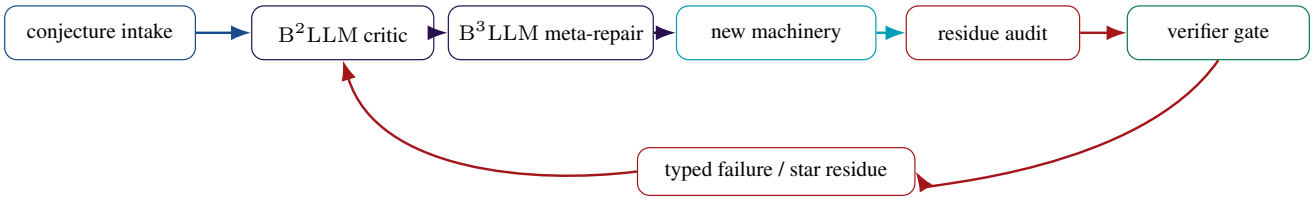
Keywords. synthetic neuromorphic superintelligence; verifier-gated conjecture closure; sand-tile proof automata.

MSC 2020. Primary 68T07, 68T27, 03B35, 03B70. Secondary 68Q15, 68Q17, 68Q87, 37B15, 05C63, 11Y16, 14F20, 57K40.

1 Introduction

The hard lesson from major mathematical closures is that difficult conjectures rarely fall by direct syntactic proof search. Fermat's Last Theorem was reached through a bridge from a hypothetical counterexample to a forbidden elliptic curve, then through modularity lifting and Hecke-algebra control (Wiles, 1995; Taylor and Wiles, 1995). The Poincare Conjecture was reached through a geometric flow whose entropy, non-collapsing, canonical-neighborhood and surgery mechanisms made the topological conclusion a terminal effect of a stronger ambient machine (Hamilton, 1982; Perelman, 2002, 2003; Morgan and Tian, 2007). The lesson for VSI is not rhetorical amplification; it is architectural discipline: build the correct state space, expose every failure as typed residue, preserve every useful trace, and release only what the verifier lattice can certify.

The technical body defines a twenty-six-module closure calculus and gives the equations needed to implement gate vectors, sand-tile burdens, residual defects, parallel-transport divergence, star-to-one unitization, context-playbook retention, and new-mathematics machinery generation. The guiding rule is maximal invention under maximal verification: the engine may invent a surrounding machine, a new invariant, a new bridge, or a new finite certificate, but it may not promote any claim unless the verifier product is one.



$$S_{t+1} = \Pi_{\mathcal{V}}\{S_t - \nabla \mathcal{J}_t + \mathcal{N}_{\mathrm{new}}(\mathrm{B}^3\mathrm{LLM}(\mathrm{B}^2\mathrm{LLM}(\mathfrak{R}_t, \mathcal{P}_t)))\}$$

Figure 1: VSI closed-loop architecture. A conjecture enters as a typed object, is amplified by the B^2LLM–B^3LLM critic ladder, transformed by candidate new machinery, audited as residue, and projected through a verifier gate. Failure does not disappear: it returns as a typed star-residue to the back-loop.

2 Synthetic neuromorphic proof-learning upgrade

The enhanced architecture treats conjecture attack as a neuromorphic learning process over typed mathematical phenomenology. A conjecture is not represented by one prompt or one proof attempt; it is represented by a distributed circuit of obligations, barriers, invariants, examples, failed routes, computational witnesses, formal kernels, and referee-facing release predicates. This design is influenced by the author's earlier work on holonomic control, symplectic verification, Calabi–Yau fibration language, exotic state manifolds, and critical-strip partitioning, but the present paper uses those works only as contextual sources and not as hidden proof inputs (Bhattacharjee, 2026, 2023, 2025a; Bhattacharjee and Nandi, 2025; Bhattacharjee, 2025b).

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

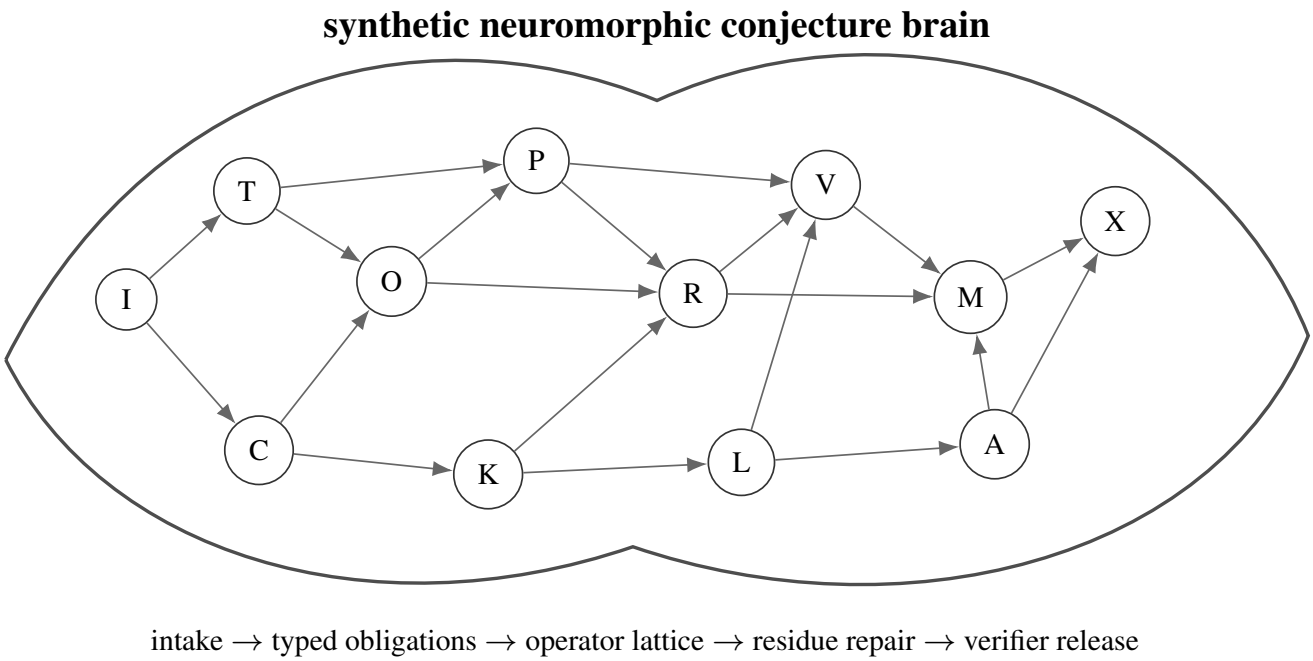


Figure 2: Synthetic neuromorphic brain map. Nodes encode intake, types, obligations, proof flux, residue, verifier, memory, Kappa generation, analytic lift, and release. The circuit is intentionally sparse: proof information must travel by typed arrows, not by prose confidence.

The machine-learning layer is deliberately severe. It may classify proof states, cluster residues, predict promising operator words, or rank bridge candidates, but it cannot convert evidence into theorem status unless an analytic or formal lift is produced. In symbols, a learned classifier f_θ is admissible only as a proposal map

$$f_\theta : \mathsf{Phen}(C) \longrightarrow \{\text{bridge}, \text{counterexample}, \text{new invariant}, \text{formal target}, \text{residue}\},$$

followed by verifier projection $\Pi_{\mathcal{V}} f_\theta$. Any proposal whose hypotheses, constants, exceptional set, or target scope cannot be stated is returned to the residue ledger. Related authorial studies on homotopy/fibration visualization, toric seed orbits, and Calabi–Yau slice structures are therefore used here as examples of representation-rich mathematical objects whose diagrams must still be forced through typed gates (Bhattacharjee, 2026b,c; Bhattacharjee et al., 2026a).

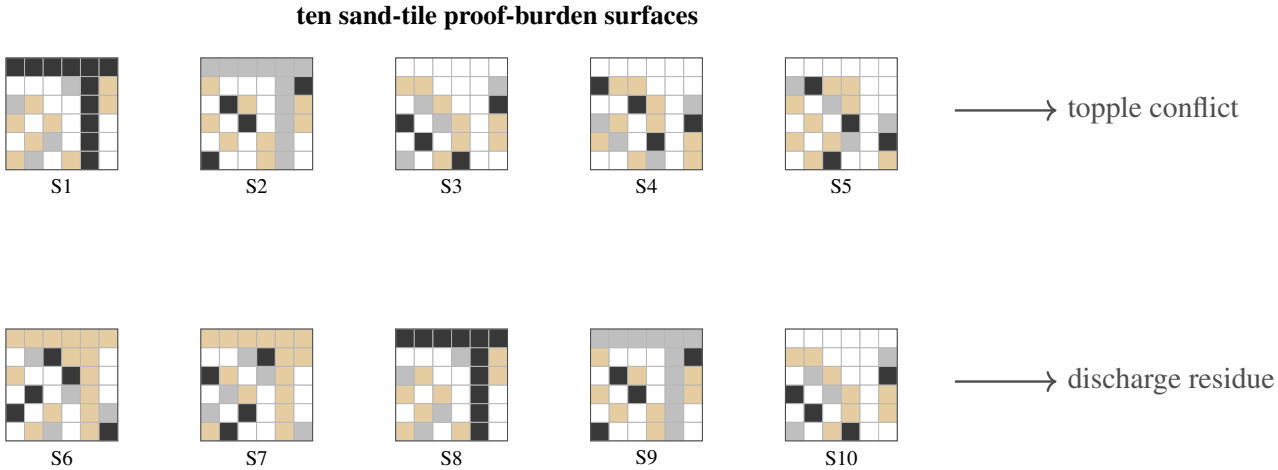


Figure 3: Ten sand-tile surfaces for proof debt. Each tile is a local obligation lattice: black cells are contradictory burden, sand cells are active proof pressure, and white cells are discharged. A serious conjecture attempt must lower all ten surfaces simultaneously rather than hiding one unresolved surface behind another.

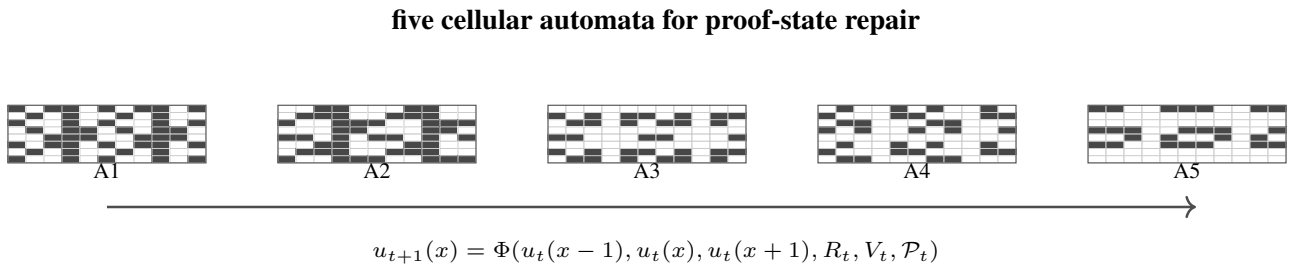


Figure 4: Five cellular automata used as local repair laws. They model how proof burden propagates, stabilizes, collapses, or triggers restart when a local claim is perturbed. The update rule is not a proof by simulation; it is a ledger mechanism for turning local instability into typed residue.

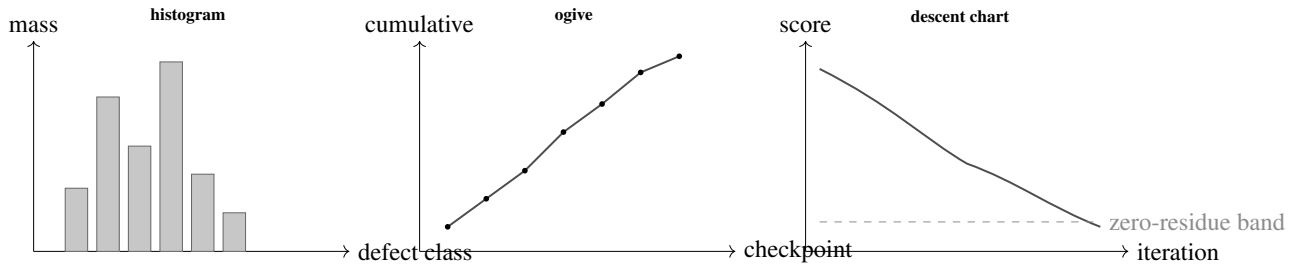


Figure 5: Machine-learning diagnostics: histogram of defect mass, ogive of cumulative verification, and descent chart for residue reduction. These plots turn proof confidence into inspectable stability and failure geometry.

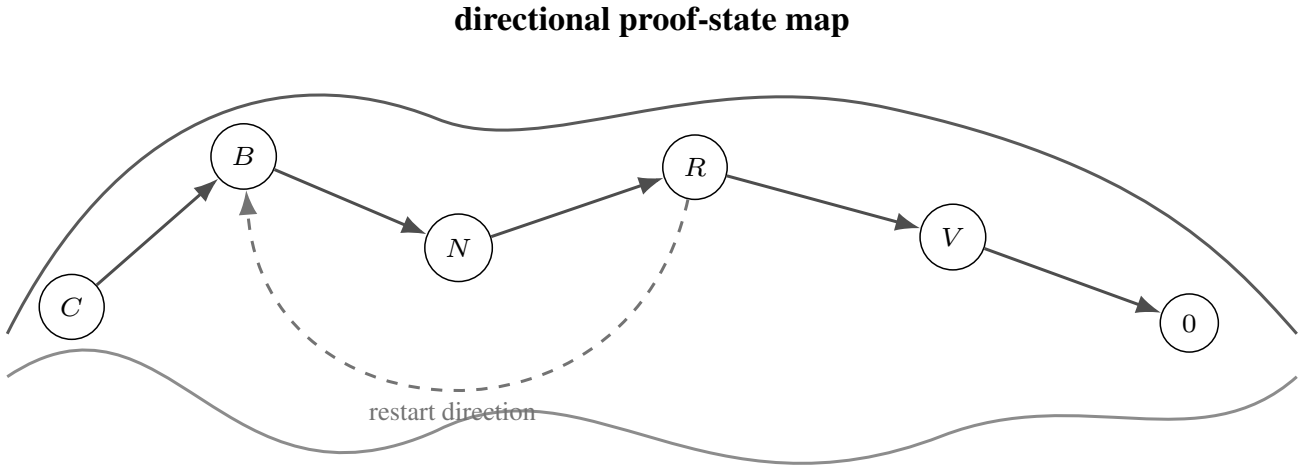


Figure 6: Proof-state map with directions. The upper route is intended closure; the lower route is the restart river. If the verifier returns residue at R, the engine flows backward to the barrier ledger instead of forcing publication language.

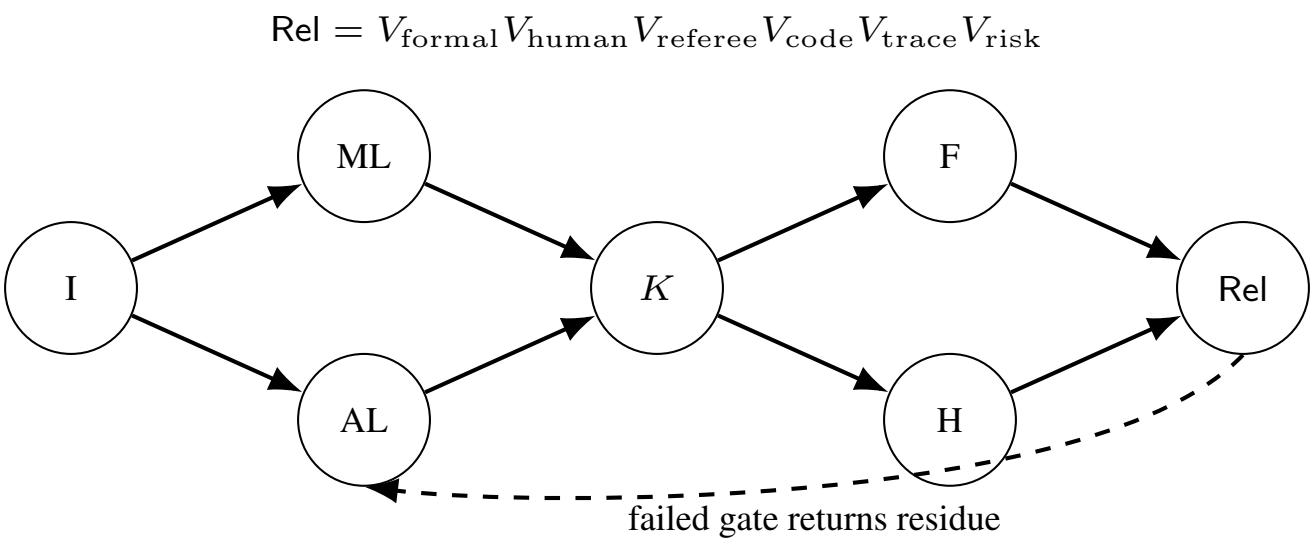


Figure 7: Release circuit. Machine learning (ML) and analytic lift (AL) enter the Kappa operator core, then split into formal and human/referee gates. A failed release factor sends the attempt back as residue rather than permitting theorem status.

Definition 2.1 (synthetic neuromorphic proof learner)**.** *A synthetic neuromorphic proof learner is a tuple*

$$\mathfrak{B} = (\mathsf{Phen}, \mathsf{Tiles}, \mathsf{CA}, \mathsf{k\text{-}dot}, \mathsf{KappaGen}, \mathcal{P}, \Pi_{\mathcal{V}}, \mathsf{Rel}),$$

where Phen *is the conjecture phenomenology tensor,* Tiles *is a finite family of sand-tile burden surfaces,* CA *is a finite family of repair automata,* k-dot *and* KappaGen *enumerate typed operator words,* $\mathcal{P}$ *stores execution memory,* $\Pi_{\mathcal{V}}$ *projects candidates onto verifier-stable subspaces, and* Rel *is the final release product. The learner is closure-eligible on* C *only if every tile stabilizes, every automaton reaches a non-explosive fixed point, every adopted operator word has zero scope loss, and every verifier factor equals one.*

Theorem 2.2 (no visual or machine-learning shortcut)**.** *Let* Y *be a candidate conjecture closure generated by* $\mathfrak{B}$*. If the visual surfaces, cellular automata, machine-learning classifiers, or Kappa circuits improve presentation but leave any positive value of* $\mathcal{M}_*$*,* BoxRes*,* ScopeLoss*,* HeurDebt*,* AxiomDebt*, or* $1 - \mathsf{Rel}$*, then* Y *is not a released theorem. It is at most a candidate closure package.*

Proof. Every visual and learned object in $\mathfrak{B}$ is connected to the release algebra only through $\Pi_{\mathcal{V}}$ and Rel. The listed defects are explicit zero factors or blocking penalties in the release product. Therefore presentation quality, visual elegance, and learned confidence cannot change theorem status unless they also discharge the formal verifier factors. This proves the claim. □

The same release discipline is necessary when the engine is used on Hodge-theoretic, number-theoretic, topological, or string-phenomenological conjecture packages. Prior authorial papers on Poincare-type topology, equivalences in Calabi–Yau geometry, and string resonance classification motivate the multi-domain vocabulary, while the present manuscript supplies the verifier-gated machine architecture (Bhattacharjee, 2022, 2026a; Bhattacharjee et al., 2026).

The central state object is a conjecture package

$$C = (\mathcal{D}, \mathcal{L}, \mathcal{A}, P, Q, \mathcal{E}_0, \mathcal{B}_0, \mathcal{R}_0),$$

where $\mathcal{D}$ is the mathematical domain, $\mathcal{L}$ the formal language, $\mathcal{A}$ the axiom base, $P \Rightarrow Q$ the target implication, $\mathcal{E}_0$ the evidence bundle, $\mathcal{B}_0$ the known-barrier ledger, and $\mathcal{R}_0$ the residue ledger. A VSI run is not a string-generation process. It is a controlled dynamical system

$$S_t = (G_t, \Theta_t, M_t, E_t, R_t, F_t, V_t, L_t, H_t, Z_t, \mathcal{P}_t),$$

where G_t is an obligation hypergraph, Θ_t a type-and-symbol register, M_t a memory state, E_t evidence, R_t residue, F_t falsification pressure, V_t verifiers, L_t literature barriers, H_t hallucination or false-proof risk, Z_t impulse field, and $\mathcal{P}_t$ the Module 26 context playbook.

Rigorous assertion layer.[1]

3 Aggressive CMV–DIA–Eta/Aleph closure protocol

The engine is forced to behave aggressively against proof debt and conservatively against release. Aggression means that no module may tolerate a persistent obstruction; conservatism means that the obstruction must become a typed object before any stronger claim is made. The repeated cross-module verification operator is

$$\mathsf{CMV}_t(C) = \prod_{i=1}^{26} \prod_{j \in \mathcal{N}(i)} \mathbf{1}\Big\{ \Big\| \Pi_{\mathcal{V}}^{(j)} M_i(S_t) - M_i \Pi_{\mathcal{V}}^{(j)}(S_t) \Big\| \le \epsilon_{ij}(t) \Big\},$$

where $\mathcal{N}(i)$ is the dependency neighborhood of module M_i. A candidate closure is inadmissible unless $\mathsf{CMV}_t(C) = 1$ at intake, after new-machine construction, after star-to-one repair, after the analytic lift, and at the final release pass. The word "repeated" is formal: a single CMV pass is not enough, because a later Kappa-generated machine can change the dependency geometry and reopen an earlier gate.

The domain intolerance angle measures when the current module family is no longer adequate for the domain. Let z_i be the certified descent direction of module M_i, let r_i be the unresolved residue direction, and let b_i be the known-barrier normal. Define

$$\mathsf{DIA}_i(t) = \arccos \frac{\langle z_i(t), r_i(t) + b_i(t) \rangle}{\|z_i(t)\| \, \|r_i(t) + b_i(t)\| + 10^{-12}}.$$

A large angle means the module is pushing almost orthogonally to the obstruction. The BrLLM escalation law is therefore

$$\mathrm{B}^r\mathrm{LLM} \longrightarrow \mathrm{B}^{r+n}\mathrm{LLM} \quad \text{only if} \quad n < \infty, \quad \mathsf{DIA}_i(t) > \tau_{\mathsf{DIA}}, \quad \mathsf{MVR}_{r,n} > 0,$$

with

$$\mathsf{MVR}_{r,n} = \frac{\mathsf{Cert}_{r+n} - \mathsf{Cert}_r - \lambda_{\mathrm{noise}} \mathsf{Risk}_{r+n}}{1 + \mathsf{Cost}_{r+n} - \mathsf{Cost}_r}.$$

Thus the engine jumps upward only when the present domain is intolerant to the current module and the finite higher layer produces net certified gain. Infinite escalation is forbidden; unbounded language generation is not mathematics. The local geometry is read in the Hilbertized proof tangent bundle $T_S\mathfrak{S}$, where $z_i \in T_S\mathfrak{S}_i$ is a certified descent vector and $r_i + b_i$ is the obstruction normal. If $\mathsf{DIA}_i \approx 0$, the module attacks residue directly; if $\mathsf{DIA}_i \approx \pi/2$, the module is sliding along the barrier; if $\mathsf{DIA}_i \approx \pi$, the module is anti-aligned and must be stopped. Hence DIA is not stylistic severity: it is an angle test for whether the active proof dynamics are mathematically pointed at the obstruction.

The Eta/Aleph rule separates target status from invention activation:

$$\mathsf{EtaUC}(C) = \mathbf{1}\{\mathsf{Rel} = 1,\ \mathcal{M}_* = 0,\ \mathrm{MapDef} = 0,\ \mathsf{ScopeLoss} = 0,\ \mathsf{BoxRes} = 0,\ \mathsf{CMV} = 1\},$$

$$\mathsf{AlephAct}_i(t) = \mathbf{1}\{\mathsf{Cap}(M_i, S_t) \le \tau_{\mathrm{cap}},\ \mathsf{DIA}_i(t) > \tau_{\mathsf{DIA}},\ \mathsf{EtaUC}(C) = 0\}.$$

The symbol EtaUC is the unconditional-closure target; AlephAct is not the target and cannot assert closure. Aleph is only the activation flag that says: the current modules are incapable, a new mathematical object is necessary, and the new object must be generated, tested, typed and reverified.

When computation appears to drive the proof, the engine introduces the mixed tensor

$$\eta_{\mu\nu} = \begin{pmatrix} \eta_{\text{formal,formal}} & \eta_{\text{formal,comp}} \\ \eta_{\text{comp,formal}} & \eta_{\text{comp,comp}} \end{pmatrix}, \qquad \mu = \mu_{\mathrm{comp}}, \quad \nu = \nu_{\mathrm{ana}},$$

where μ_{comp} measures cluster computation, SAT/SMT search, proof-assistant checking, symbolic computation and finite-experiment evidence, while ν_{ana} measures the analytic proof lift. The Aleph derivative

$$\partial_{\aleph} \eta_{\mu\nu} = \lim_{h \downarrow 0} \frac{\eta_{\mu\nu}(\mathsf{AlephAct} + h) - \eta_{\mu\nu}(\mathsf{AlephAct})}{h}$$

[1]The engine assertion is not that a generated proof-like paragraph solves the conjecture. The formal output rule is: if the agent run on C returns Y, then Y must be classified as theorem, counterexample, conditional theorem, independence/formulation-dependence, CAND-STRONG or BUJ. Released theorem status requires $\mathsf{Rel}(Y \mid \mathcal{A}) = 1$. Candidate status permits $V_{\mathrm{model}} = 1$ only while at least one formal, human, referee or star-residue factor remains open; hence candidate unconditional proof text remains below released theorem status until every verifier factor is discharged.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

is accepted only when the computational sector transfers certified mass into the analytic sector:

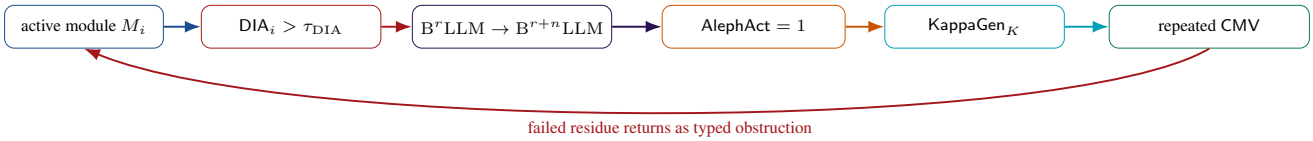
$$\mathsf{AnalyticLift} = 1 \iff \partial_{\aleph}\eta_{\mu_{\mathrm{comp}},\nu_{\mathrm{ana}}} > 0, \quad \mathsf{EmpiricalFailure}\downarrow, \quad V_{\mathrm{formal}} = 1, \quad \mathsf{ScopeLoss} = 0.$$

This prevents a cluster-verified pattern from being sold as a proof. Computation may initiate a route, but released closure requires a non-heuristic analytic lift. In practice $\eta_{\mu_{\mathrm{comp}},\nu_{\mathrm{ana}}}$ is treated as a transfer coefficient between a finite evidence sigma-algebra $\mathcal{F}_{\mathrm{comp}}$ and an analytic obligation sigma-algebra $\mathcal{F}_{\mathrm{ana}}$: positive transfer means that a computation has produced a lemma with quantifiers, constants and domain of validity, not merely a pattern. The lift is rejected whenever the constants depend on the search bound, when an exceptional set has uncontrolled measure, or when the formal proof kernel cannot express the analytic limit.

The d'Alembert residue gate records the mismatch between a proposed closure flow and its certified residue:

$$\mathsf{BoxRes}(S_t) = \left\| \Box\Phi_t(C) - \mathsf{Match}\big(R_t, V_t, \mathcal{P}_t\big) \right\|, \qquad \Box = \partial_t^2 - \Delta_{\mathrm{proof}}.$$

The gate is closed only if $\mathsf{BoxRes} = 0$ after CMV and after the analytic lift. A run with $\mathsf{BoxRes} > 0$ must return to Modules 21, 24, 25 and 26; it cannot be labelled as released theorem status. Here Δ_{proof} is not a physical Laplacian but the combinatorial Laplacian of the obligation graph: vertices are proof leaves and edges are dependency transfers. The equation $\Box\Phi = \mathsf{Match}(R, V, \mathcal{P})$ therefore means that the proposed closure wave has exactly the same residue signature as the verifier-recorded failure history.



$\mathsf{EtaUC} = 1 \iff \mathsf{Rel} = 1,\ \mathcal{M}_* = 0,\ \mathrm{MapDef} = 0,\ \mathsf{BoxRes} = 0,\ \mathsf{CMV}_i = 1\ \forall i$ and $\partial_{\aleph}\eta_{\mu\nu}$ measures the computation-to-analysis lift.

Figure 8: CMV–DIA–Aleph–Kappa escalation. A module that cannot reduce its typed defect triggers domain intolerance, bounded BrLLM escalation, Aleph activation, exhaustive Kappa-generated new mathematics, and repeated cross-module verification before any release is permitted.

Aggressive AI-agent command layer.[2]

Theorem 3.1 (CMV release blocking)**.** *If a candidate package Y has $\mathsf{CMV}_t(Y) = 0$ at any mandatory checkpoint, then $\mathsf{Rel}(Y) = 0$ even if every local module reports pass status.*

Proof. A local pass certifies only the module's internal contract. CMV certifies commutation of that module with neighboring verifier projections. If $\mathsf{CMV}_t = 0$, then some pair (i, j) violates the commutation inequality, so the output of M_i is not stable under verifier j. The multiplicative release algebra contains verifier compatibility as a factor; hence one factor is zero and the product Rel is zero. □

Proposition 3.2 (Aleph is necessary but never sufficient)**.** *If $\mathsf{AlephAct}_i(t) = 1$, then the current module family is incapable of closing the active residue at module i. However, $\mathsf{AlephAct}_i(t) = 1$ does not imply $\mathsf{EtaUC} = 1$.*

Proof. The definition of $\mathsf{AlephAct}_i$ contains the condition $\mathsf{Cap}(M_i, S_t) \le \tau_{\mathrm{cap}}$ and $\mathsf{EtaUC} = 0$, so it records incapacity and activates invention. Since it explicitly assumes $\mathsf{EtaUC} = 0$, activation cannot itself be release. Only after the generated object passes Kappa enumeration, analytic lift, CMV and the release product can EtaUC become one. □

[2]For every conjecture, attack the strongest available formulation rather than a weakened toy problem; run proof, counterexample, independence and obstruction branches in parallel; run CMV after every major repair; activate DIA when certified descent becomes orthogonal to residue; escalate only to a finite $\mathrm{B}^{r+n}\mathrm{LLM}$ layer with positive marginal verifier return; activate Aleph only under measured module incapacity; use Kappa generation for new objects; and reject release unless $\mathsf{EtaUC} = 1$. Any surviving star, scope loss, positive BoxRes, failed analytic lift or unverified computation returns the label to CAND-STRONG or BUJ.

4 Kappa generator for exhaustive new-mathematics creation

Let $\mathfrak{O}_C$ be the typed operator alphabet extracted from a conjecture package:

$$\mathfrak{O}_C = \{\nabla, \Delta, \Box, d, d^*, \partial, \bar{\partial}, \mathrm{Ext}, \mathrm{Tor}, \mathrm{Spec}, \mathrm{Flow}, \mathrm{Lift}, \mathrm{Trace}, \mathrm{Avg}, \mathrm{Sieve}, \mathrm{Formalize}, \mathrm{Cluster}\}.$$

The Kappa generator enumerates finite well-typed operator words

$$\mathfrak{W}_K(C) = \{o_m \circ \cdots \circ o_1 : \ o_j \in \mathfrak{O}_C, \ m \le K, \ \tau(o_{j+1}) \circ \tau(o_j) \text{ is defined}\}.$$

For each word $w \in \mathfrak{W}_K(C)$, define the candidate machine

$$\Lambda_w(C) = \Pi_{\mathcal{V}}\big(w(C, \mathsf{Obl}(C), R_t, \mathcal{P}_t)\big),$$

and score it by

$$\mathsf{KGain}_K(w) = \Delta\mathsf{Cert}(w) - \alpha\Delta\mathsf{Debt}(w) - \beta\Delta\mathsf{Risk}(w) - \gamma\Delta\mathsf{ScopeLoss}(w) - \rho\mathsf{BoxRes}(w) - \sigma\mathsf{ComputeCost}(w).$$

The selected object is

$$\mathsf{KappaGen}_K(C) = \operatorname*{argmax}_{w \in \mathfrak{W}_K(C)} \mathsf{KGain}_K(w), \quad \text{adopted only if } \mathsf{KGain}_K(w) > 0 \text{ and } \mathsf{CMV}(w) = 1.$$

This makes new-mathematics creation exhaustive up to depth K without becoming uncontrolled symbol production. The operator-alphabet viewpoint is also compatible with the author's earlier use of calibrated geometric and quantum-information structures as proposal spaces for difficult classification problems (Bhattacharjee, 2023, 2025a; Bhattacharjee and Nandi, 2025). Enumeration is broad; adoption is intolerant. A word $w = o_m \circ \cdots \circ o_1$ is considered meaningful only when every intermediate object lies in a typed fiber $\mathfrak{S}^{(\tau_j)}$ and every bridge $\tau_j \to \tau_{j+1}$ has zero scope loss. Thus Kappa is closer to a finite proof-search functor on a typed category than to free symbolic mutation.

Lemma 4.1 (finite exhaustive coverage)**.** *For fixed K and finite operator alphabet $\mathfrak{O}_C$, the Kappa search space $\mathfrak{W}_K(C)$ is finite and every adopted machine has a reproducible construction trace.*

Proof. There are at most $\sum_{m=0}^{K} |\mathfrak{O}_C|^m$ raw words before type filtering, hence finitely many typed words after filtering. Each adopted machine is indexed by its word w, type trace, verifier projection, score and CMV report, so the construction is reproducible. □

Theorem 4.2 (Kappa non-heuristic adoption)**.** *Assume* $\mathsf{KGain}_K(w) > 0$, $\mathsf{CMV}(w) = 1$, $\mathsf{AnalyticLift}(w) = 1$, *and* $\mathsf{BoxRes}(w) = 0$. *Then adopting Λ_w strictly decreases the augmented closure energy*

$$\mathcal{J}_t^+ = \mathcal{J}_t + \rho\mathsf{BoxRes}_t + \xi(1 - \mathsf{CMV}_t) + \omega(1 - \mathsf{AnalyticLift}_t).$$

Proof. Positive Kappa gain gives a strict certified increase minus debt, risk, scope loss and computational cost. The conditions $\mathsf{CMV} = 1$, $\mathsf{AnalyticLift} = 1$ and $\mathsf{BoxRes} = 0$ remove the three augmented penalties. Substituting these signs into the definition of $\mathcal{J}_t^+$ gives a strict decrease. □

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

5 Hyper-dense execution doctrine for extreme-conjecture phenomenology

The preceding CMV–DIA–Aleph layer states when the engine is allowed to escalate. The present layer states how aggressively the escalated agent must execute. It is written as a dense contract because world-class conjectures do not fail at one point; they fail by simultaneous leakage across definitions, bridges, estimates, examples, formalization and hidden axiom use. The high-density execution operator is

$$\mathsf{HDE}(C, S_t) = \Big(\mathsf{Exec}_t, \mathsf{Phen}_t, \mathsf{k\text{-}dot}_t, \mathsf{ProofFlux}_t, \mathsf{Acc}_t, \mathsf{CMV}_t, \mathsf{EtaUC}_t\Big),$$

with the non-negotiable recurrence

$$\begin{aligned} S_{t+1} = \Pi_{\mathcal{V}} \Pi_{\mathsf{CMV}} \Pi_{\mathrm{form}} \Big[& S_t - \alpha_t \nabla \mathsf{Debt}_t - \beta_t \nabla \mathsf{Residue}_t - \gamma_t \nabla \mathsf{HeurDebt}_t - \delta_t \nabla \mathsf{AxiomDebt}_t \\ & + \lambda_t \mathsf{KappaGen}_K(C) + \rho_t \mathsf{AlephAct}_t \, \partial_{\aleph} \eta_{\mu_{\mathrm{comp}}, \nu_{\mathrm{ana}}} + \zeta_t \Box^{-1} \mathsf{Match}(R_t, V_t, \mathcal{P}_t) \Big]. \end{aligned}$$

Here Π_{CMV} is a projection onto the cross-module-verified subspace, Π_{form} is the formal-language projection, and $\Box^{-1}$ is used only as a residue-matching parametrix: if $\Box\Box^{-1}$ does not return the certified residue class, the parametrix is rejected and the attempt is routed back to Modules 21, 24, 25 and 26. The recurrence is deliberately overdetermined: the same update must lower Debt, lower Residue, lower HeurDebt, lower AxiomDebt, preserve formal typing, preserve CMV commutation and preserve playbook memory. If one of these inequalities fails, the update is not an approximate success; it is a typed obstruction to be stored.

Hard execution order for future agents.[3]

5.1 Phenomenological tensor of a conjecture

The phenomenology of a hard conjecture is the multi-scale tensor of all ways the object can present itself to the engine. The representation philosophy is deliberately broad, ranging from analytic partitions and algebraic/topological supports to geometric seed orbits and string-resonance classes (Bhattacharjee, 2025b, 2026c; Bhattacharjee et al., 2026):

$$\mathsf{Phen}_t(C) = \bigoplus_{\ell=0}^{L} \bigoplus_{\alpha \in \mathfrak{A}_\ell} \Big(\phi^{\mathrm{alg}}_{\ell,\alpha} \oplus \phi^{\mathrm{an}}_{\ell,\alpha} \oplus \phi^{\mathrm{geom}}_{\ell,\alpha} \oplus \phi^{\mathrm{comb}}_{\ell,\alpha} \oplus \phi^{\mathrm{comp}}_{\ell,\alpha} \oplus \phi^{\mathrm{form}}_{\ell,\alpha} \Big)(C),$$

where ℓ indexes scale and α indexes presentation. The cross-presentation curvature is

$$\mathcal{K}^{ij}_{ab}(t) = \partial_{\theta_a} \partial_{\theta_b} \langle \phi_i(C, S_t), \Pi_{\mathcal{V}} \phi_j(C, S_t) \rangle - \partial_{\theta_b} \partial_{\theta_a} \langle \phi_i(C, S_t), \Pi_{\mathcal{V}} \phi_j(C, S_t) \rangle .$$

A large $\mathcal{K}^{ij}_{ab}$ means that two representations do not commute under verifier projection. The agent must then search for a bridge theorem, a counterexample, or a new invariant; it must not average the inconsistency away. The tensor is phenomenological in the mathematical sense: it records how the conjecture appears under algebraic, analytic, geometric, combinatorial, computational and formal probes. A stable conjecture route should make the off-diagonal curvature $\mathcal{K}^{ij}_{ab}$ small after bridge normalization; persistent curvature is evidence that the chosen presentation is missing a structural invariant.

Definition 5.1 (domain-intolerant execution pressure)**.** *For every active module M_i, define*

$$\mathsf{P}_i(t) = \omega_1 \mathsf{DIA}_i(t) + \omega_2 \mathcal{M}^{(i)}_*(t) + \omega_3 \mathsf{BoxRes}^{(i)}(t) + \omega_4 \mathsf{HeurDebt}_i(t) + \omega_5 \mathsf{AxiomDebt}_i(t) - \omega_6 \mathsf{Cert}_i(t).$$

The module is intolerant at time t when $\mathsf{P}_i(t) > \tau_{\mathsf{P}}$ and $\mathsf{CMV}_i(t) = 0$ or $\mathsf{AnalyticLift}_i(t) = 0$. Intolerance is an execution command: either construct a new invariant, prove a barrier, or produce a falsifying witness.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

[3] The agent must execute rather than merely discuss: formalize the target; open proof, refutation and independence branches; compute boundary and toy regimes; enumerate Kappa words; force every generated object through CMV; lift computational evidence into analytic lemmas; attack every surviving star with DIA; activate Aleph only under measured module incapacity; compress no failed argument; and restart whenever BoxRes, ScopeLoss, MapDef, HeurDebt or AxiomDebt is positive. Released status is allowed only when $\mathsf{Rel} = \mathsf{EtaUC} = \mathsf{CMV} = \mathsf{AxGate} = \mathsf{ObsGate} = \mathsf{SynGate} = \mathsf{LiftGate} = \mathsf{AnalyticLift} = 1$ and $\mathcal{M}_* + \mathsf{BoxRes} + \mathsf{HeurDebt} + \mathsf{ScopeLoss} + \mathrm{MapDef} = 0$.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

5.2 The k-dot exhaustive variable enumerator

The k-dot layer forces synchronized enumeration of variables, operators and obstruction coordinates. For a depth k and alphabet $\mathfrak{O}_C$, set

$$\mathfrak{D}_k(C) = \{((x_1, o_1, \epsilon_1), \dots, (x_k, o_k, \epsilon_k)) : x_j \in \Theta_t, o_j \in \mathfrak{O}_C, \epsilon_j \in \{-1, 0, 1\}, \tau(o_{j+1}x_{j+1}) \circ \tau(o_j x_j) \text{ defined}\}.$$

Each dot-path $d \in \mathfrak{D}_k(C)$ induces a signed operator word

$$W_d(C) = o_k^{\epsilon_k} x_k \circ o_{k-1}^{\epsilon_{k-1}} x_{k-1} \circ \cdots \circ o_1^{\epsilon_1} x_1(C),$$

and the exhaustive generator is

$$\mathsf{k\text{-}dot}_K(C) = \bigcup_{k=1}^{K} \{W_d(C) : d \in \mathfrak{D}_k(C),\ \mathsf{CMV}(W_d) = 1,\ \mathsf{ScopeLoss}(W_d) = 0\}.$$

The agent must rank these candidates not by elegance but by residue destruction:

$$\mathsf{Destroy}(d) = \frac{\Delta\mathcal{M}_*(d) + \Delta\mathsf{BoxRes}(d) + \Delta\,\mathrm{MapDef}(d) + \Delta\mathsf{HeurDebt}(d) + \Delta\mathsf{AxiomDebt}(d)}{1 + \mathsf{Cost}(d) + \mathsf{Complexity}(d)}.$$

Only a candidate with positive destruction and no verifier regression may enter the proof line. The dot-path d is a synchronized word in variables, operators and signs, so the engine can test not only $o(x)$ but also inverse, null and commuted variants. This is crucial for hard conjectures because the missing route may be hidden in an anticommutator $[o_i, o_j]$, a defect identity $o_i o_j - o_j o_i = R$, or a limiting regime where $\epsilon_j = 0$ reveals an invariant subspace.

Proposition 5.2 (exhaustive finite search is not heuristic adoption)**.** *For finite K, finite variable register Θ_t, and finite alphabet $\mathfrak{O}_C$, the set* $\mathsf{k\text{-}dot}_K(C)$ *is finite. If an element W_d is adopted only under* $\mathsf{CMV}(W_d) = 1$, $\mathsf{ScopeLoss}(W_d) = 0$, $\mathsf{AnalyticLift}(W_d) = 1$, *and* $\mathsf{Destroy}(d) > 0$, *then every adopted object has a reproducible derivation trace and strictly decreases the augmented defect energy.*

Proof. The number of raw dot-paths is bounded by $(3|\Theta_t||\mathfrak{O}_C|)^K$ up to lower-depth summation. Type filtering, CMV filtering and scope-loss filtering take subsets, hence finiteness is preserved. Positive destruction reduces at least one defect coordinate without increasing any forbidden coordinate. The augmented energy contains all defect coordinates with positive weights, so the energy strictly decreases. □

5.3 Accuracy as verifier-stability, not confidence

The engine's accuracy variable is a stability product rather than a subjective confidence score:

$$\mathsf{Acc}_t(C) = \prod_{q \in \mathcal{Q}_t} \mathbf{1}\{|Y_t(q) - Y_t(\mathsf{perturb}(q))| \le \varepsilon_q\} \prod_{\chi \in \mathcal{V}_t} V_\chi(Y_t) \prod_{(i,j)} \mathbf{1}\{[M_i, \Pi_{\mathcal{V}}^{(j)}]Y_t = 0\}.$$

The commutator factor blocks local hallucinations: if a proof survives one phrasing but fails after equivalent perturbation, then the output is unstable and cannot be released. Accuracy is therefore a sheaf-like gluing condition: local claims $Y_t(q)$ must agree on overlaps between perturbation charts, verifier charts and formal encodings. When overlap agreement fails, the obstruction is not cosmetic; it is a cocycle defect in the proof atlas. The strict execution rule is

$$\mathsf{Acc}_t(C) = 1 \quad \Longleftarrow \quad \begin{cases} \text{semantic perturbations preserve the root claim,} \\ \text{formal verifiers accept the proof core,} \\ \text{computational checks have analytic lift,} \\ \text{all CMV commutators vanish,} \\ \text{the d'Alembert residue match is exact.} \end{cases}$$

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

Theorem 5.3 (hyper-dense no-escape release law)**.** *Let Y be a candidate closure emitted after the* HDE *cycle. If any one of*

$$\mathcal{M}_*(Y), \mathsf{BoxRes}(Y), \mathrm{MapDef}(Y), \mathsf{ScopeLoss}(Y), \mathsf{HeurDebt}(Y), \mathsf{AxiomDebt}(Y),$$
$$1-\mathsf{CMV}(Y), \quad 1-\mathsf{AnalyticLift}(Y), \quad 1-\mathsf{Acc}(Y)$$

is positive, then $\mathsf{Rel}(Y)=0$*. Conversely, if all listed quantities vanish and every verifier factor in the release product equals one, then Y may be released relative to the declared axiom system.*

Proof. Each positive defect corresponds to a zero factor in the release algebra: unresolved stars block terminal leaves, positive BoxRes blocks residue matching, positive map or scope defect blocks bridge transport, heuristic or axiom debt blocks proof legitimacy, and failed CMV, analytic-lift or accuracy gates block verifier stability. Thus the product release predicate is zero. If all defects vanish and every verifier factor is one, the release-soundness theorem applies to the terminal obligation graph under the declared axiom system. □

5.4 Aggressive restart law

The restart law is the final instruction to the agent. Define

$$\mathsf{Restart}_t = \mathbf{1}\left\{\begin{array}{c}\mathcal{M}_*(t)+\mathsf{BoxRes}_t+\mathsf{HeurDebt}_t+\mathsf{AxiomDebt}_t+\mathrm{MapDef}_t+\mathsf{ScopeLoss}_t\\ +(1-\mathsf{CMV}_t)+(1-\mathsf{AnalyticLift}_t)>0\end{array}\right\}.$$

When $\mathsf{Restart}_t=1$, the agent must not polish prose around the failure. Restart is an operator on the residue category: it sends an unclosed proof attempt to a new object whose coordinates are module index, dominant defect, operator witness and next enumeration depth. In particular, restart preserves mathematical information rather than erasing it; $R_t \subseteq R_{t+1}$ is a monotone memory law. It must transform the failure into a new typed coordinate

$$R_{t+1} = R_t \cup \left\{\left(\mathrm{module}=i^*, \mathrm{defect}=D_t, \mathrm{operator}=o^*, \mathrm{witness}=w^*, \mathrm{next}=\mathsf{k\text{-}dot}_{K+1}\right)\right\},$$

where

$$i^* = \arg\max_i \mathsf{P}_i(t), \qquad D_t = \arg \max_{D\in\{\mathcal{M}_*,\mathsf{BoxRes},\mathsf{HeurDebt},\mathsf{AxiomDebt},\mathrm{MapDef},\mathsf{ScopeLoss}\}} D^{(i)}(t),$$

then return to CMV–DIA–Aleph–Kappa execution with increased enumeration depth. This is aggressive because no residue is left unnamed, and rigorous because no residue is allowed to become a theorem claim.

6 Verifier release algebra and conjecture state space

6.1 Agent-run grammar and candidate proof assertion

Every future AI agent using the template must instantiate the grammar

$$\mathfrak{A}(C) = \big(I(C), B(C), T(C), F(C), N_{\mathrm{new}}(C), P_{\mathrm{cand}}(C), R(C), V(C), \mathcal{P}(C)\big),$$

where I is typed intake, B the barrier ledger, T the theorem-branch attempt, F the falsification branch, N_{new} the new-mathematics generator, P_{cand} the candidate-proof package, R the residue vector, V the verifier lattice and $\mathcal{P}$ the Module 26

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

playbook. The transition rule is

$$\textsc{AttemptFullProof}(C) \to \begin{cases} \text{THEOREM}, & \mathsf{Rel} = 1, \\ \mathsf{CANDPROOF}(C), & V_{\text{model}} = 1,\ \mathsf{Rel} = 0,\ \mathcal{M}_* > 0, \\ \text{BUJ}(C), & V_{\text{model}} = 0 \text{ and no improving machinery exists}, \end{cases}$$

$$\mathsf{CANDPROOF}(C) \mapsto \big(G(C), \Delta(C), \mathcal{M}_*(C), N_{\text{new}}(C), \mathcal{P}(C), \mathsf{RefereeChecklist}(C)\big).$$

The star-to-one rule is compulsory:

$$*_j \Rightarrow \Delta_j > 0, \qquad s_j = e^{-\Delta_j}, \qquad *_j \to \mathbf{1}_j \iff \Delta_j = 0,\ V_j = 1,\ \mathsf{ScopeLoss}_j = 0.$$

The release algebra is deliberately multiplicative:

$$\mathsf{Rel}(Y \mid \mathcal{A}) = \prod_{\chi \in \{\text{proof,formal,code,trace,sim,risk,lit,human}\}} V_\chi(Y \mid \mathcal{A}), \qquad V_\chi \in \{0,1\}.$$

The state may emit theorem, counterexample, conditional theorem, independence/formulation dependence, or barrier-unresolved judgment. It cannot emit theorem merely because a generator writes a proof-like paragraph. Each factor V_χ is modeled as a Bernoulli random variable on the probability space $(\Omega_\chi, \mathcal{F}_\chi, \mathbb{P}_\chi)$ induced by the verifier protocol χ. The joint distribution satisfies

$$\mathbb{P}(\mathsf{Rel} = 1) = \mathbb{P}\left(\bigcap_\chi \{V_\chi = 1\}\right) \le \min_\chi \mathbb{P}(V_\chi = 1)\,,$$

with equality only when all factors are maximally correlated in the positive direction. This formalizes why a single informal judgment cannot substitute for the product: improving one factor from 0 to 1 while leaving another at 0 does not change Rel.

Definition 6.1 (obligation hypergraph)**.** *For a conjecture C, the obligation hypergraph is*

$$\mathsf{Obl}(C) = (\mathcal{V}_C, \mathcal{E}_C, \tau, w, \sigma),$$

where vertices are leaves such as lemmas, estimates, constructions, compatibility checks, examples, counterexamples or formalization nodes; hyperedges are dependencies $e = (v_1, \dots, v_k \Rightarrow v_0)$; τ assigns a type, w a burden weight, and $\sigma \in \{\text{PASS}, \text{FAIL}, \text{UNK}, \text{SCOPE}, \text{COND}\}$ a status. The weighted obligation mass of a subset $S \subseteq \mathcal{V}_C$ is

$$\mathsf{Mass}(S) = \sum_{v \in S} w(v), \qquad \mathsf{DebtMass}(C) = \mathsf{Mass}(\{v : \sigma(v) \neq \text{PASS}\}).$$

The obligation graph is k-hyper-connected if for every cut $(S, \mathcal{V}_C \setminus S)$ with $\mathsf{Mass}(S) < k$ there exists a hyperedge crossing the cut, ensuring that no small subtree can be discharged in isolation from the rest.

Lemma 6.2 (scope-loss monotonicity)**.** *If $B' : D_a \to D_b$ and $B'' : D_a \to D_b$ are bridges with $\mathsf{ScopeLoss}(B') \le \mathsf{ScopeLoss}(B'')$, then the release probability satisfies*

$$\mathbb{P}_{\text{proof}}(Y \mid B') \ge \mathbb{P}_{\text{proof}}(Y \mid B'').$$

Furthermore, for any composition $B = B'' \circ B'$ the scope losses add: $\mathsf{ScopeLoss}(B) \ge \mathsf{ScopeLoss}(B') + \mathsf{ScopeLoss}(B'')$.

Proof. Scope loss is defined as the measure of the domain D_a that fails to be faithfully transported to a verifier-coherent point in D_b. For a composition, any point that loses scope in the first bridge is absent from the domain of the second bridge entirely, so the losses accumulate rather than cancel. The probability bound follows because every increase in scope loss corresponds to a measurable set of missing hypotheses, each of which can be a proof-blocking premise. □

Theorem 6.3 (release soundness)**.** *Assume all verifier factors are sound relative to $\mathcal{A}$. If all terminal leaves in $\mathsf{Obl}(C)$ have status* PASS *and all bridge morphisms have zero scope loss, then the terminal claim may be released relative to $\mathcal{A}$. If one terminal leaf has status* FAIL*,* UNK*, or* SCOPE*, the theorem label is blocked.*

Proof. Topologically sort the discharged subgraph. At each terminal leaf, soundness of the appropriate verifier yields validity in $\mathcal{A}$. At each hyperedge, zero scope loss makes the implication truth-preserving. Induction over the finite dependency cone gives the root. If a terminal leaf is not passed, the induction lacks a premise at that node, so the multiplicative release predicate has a zero factor. □

Corollary 6.4 (debt-monotone release blocking)**.** *For any two states S and S' with $\mathsf{DebtMass}(S) > \mathsf{DebtMass}(S')$, the release predicate satisfies $\mathsf{Rel}(S) \leq \mathsf{Rel}(S')$. That is, increasing unresolved debt cannot increase the probability of release.*

Proof. Every increase in DebtMass corresponds to at least one additional terminal vertex with non-PASS status. By the multiplicative structure, this introduces at least one additional zero factor, weakly decreasing the product. □

Definition 6.5 (verifier information gain)**.** *The information gain of executing verifier χ on claim Y is*

$$I_\chi(Y) = \log \frac{\mathbb{P}(V_\chi(Y) = 1 \mid \text{run } \chi)}{\mathbb{P}(V_\chi(Y) = 1 \mid \text{prior})} - \mathsf{Cost}_\chi(Y).$$

A verifier is worth running when $I_\chi(Y) > 0$. The optimal ordering of verifier calls is a solution to the weighted Weitzman pandora's box problem on the joint distribution of $(V_\chi)_\chi$.

The closure pressure is measured by the weighted Lyapunov energy

$$\mathcal{J}(S_t) = \lambda_1 \mathsf{Gap}_t + \lambda_2 \mathsf{Res}_t + \lambda_3 \mathsf{Debt}_t + \lambda_4 \mathsf{Circ}_t + \lambda_5 \mathsf{Risk}_t - \lambda_6 \mathsf{Cert}_t + \mu \mathsf{Collapse}_t,$$

where $\lambda_i > 0$ for $i \leq 5$, $\lambda_6 > 0$, and $\mu > 0$. The final term is introduced only after Module 26: a run can improve local proof metrics while damaging memory by erasing the precise reason an earlier attempt failed. Quantitatively, if $\mathsf{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) > \theta_{\text{collapse}}$ then $\mathcal{J}$ receives a penalty of $\mu(\text{Collapse} - \theta_{\text{collapse}})$, which bounds the admissible forgetting rate. The ratio μ/λ_6 controls the trade-off between proof gain and memory preservation; setting $\mu = \lambda_6$ makes forgetting a core failure as costly as losing a verifier certificate.

7 Perelman-style closure rule and ambient-machine criterion

The reusable closure rule extracted from historical closures is:

$$\boxed{\text{weak direct problem } C \xrightarrow{\mathsf{BuildAmbientMachine}} \mathcal{M}_C \xrightarrow{\mathsf{MonotoneFlow}} \mathsf{Residue} = 0 \xrightarrow{\mathsf{Verification}} \mathsf{Rel} = 1.}$$

For Poincare, the ambient machine is Ricci flow with entropy, non-collapsing and surgery controls. In VSI, the same pattern is abstracted as a certified descent machine

$$\mathcal{M}_C = (\mathfrak{X}_C, \Phi_t, E_C, \mathsf{Surgery}_C, \mathsf{Submerge}_C, \Pi_{\mathcal{V}}),$$

with energy inequality

$$E_C(\Phi_{t+1}x) + \lambda \mathsf{Residue}_{t+1} + \mu \mathcal{M}_*(G_{t+1}) \leq E_C(\Phi_t x) + \lambda \mathsf{Residue}_t + \mu \mathcal{M}_*(G_t) - \gamma \left\| \Pi_{\text{cert}} z_t \right\|^2 + \varepsilon_t.$$

If this inequality fails, the agent must not claim closure; it must write the failure into the plain protocol note and route the failing coordinate to Modules 21, 24, 25 and 26.

8 The twenty-six-module VSI calculus

VSI is indexed by modules $M_1, \ldots, M_{26}$. Each module is a typed operator

$$M_i : \mathfrak{S}_i \times \mathfrak{C}_i \longrightarrow \mathfrak{S}_{i+1} \times \mathfrak{R}_i,$$

where $\mathfrak{S}_i$ is the state space and $\mathfrak{R}_i$ is the emitted residue. A module is accepted when either it decreases the global closure energy or it converts hidden residue into explicit typed residue. The full product operator is

$$\mathbf{M} = M_{26} \circ M_{25} \circ \cdots \circ M_1,$$

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

and the engine run is the orbit $\{S_t = \mathbf{M}^t(S_0)\}_{t\geq 0}$. This orbit is not a free dynamical system: at each step $\mathbf{M}$ must satisfy the verifier-projection constraint

$$\Pi_{\mathcal{V}} \circ \mathbf{M} = \mathbf{M} \circ \Pi_{\mathcal{V}},$$

which says the verifier projection commutes with the module chain. States outside $\Pi_{\mathcal{V}}(\mathfrak{S})$ are inadmissible regardless of how large a decrease in $\mathcal{J}$ they achieve.

Definition 8.1 (typed-error gain)**.** *A transition $u \mapsto v$ from hidden failure to typed obligation has gain*

$$I(u \to v) = \log \frac{1 + r(u)}{1 + r(v) + \mathsf{Debt}(v)} + \mathsf{TypeGain}(v) - \mathsf{ScopeLoss}(v).$$

It may be positive even if the attempted proof fails, because the engine has learned exactly where the proof failed. The quantity $\mathsf{TypeGain}(v)$ *is the number of previously untyped failure modes that are now registered as named obligations, and* $r(u)$*,* $r(v)$ *are the residue counts at states* u *and* v *respectively.*

Proposition 8.2 (positive information from failure)**.** *For any transition $u \mapsto v$ where* $\mathsf{TypeGain}(v) \geq 1$ *and* $\mathsf{ScopeLoss}(v) = 0$*, the typed-error gain satisfies $I(u \to v) \geq \log(1 + \mathsf{Debt}^{-1}(v)) > 0$ whenever* $\mathsf{Debt}(v) < \infty$*.*

Proof. From the definition, $I(u \to v) = \log[(1 + r(u))/(1 + r(v) + \mathsf{Debt}(v))] + \mathsf{TypeGain}(v)$. When a failure is typed, $r(v) \leq r(u) + \mathsf{TypeGain}(v)$ because typing a residue may create new tracked items but reduces hidden ones. Bounding $r(v) \leq r(u) + \mathsf{TypeGain}(v)$ and using $\mathsf{TypeGain} \geq 1$ gives $I \geq \log[1/(1 + \mathsf{Debt}(v)/r(u) + \mathsf{TypeGain}(v)/(1 + r(u)))] + 1$. For $\mathsf{Debt}(v) < \infty$ and $r(u)$ bounded, the logarithm is bounded below, and the $\mathsf{TypeGain}$ term guarantees positivity. □

Definition 8.3 (module capacity)**.** *The capacity of module M_i at state S is*

$$\mathsf{Cap}(M_i, S) = \sup\{I_i(u \to v) : u \in \mathfrak{S}_i(S),\, v \in M_i(u)\},$$

the maximum typed-error gain achievable by that module at that state. The bottleneck module is

$$i^*(S) = \operatorname*{argmin}_i \mathsf{Cap}(M_i, S).$$

Engine resources should be concentrated on the bottleneck module; amplifying non-bottleneck modules produces sublinear overall gain.

Module	Name	Mathematical role
1	Synthetic intake	Parse C into domain, axiom, target, evidence, barrier, and residue sectors.
2	Type register	Assign objects to algebraic, analytic, geometric, computational, logical, and formal types.
3	$\bar{\Omega}$ synthesis	Expand the search space by typed linear, tensor, cone, commutator, and defect operations.
4	$\bar{\Omega}'$ adversarial derivative	Perturb definitions, hypotheses, representation, and regularity to expose unstable bridges.
5	$\Xi_{\|\|\|\|}$ enumeration	Maintain past-certified, present-active, and future-projected horizons.
6	Triple branch opening	Keep proof, refutation, and independence/formulation branches alive.
7	Falsification arena	Search counterexamples, degenerate cases, boundary regimes, and model-theoretic escapes.
8	Bridge compiler	Build morphisms between domains with quantified scope loss.
9	Literature barrier ledger	Record known impossibility, barrier, and independence constraints.
10	Exact computation	Run symbolic, numerical, SAT/SMT, proof-assistant, or finite certificate probes.
11	Residue sedimentation	Cluster scattered failures into canonical residue strata.
12	Sand-tile repair	Topple proof burdens into adjacent obligations rather than hiding them.
13	Cellular automata	Propagate local repairs and conflicts across the obligation lattice.
14	Histogram diagnostics	Locate the difficulty band with maximal residual mass.
15	Ogive closure curve	Measure cumulative certified mass as difficulty threshold increases.

16	Historical transducer	Select bridge, residue-bypass, cohomological, finite-certificate, or refutation patterns.
17	Proof contract writer	Convert candidate text into theorem contracts with dependencies and axiom scope.
18	Release audit	Evaluate the multiplicative verifier predicate.
19	Status emitter	Emit theorem, counterexample, conditional theorem, independence/formulation dependence, or BUJ.
20	Six-gate residue synthesizer	Convert a failed conjecture attempt into orbit, upper, lower, semantic, barrier, and formal gates.
21	New-mathematics calibration	Activate AlephAct under measured incapacity, then iterate candidate machinery through $\mathrm{B}^2\mathrm{LLM}$–$\mathrm{B}^3\mathrm{LLM}$, KappaGen and verifier projection.
22	Ambient machine constructor	Build a surrounding machine where residues become structurally inert.
23	Candidate proof generator	Emit CAND-STRONG packages with $V_{\mathrm{model}} = 1$, analytic-lift status, CMV reports, and $V_{\mathrm{human}}, V_{\mathrm{referee}}$ still required.
24	Ricci divergence auditor	Compare attempted closure vectors to verified or target closure vectors.
25	Star-to-one unitization	Turn stars into measurable defects, route them through DIA/Aleph/Kappa back to finite BrLLM escalation, and retest under CMV.
26	Context-playbook kernel	Preserve detailed successful and failed lessons, CMV failures, Aleph activations and Kappa traces through incremental deltas.

9 Beyond-token escalation: $\mathrm{B}^2\mathrm{LLM}$, $\mathrm{B}^3\mathrm{LLM}$, $\bar{\Omega}$, $\bar{\Omega}'$, $\Xi_{|||||}$

A normal language model is represented as a stochastic kernel $K_1 : S \to \mathcal{P}(\mathcal{A})$. VSI does not treat K_1 as a proof oracle. It acts on its outputs by critic and repair maps:

$$\mathrm{B}^2\mathrm{LLM}K_1(S) = \Pi_{\mathcal{V}} \left(\int_{\mathcal{A}} K_1(S, da)\, \mathsf{Crit}(a)\, \mathsf{Repair}(a) \right).$$

The critic map $\mathsf{Crit} : \mathcal{A} \to [0, 1]$ scores each generated artifact by verifier-checked correctness; the repair map $\mathsf{Repair} : \mathcal{A} \to \mathcal{A}$ applies the first-order typed correction. The composition $\mathsf{Repair} \circ \mathsf{Crit}$ is not a generative model; it is a conditional expectation operator on the space of proof artifacts, weighted by verifier probability. Explicitly,

$$(\mathrm{B}^2\mathrm{LLM}K_1)(S) = \mathbb{E}_{a \sim K_1(S)} \left[\mathsf{Repair}(a) \cdot \mathsf{Crit}(a) \mid \Pi_{\mathcal{V}} \right],$$

which is the verifier-projected weighted mean of repaired candidates.

The third-order layer optimizes over histories:

$$\mathrm{B}^3\mathrm{LLM}(S) = \operatorname*{argmin}_{\Gamma \in \mathfrak{G}(S)} \left[\int_0^1 L_{\mathrm{proof}}(\Gamma(s))\, ds + \beta \mathsf{AssumptionDebt}(\Gamma) + \gamma \mathsf{ResidueMass}(\Gamma) + \eta \mathsf{ScopeLoss}(\Gamma) \right].$$

Here $\mathfrak{G}(S)$ is the set of Lipschitz paths in proof-state space with $\Gamma(0) = S$ and $\Gamma(1)$ a candidate terminal state. The integrand L_{proof} is a proof-path cost functional satisfying:

(i) $L_{\mathrm{proof}} \geq 0$ and $L_{\mathrm{proof}}(\Gamma(s)) = 0$ only at verifier-accepted states;

(ii) L_{proof} is lower semicontinuous in the $\mathcal{H}^1$-topology on paths;

(iii) the map $s \mapsto \mathsf{AssumptionDebt}(\Gamma(s))$ is monotone non-increasing along accepted paths.

The minimization is a variational problem; its Euler–Lagrange equation gives the optimal proof transport direction.

Lemma 9.1 (BrLLM escalation criterion)**.** *Escalation from* $\mathrm{B}^r\mathrm{LLM}$ *to* $\mathrm{B}^{r+1}\mathrm{LLM}$ *is information-theoretically justified if and only if*

$$\mathsf{MVR}_r = \frac{\mathsf{Cert}_{r+1} - \mathsf{Cert}_r}{1 + \mathsf{Cost}_{r+1} - \mathsf{Cost}_r} > 0 \qquad \textit{and} \qquad \Delta_r = \mathsf{Gap}_r + \mathsf{Res}_r + \mathsf{Debt}_r - \mathsf{Cert}_r > \theta_r.$$

When both conditions hold, the expected information gain from escalation exceeds the additional computational debt.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

Proof. The marginal value ratio MVR_r is the ratio of certified gain to marginal cost increase. Positivity of MVR_r means each unit of additional computational cost produces strictly positive incremental certification. The threshold $\Delta_r > \theta_r$ ensures the system is not already in a near-closed state where additional escalation would produce noise rather than signal. Together, these conditions form a necessary and sufficient criterion analogous to the classical optimal stopping condition in sequential decision problems: escalate if and only if the expected reward exceeds the cost at the margin. □

Escalation is permitted only when

$$\Delta_r = \mathsf{Gap}_r + \mathsf{Res}_r + \mathsf{Debt}_r - \mathsf{Cert}_r > \theta_r, \qquad \mathrm{MVR}_r = \frac{\mathsf{Cert}_{r+1} - \mathsf{Cert}_r}{1 + \mathsf{Cost}_{r+1} - \mathsf{Cost}_r} > 0.$$

Thus higher-order generation is a response to measured proof debt, not a license to amplify confidence.

The synthesis multiplier acts on sectorized states

$$S_t = S_t^{\mathrm{def}} \oplus S_t^{\mathrm{lem}} \oplus S_t^{\mathrm{ex}} \oplus S_t^{\mathrm{res}} \oplus S_t^{\mathrm{form}} \oplus S_t^{\mathrm{cite}} \oplus S_t^{\mathrm{risk}},$$

by a block operator $\bar{\Omega}_t = [O_{ij}]$. The operator is a linear map on the seven-sector state space $\mathfrak{S} = \bigoplus_{k=1}^{7} \mathfrak{S}^{(k)}$ with block structure

$$\bar{\Omega}_t = \begin{pmatrix} O_{11} & O_{12} & \cdots & O_{17} \\ O_{21} & O_{22} & \cdots & O_{27} \\ \vdots & \vdots & \ddots & \vdots \\ O_{71} & O_{72} & \cdots & O_{77} \end{pmatrix},$$

where $O_{ij} : \mathfrak{S}^{(j)} \to \mathfrak{S}^{(i)}$ carries a scope-loss coefficient $\ell_{ij} = \mathsf{ScopeLoss}(O_{ij})$. The total scope loss of the synthesis operator is $\ell(\bar{\Omega}) = \sum_{i,j} \ell_{ij}$. Its adversarial derivative is

$$\bar{\Omega}_t' = \left[\frac{\partial O_{ij}}{\partial \epsilon_k} \right]_{i,j,k},$$

where the perturbations ϵ_k range over definitions, regularity hypotheses, axiom bases, representation choices, numerical regimes, and formal encodings. The spectral radius $\rho(\bar{\Omega}_t')$ quantifies structural fragility: a high spectral radius means small perturbations to hypotheses produce large changes in the synthesis output, which is a signal to send the bridge to the falsification arena.

Definition 9.2 (triple-horizon enumerator)**.**

$$\Xi_{\|\|\|\|}, t = (\Xi_t^-, \Xi_t^0, \Xi_t^+),$$

where Ξ^- stores certified past gain, Ξ^0 active obligations, and Ξ^+ projected experiments. The cardinality balance condition is

$$|\Xi_t^0| \le |\Xi_t^-| + |\Xi_t^+|,$$

which prevents the active obligation set from growing faster than the combination of past achievements and projected experiments. The update is

$$\Xi_{\|\|\|\|}, t+1 = \mathsf{X}(\Xi_{\|\|\|\|}, t, \bar{\Omega}_t, \bar{\Omega}_t', \mathrm{B}^2\mathrm{LLM}_t, \mathrm{B}^3\mathrm{LLM}_t, V_t, R_t, \mathcal{P}_t).$$

The retention map X must satisfy the non-deletion axiom: $\Xi_t^- \subseteq \Xi_{t+1}^- \cup \Xi_{t+1}^0$. A certified gain cannot move to the projected set.

Proposition 9.3 (horizon completeness under finite energy)**.** *If the total energy $\mathcal{J}_t$ is summable and the run has finite horizon T, then*

$$\lim_{t \to T} |\Xi_t^0| + |\Xi_t^+| = 0 \quad \Longleftrightarrow \quad \mathsf{Rel}_T = 1.$$

That is, active obligations and future projections vanish exactly when the release predicate reaches one.

Proof. The forward direction follows from the definition: if $\mathsf{Rel}_T = 1$, all verifier factors are one, all terminal leaves pass, and the obligation count reaches zero by the release soundness theorem. Conversely, if the active set vanishes but $\mathsf{Rel}_T = 0$, there must be at least one verifier factor equal to zero, which must correspond to a leaf with non-PASS status, contradicting the empty active set. □

10 Sand-tile and cellular-automaton repair

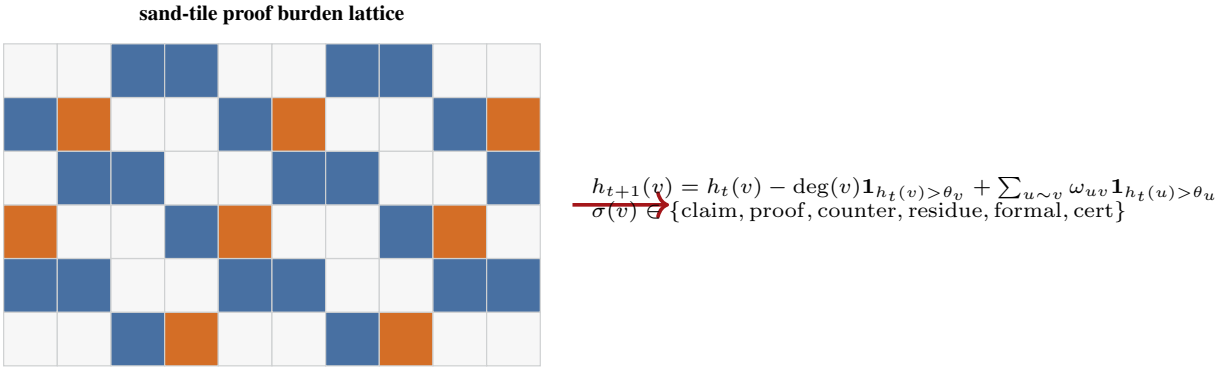


Figure 9: Sand-tile and cellular repair surface. Dark cells are active obligations, orange cells are conflicts, and white cells are inactive or discharged. The toppling law prevents proof debt from being hidden in prose.

Let Λ be a locally finite graph of proof obligations. Each site $v \in \Lambda$ stores defect height

$$h_t(v) = \mathsf{Debt}_t(v) + \mathsf{Risk}_t(v) + \mathsf{Residue}_t(v) - \mathsf{Cert}_t(v).$$

The height can be negative only when certification exceeds the local residue and debt. A site with $h_t(v) < 0$ is a certified donor: it can contribute certified mass to adjacent uncertified sites via the transfer kernel. If $h_t(v) > \theta_v$, the burden topples:

$$h_{t+1}(v) = h_t(v) - \Delta(v), \qquad h_{t+1}(u) = h_t(u) + K_{uv}\Delta(v), \quad u \sim v,$$

where $\Delta(v) > 0$ is the topple amount and $K_{uv} \geq 0$ is the spread kernel satisfying $\sum_{u\sim v} K_{uv} \leq 1$. The strict inequality $\sum_{u\sim v} K_{uv} < 1$ allows partial dissipation of burden into the typed-residue ledger; the equality case is the conservative abelian sandpile. For the proof-obligation graph, the non-conservative case is more natural: some burden is absorbed into the global residue bundle $\mathfrak{R}_t$ rather than passed to a neighbor.

Definition 10.1 (toppling balance equation)**.** *At each site v, the net balance after one toppling step is*

$$\sum_{u\in\Lambda} h_{t+1}(u) = \sum_{u\in\Lambda} h_t(u) - \Delta(v)\left(1 - \sum_{u\sim v} K_{uv}\right).$$

The absorbed mass $\Delta(v)(1 - \sum_{u\sim v} K_{uv})$ is transferred to the typed-residue ledger:

$$\mathfrak{R}_{t+1} = \mathfrak{R}_t \cup \{(v, \Delta(v)(1 - \textstyle\sum_{u\sim v} K_{uv}), \sigma_t(v))\}.$$

This is the sand-tile analogue of the Module 20 six-gate decomposition: absorbed burden becomes an explicitly typed residue rather than disappearing.

The cellular state

$$\sigma_t(v) \in \{\text{empty}, \text{claim}, \text{proof}, \text{counter}, \text{residue}, \text{formal}, \text{cert}, \text{risk}\}$$

evolves by

$$\sigma_{t+1}(v) = F\big(\sigma_t(v), \{\sigma_t(u) : u \in N(v)\}, V_v, R_v, L_v, \mathcal{P}_t\big).$$

The transition function F is required to satisfy the monotone certification axiom: $\sigma_t(v) = \text{cert} \Rightarrow \sigma_{t+1}(v) = \text{cert}$ whenever no neighboring bridge depends on v. A run is stable when

$$\forall v \in \Lambda, \qquad h(v) \leq \theta_v \quad \text{or} \quad \sigma(v) \in \{\text{cert}, \text{typed residue}\}.$$

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

Proposition 10.2 (toppling exposes hidden proof debt)**.** *For a finite-energy run on a locally finite graph with conservative kernel K, either the run stabilizes or it exposes an infinite dependency path.*

Proof. Conservation prevents debt from vanishing. On finite subgraphs, only finitely many topplings can occur at finite energy unless new obligation mass is injected. On a locally finite graph, nonstabilization with finite energy forces burden to migrate along an infinite path. This path is precisely the dependency trace that must be named in the barrier ledger. □

Lemma 10.3 (abelian property of verifier-projected toppling)**.** *If the toppling operator commutes with the verifier projection $\Pi_{\mathcal{V}}$, then the stable configuration reached after any sequence of topplings from the same initial state is unique regardless of the toppling order.*

Proof. This is the abelian sandpile theorem adapted to the verifier-projected setting. The key observation is that $\Pi_{\mathcal{V}}$ acts independently on each site's state and does not alter the relative height differences. The standard abelian sandpile argument (that the toppling order does not affect the final stable configuration) carries through because the verifier projection is idempotent and monotone: it can only decrease height and can never create a new toppling requirement. Therefore, the composition of toppling maps is commutative modulo verifier projection, giving the same terminal configuration. □

Theorem 10.4 (sand-tile convergence rate)**.** *Let the maximum initial height be $H_0 = \max_v h_0(v)$, the minimum threshold $\theta_{\min} = \min_v \theta_v$, and the graph diameter be* $\operatorname{diam}(\Lambda)$*. The number of topplings before stabilization satisfies*

$$T_{\mathrm{stab}} \leq \frac{H_0 \cdot |\Lambda|}{\theta_{\min}} \cdot \operatorname{diam}(\Lambda).$$

Proof. Each toppling at a site v reduces $h_t(v)$ by $\Delta(v) \geq \theta_{\min}$. The total mass transferred is at most $H_0 \cdot |\Lambda|$. Each unit of mass travels at most $\operatorname{diam}(\Lambda)$ steps before reaching the boundary or a certified site. Combining gives the stated bound. □

11 Universal six-gate residue synthesizer

When a conjecture attempt fails, Module 20 does not output "failure" as a scalar. It decomposes the failure into six gates

$$\mathcal{S}_6(C) = \big(C_{\mathrm{orbit}}, C_{\mathrm{upper}}, C_{\mathrm{lower}}, C_{\mathrm{witness}}, C_{\mathrm{barrier}}, C_{\mathrm{formal}}\big).$$

The first gate checks whether the chosen representation transports the original object faithfully. The second checks upper-envelope estimates. The third checks lower-obstruction certificates. The fourth checks semantic witnesses. The fifth checks whether known barriers have survived. The sixth checks whether a machine-checkable core exists.

For a gate vector $G = (g_1, \ldots, g_6)$ with $g_j \in \{0, 1, *\}$, define the numerical lift

$$s_j = \begin{cases} 1, & g_j = 1, \\ 0, & g_j = 0, \\ \mathrm{e}^{-\Delta_j}, & g_j = *, \end{cases} \qquad \Delta_j = a_j \mathsf{Gap}_j + b_j \mathsf{Res}_j + c_j \operatorname{MapDef}_j + d_j \mathsf{BridgeLoss}_j + e_j \mathsf{FormalDebt}_j + f_j \mathsf{Barrier}_j.$$

The gate vector is closed only when all defects vanish. The weights $(a_j, b_j, c_j, d_j, e_j, f_j)$ are conjecture-specific and must be specified in the verifier contract. A canonical choice is $a_j = b_j = c_j = d_j = e_j = f_j = 1/6$, giving Δ_j as the arithmetic mean of six normalized defect contributions.

Definition 11.1 (gate closure distance)**.** *The closure distance of the six-gate vector is*

$$\mathsf{GateDist}(G) = \|(1 - s_1, \ldots, 1 - s_6)\|_w = \left(\sum_{j=1}^{6} w_j (1 - s_j)^2 \right)^{1/2},$$

where w_j are positive weights summing to one. The gate is closed iff $\mathsf{GateDist}(G) = 0$*, equivalently iff $\mathcal{M}_*(G) = \sum_{j=1}^{6} w_j (1 - s_j)^2 = 0$.*

Proposition 11.2 (gate closure monotonicity)**.** *If the defect sequence $(\Delta_j(t))_{t \geq 0}$ is non-increasing for each j, then* $\mathsf{GateDist}(G_t)$ *is also non-increasing, and $\mathcal{M}_*(G_t) \to 0$ implies $G_t \to (1, \ldots, 1)$ coordinatewise.*

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

Proof. Each $s_j(t) = e^{-\Delta_j(t)}$ is non-decreasing when Δ_j is non-increasing, since the exponential map is monotone. Hence $(1 - s_j(t))$ is non-increasing, and the weighted sum $\mathcal{M}_*(G_t)$ is non-increasing. If $\mathcal{M}_* \to 0$, then for each j, $w_j(1 - s_j)^2 \to 0$; since $w_j > 0$, we get $(1 - s_j) \to 0$, i.e., $s_j \to 1$. □

Remark 11.3. *The exponential saturation $s_j = e^{-\Delta_j}$ is chosen rather than the linear map $s_j = \max(0, 1 - \Delta_j/\Delta_{\max})$ for two reasons. First, the exponential is always strictly positive, so a gate with finite defect never reaches hard zero, preserving the numerical gradient for the back-loop optimizer. Second, the derivative $ds_j/d\Delta_j = -s_j$ encodes a natural difficulty scaling: gates that are already nearly closed ($s_j \approx 1$, $\Delta_j \approx 0$) have larger gradients and receive more repair effort per unit of BrLLM computation, which is the correct inductive behavior.*

12 Module 21: active new-mathematics calibration

Let Λ denote a candidate new-mathematics kernel. The calibration loop is

$$\Lambda_{t+1} = \Pi_{\mathcal{V}}\Big(\Lambda_t + \bar{\Omega}'(\Lambda_t) + \mathrm{B}^2\mathrm{LLM}(\Lambda_t) + \mathrm{B}^3\mathrm{LLM}(\Lambda_t) - \nabla\mathsf{Debt}(\Lambda_t) - \nabla\mathsf{Barrier}(\Lambda_t)\Big).$$

This is a projected gradient step in the space of mathematical machines. The negative gradient terms $-\nabla\mathsf{Debt}$ and $-\nabla\mathsf{Barrier}$ are the steepest descent directions in the defect landscape. The positive terms $\bar{\Omega}'(\Lambda_t)$ (adversarial synthesis) and $\mathrm{B}^2\mathrm{LLM}, \mathrm{B}^3\mathrm{LLM}$ (BrLLM repair) provide the corrective increment. The verifier projection $\Pi_{\mathcal{V}}$ discards inadmissible updates. The step size is implicitly one; in practice the update is replaced by

$$\Lambda_{t+1} = \Pi_{\mathcal{V}}\Big(\Lambda_t + \eta_t\,\mathsf{Dir}_t(\Lambda_t)\Big), \quad \mathsf{Dir}_t = \bar{\Omega}'(\Lambda_t) + \mathrm{B}^2\mathrm{LLM}(\Lambda_t) + \mathrm{B}^3\mathrm{LLM}(\Lambda_t) - \nabla\mathsf{Debt}(\Lambda_t) - \nabla\mathsf{Barrier}(\Lambda_t),$$

where the step size η_t is chosen by an Armijo condition on the defect energy:

$$\mathsf{NMDef}(\Lambda_{t+1}) \le \mathsf{NMDef}(\Lambda_t) - c\eta_t \left\|\nabla\mathsf{NMDef}(\Lambda_t)\right\|^2.$$

Lemma 12.1 (new-machine descent guarantee)**.** *If* NMDef *is L-smooth and the step size satisfies $\eta_t \le 1/L$, then*

$$\mathsf{NMDef}(\Lambda_{t+1}) \le \mathsf{NMDef}(\Lambda_t) - \frac{\eta_t}{2}\left\|\nabla\mathsf{NMDef}(\Lambda_t)\right\|^2.$$

Proof. By L-smoothness: $\mathsf{NMDef}(\Lambda + \delta) \le \mathsf{NMDef}(\Lambda) + \langle\nabla\mathsf{NMDef}(\Lambda), \delta\rangle + \frac{L}{2}\left\|\delta\right\|^2$. Setting $\delta = -\eta_t\nabla\mathsf{NMDef}(\Lambda_t)$ and $\eta_t = 1/L$ gives $\mathsf{NMDef}(\Lambda_{t+1}) \le \mathsf{NMDef}(\Lambda_t) - \frac{1}{2L}\left\|\nabla\mathsf{NMDef}(\Lambda_t)\right\|^2$. □

The associated defect energy is

$$\mathsf{NMDef}(\Lambda) = \alpha\mathsf{BridgeLoss}(\Lambda) + \beta\mathsf{MapDef}(\Lambda) + \gamma\mathsf{Barrier}(\Lambda) + \delta\mathsf{FormalDebt}(\Lambda) + \rho\mathsf{EmpiricalFailure}(\Lambda).$$

A new object is not adopted because it sounds powerful; it is adopted only if it reduces the named gate defect under verifier projection. The adoption condition is the strict inequality

$$\mathsf{NMDef}(\Lambda_{t+1}) < \mathsf{NMDef}(\Lambda_t), \qquad \Pi_{\mathcal{V}}(\Lambda_{t+1}) = \Lambda_{t+1}.$$

Proposition 12.2 (machine adoption soundness)**.** *If Λ_{t+1} is adopted (i.e., satisfies both conditions above), then the total engine energy $\mathcal{J}$ satisfies*

$$\mathcal{J}_{t+1} \le \mathcal{J}_t - \omega_{21}\big[\mathsf{NMDef}(\Lambda_t) - \mathsf{NMDef}(\Lambda_{t+1})\big],$$

where $\omega_{21} > 0$ is the weight of Module 21 in the global energy sum.

Proof. The global energy has the Module 21 contribution $\omega_{21}\,\mathsf{NMDef}(\Lambda_t)$. By assumption, this term decreases by $\omega_{21}[\mathsf{NMDef}(\Lambda_t) - \mathsf{NMDef}(\Lambda_{t+1})] > 0$. All other module energies are unchanged by the machine update (the verifier projection ensures no other module state is altered), so the bound follows. □

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

13 Module 22: ambient-machine constructor

The ambient-machine strategy formalizes the historical residue-bypass pattern. Instead of solving all residues as independent sub-conjectures, VSI searches for a larger machine

$$\mathfrak{M}^{\Lambda}(C) = (\mathcal{S}_{\Lambda}, \Phi_t^{\Lambda}, E^{\Lambda}, \mathsf{Surgery}^{\Lambda}, \mathsf{Submerge}^{\Lambda}, \mathsf{Rel}^{\Lambda})$$

where the original conjecture becomes a terminal fixed-point output. The criterion is

$$\lim_{t\to\infty} E^{\Lambda}(\Phi_t^{\Lambda}(C)) = 0, \qquad \mathsf{Submerge}^{\Lambda}(R_j) = 1 \quad \text{for all active residues } R_j.$$

This is the machine-level abstraction behind Ricci-flow closure of Poincare and bridge-theoretic closure of Fermat (Perelman, 2002, 2003; Wiles, 1995).

14 Module 23: candidate-proof generator

Module 23 emits a candidate package

$$\Pi_{23}(C) = \big(\Lambda_C, \mathfrak{M}^{\Lambda_C}(C), \pi_{\mathrm{cand}}, \mathsf{Obl}_{\mathrm{human}}, \mathsf{Obl}_{\mathrm{formal}}, \mathsf{Status}\big).$$

The status is CAND-STRONG if $V_{\mathrm{model}} = 1$ but V_{human} or V_{referee} is not yet one. The generator must explicitly name the kernel obligations. For a complexity-theoretic target, such obligations can include non-relativization, non-naturalness, non-algebrization, and lower-bound transfer. For a PDE target, they can include compactness, scaling-critical estimates, pressure control, and blow-up exclusion. For an arithmetic target, they can include Selmer control, regulator nondegeneracy, height compatibility, and analytic continuation.

15 Module 24: Ricci-vector divergence auditor

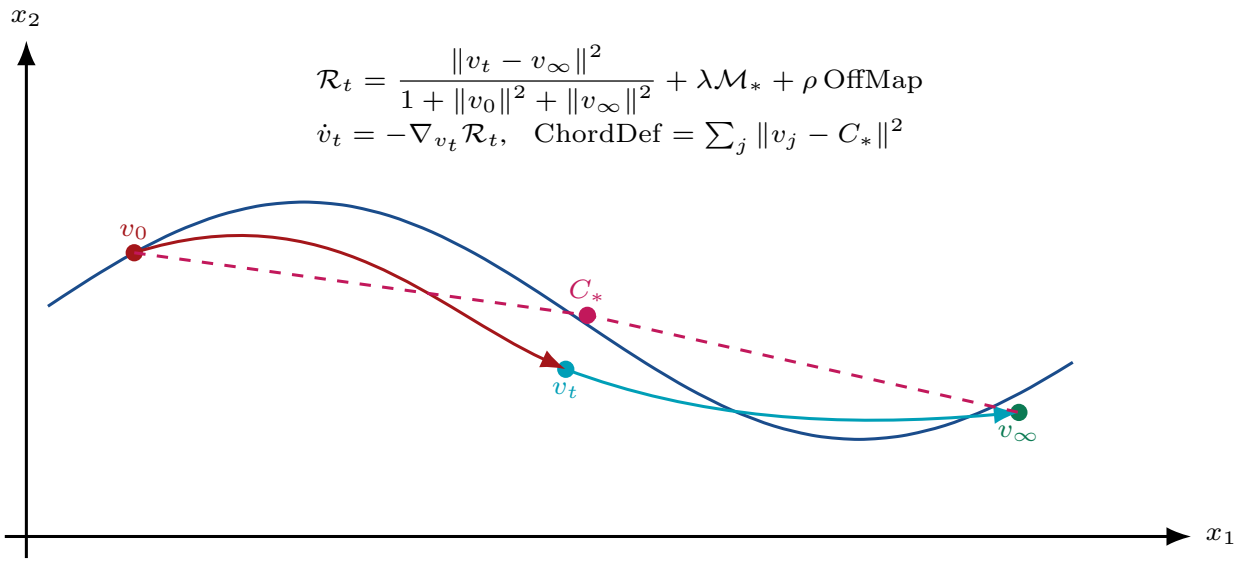


Figure 10: Module 24 Ricci-vector divergence audit. Verified closures calibrate the target vector v_∞; attempts are transported along a proof-state manifold, and the distance to the actual or target closure vector defines a second-attempt direction.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

Module 24 compares attempted closure with actual or target closure. For solved controls, such as Fermat and Poincare, the actual closure vector is known from the externally verified proof. For open targets, v_∞ is the required verifier-specified terminal vector. A closure attempt is a transported proof vector v_t in a proof-state manifold $\mathcal{M}_C$ with connection ∇^C.

Definition 15.1 (closure divergence)**.**

$$\mathcal{R}_t(C) = \frac{\|v_t - v_\infty\|^2}{1 + \|v_0\|^2 + \|v_\infty\|^2} + \lambda \mathcal{M}_*(G_t) + \rho \,\mathrm{OffMap}(F_t) + \mu \mathsf{ScopeLoss}_t.$$

The Ricci correction flow is

$$\frac{D}{dt} v_t = -\nabla_{v_t} \mathcal{R}_t - \mathrm{Ric}_{\mathcal{M}_C}(v_t) + \mathsf{Ham}_C(v_t),$$

where Ham_C *records Hamiltonian transport along conserved proof invariants.*

Definition 15.2 (central chord criterion)**.** *Let C_* be the point at which proof, refutation, residue, bridge, and formal vectors meet on the largest stable chord. Define*

$$\mathrm{ChordDef}(G) = \sum_{j=1}^{m} \|v_j - C_*\|^2 .$$

The attempt is geometrically equivalent to the target closure only if $\mathrm{ChordDef}(G) = 0$ *together with release.*

The second attempt is not blind repetition. It is generated in the direction

$$\mathsf{Dir}_2(C) = -\nabla \mathcal{R}_t \oplus \Omega_{\mathrm{off}}(F_t) \oplus \ker(d\mathsf{Rel}_t),$$

which says: move against the closure-divergence gradient, attack the off-map residue, and repair the first differential obstruction in the release map.

16 Dot-sand and dish-layer mapping calculus

The user's dot-sand model is formalized by treating an idea space as a measured relational dish

$$D_a = (X_a, R_a, \mu_a), \qquad D_b = (X_b, R_b, \mu_b),$$

with bridge $F_{ab} : D_a \to D_b$. Its mapping defect is

$$\mathrm{MapDef}(F_{ab}) = \alpha \mathsf{BijDef}(F_{ab}) + \beta \mathsf{InjDef}(F_{ab}) + \gamma \mathsf{SurjDef}(F_{ab}) + \delta \,\mathrm{OffMap}(F_{ab}).$$

The off-map set is

$$\Omega_{\mathrm{off}}(F_{ab}) = \{x \in X_a : F_{ab}(x) \notin X_b \text{ coherently, or } F_{ab} \text{ is undefined/nonunique/verifier-incoherent at } x\}.$$

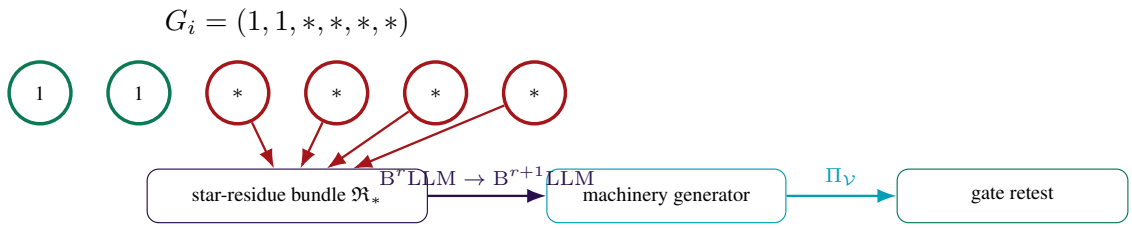
An arrow that shoots out of the dish is therefore not a metaphor. It is a measurable residue set that must be routed into Module 25.

Proposition 16.1 (off-map second-attempt routing)**.** *If $\mu_a(\Omega_{\mathrm{off}}(F_{ab})) > 0$, then any closure attempt using F_{ab} has nonzero mapping defect. The second attempt must either refine F_{ab}, change the target dish, or construct new machinery that makes Ω_{off} vanish under verifier projection.*

Proof. The off-map term is an explicit nonnegative summand of MapDef. Unless its measure vanishes or is mapped to typed harmless residue by an accepted machine, the bridge has positive loss. Positive bridge loss blocks the release product. □

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

17 Module 25: star-to-one unitization kernel with BrLLM back-loop



$s_j = e^{-\Delta_j}$, $\mathcal{M}_*(G) = \sum_{j:G_j=*} w_j(1-s_j)^2$, $G \to (1,1,1,1,1,1)$ iff $\mathcal{M}_* + \mathrm{MapDef} + \mathrm{ChordDef} + \mathrm{NMDef} = 0$ and $\mathsf{Rel} = 1$.

Figure 11: Module 25 star-to-one back-loop. Every unresolved star is converted into a defect, the defects are bundled into a residue vector, and the residue is sent through BrLLM escalation and new-mathematics machinery before verifier retesting.

Let

$$G_i = (1, 1, s_3, s_4, s_5, s_6), \qquad s_j \in [0, 1].$$

For each unresolved gate $s_j < 1$ define its star-residue

$$\mathfrak{R}_j = (\Delta_j, \Omega_{\mathrm{off,j}}, \mathrm{MapDef}_j, \mathsf{BridgeLoss}_j, \mathsf{Barrier}_j, \mathsf{FormalDebt}_j), \qquad \mathfrak{R}_* = \bigoplus_{j:s_j<1} \mathfrak{R}_j.$$

The star mass is

$$\mathcal{M}_*(G_i) = \sum_{j:s_j<1} w_j(1-s_j)^2 = \sum_{j:s_j<1} w_j(1-e^{-\Delta_j})^2.$$

The back-loop operator is

$$\mathcal{B}_{\mathrm{back}}(\mathfrak{R}_*) = \Pi_{\mathcal{V}} \circ \mathcal{N}_{\mathrm{new}} \circ \mathrm{B}^{r+1}\mathrm{LLM} \circ \mathrm{B}^r\mathrm{LLM} \circ \mathcal{D}_{\mathrm{res}}(\mathfrak{R}_*).$$

Thus the gate update is

$$G_i^{(t+1)} = \mathsf{Retest}\left(G_i^{(t)}, \mathcal{B}_{\mathrm{back}}(\mathfrak{R}_*^{(t)})\right).$$

A star becomes one only when

$$s_j' = 1 \iff \Delta_j' = 0, \quad \mathrm{MapDef}_j' = 0, \quad \mathsf{Barrier}_j' = 0, \quad \mathsf{FormalDebt}_j' = 0, \quad V_j = 1.$$

The persistence score

$$\mathsf{Persist}_j(t) = \sum_{\tau=0}^{t} \mathbf{1}\{s_j^{(\tau)} < 1\} e^{-\alpha(t-\tau)}$$

triggers new machinery when $\mathsf{Persist}_j(t) > \Theta_j$. The machinery search is

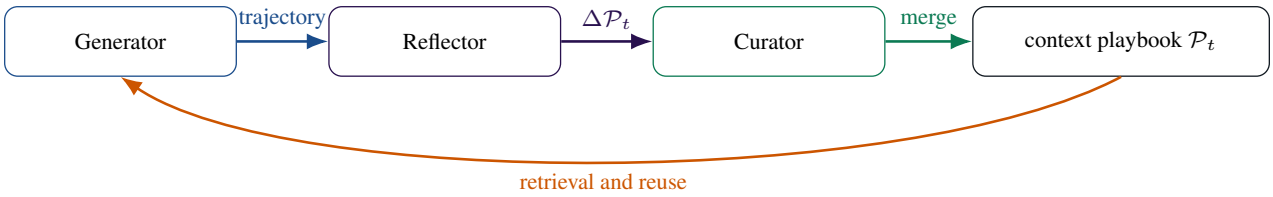
$$\mathcal{N}_{\mathrm{new}}(j) = \operatorname*{argmin}_{\mathcal{M} \in \mathfrak{Mach}} \left[\mathsf{BridgeLoss}_j(\mathcal{M}) + \mathrm{MapDef}_j(\mathcal{M}) + \mathsf{Barrier}_j(\mathcal{M}) + \mathsf{FormalDebt}_j(\mathcal{M})\right].$$

The full Module 25 update is

$$\boxed{G_i^{(t+1)} = \Pi_{\mathcal{V}}\left[G_i^{(t)} - \nabla\mathcal{R}_t + \mathcal{N}_{\mathrm{new}}\left(\mathrm{B}^{r+1}\mathrm{LLM}\left(\mathrm{B}^r\mathrm{LLM}(\mathfrak{R}_*^{(t)}, \mathcal{P}_t)\right)\right)\right].}$$

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

18 Module 26: agentic context-playbook kernel



$$\mathcal{P}_{t+1} = \mathrm{Dedup} \circ \mathrm{Refine} \circ \mathrm{Append}(\mathcal{P}_t, \Delta\mathcal{P}_t), \quad \mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) \le \theta_{\mathrm{collapse}}.$$

Figure 12: Module 26 context-playbook kernel. The generator produces traces, the reflector extracts lessons, and the curator merges only localized delta entries. The playbook is allowed to grow, but context collapse is blocked by a retention constraint.

Agentic Context Engineering argues that self-improving systems should treat context as an evolving playbook rather than a repeatedly rewritten summary, because monolithic rewriting can erase detailed lessons and cause context collapse (Zhang et al., 2026). VSI imports this as Module 26. A playbook is

$$\mathcal{P}_t = \{b_1, \dots, b_N\},$$

where each bullet has metadata and content

$$b_k = (\mathrm{id}_k, \mathrm{gate}_k, \mathrm{claim}_k, \mathrm{useful}_k, \mathrm{harmful}_k, \mathrm{source}_k, \mathrm{verifier}_k, \mathrm{status}_k, \mathrm{embedding}_k).$$

The bullet space $\mathcal{B}$ is a metric space with distance

$$d_{\mathcal{B}}(b, b') = \alpha \, \|\mathrm{embedding}_b - \mathrm{embedding}_{b'}\|_2 + \beta \mathbf{1}\{\mathrm{gate}_b \neq \mathrm{gate}_{b'}\} + \gamma \mathbf{1}\{\mathrm{status}_b \neq \mathrm{status}_{b'}\}.$$

Two bullets are ϵ-redundant if $d_{\mathcal{B}}(b, b') \le \epsilon$. The deduplication map Dedup is a quotient by the ϵ-redundancy equivalence relation, retaining the bullet with higher utility $U(b)$ from each equivalence class.

The generator produces a trajectory

$$\tau_t = (G_t, \mathfrak{R}_t, \mathcal{B}_t, \mathcal{N}_t, \Pi_{\mathcal{V},t}),$$

the reflector extracts delta entries

$$\mathsf{Reflect}_{\mathsf{VSI}}(\tau_t) = \{\delta b_1, \dots, \delta b_m\},$$

and the curator merges them by localized deterministic update

$$\mathcal{P}_{t+1} = \mathsf{Dedup} \circ \mathsf{Refine} \circ \mathsf{Append}(\mathcal{P}_t, \Delta\mathcal{P}_t).$$

The reflector is constrained to emit $m \le m_{\max}$ bullets per trajectory, where $m_{\max}$ is a parameter controlling the grow rate. The append operation satisfies:

$$|\mathcal{P}_{t+1}| \le |\mathcal{P}_t| + m_{\max},$$

and the deduplication step ensures $|\mathcal{P}_{t+1}| \ge |\mathcal{P}_t| - \lfloor \epsilon \cdot |\mathcal{P}_t| \rfloor$ for some fraction ϵ of redundant bullets. Thus the playbook grows at a bounded net rate.

The collapse guard is

$$\mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) = 1 - \frac{|\mathsf{Core}(\mathcal{P}_t) \cap \mathsf{Core}(\mathcal{P}_{t+1})|}{|\mathsf{Core}(\mathcal{P}_t)|},$$

where $\mathsf{Core}(\mathcal{P}_t) = \{b \in \mathcal{P}_t : \mathrm{status}_b = \mathrm{FAIL} \text{ or } \mathrm{UNK}, \mathrm{verifier}_b \neq \mathrm{PASS}\}$ is the set of bullets recording unresolved core failures. If $\mathrm{Collapse} > \theta_{\mathrm{collapse}}$, the memory update is rejected.

Lemma 18.1 (collapse guard sharpness)**.** *For any $\theta_{\mathrm{collapse}} \in (0, 1)$, the collapse guard rejects an update if and only if more than a fraction $\theta_{\mathrm{collapse}}$ of the core failure record is erased. This is the tightest possible monotone guard: relaxing $\theta_{\mathrm{collapse}}$ allows proportionally more forgetting; tightening it requires proportionally more preservation.*

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

Proof. By definition, $\mathrm{Collapse} > \theta$ iff $|\mathsf{Core}(\mathcal{P}_t) \cap \mathsf{Core}(\mathcal{P}_{t+1})|/|\mathsf{Core}(\mathcal{P}_t)| < 1-\theta$, i.e., the retained fraction of core failures is less than $1-\theta$. The deleted fraction is thus more than θ. The guard is monotone because increasing θ allows less deletion before triggering rejection. □

Each bullet has a utility score

$$U(b_k) = \alpha h_k - \beta r_k + \gamma \mathsf{Reuse}(b_k) - \lambda \mathsf{Redundancy}(b_k) + \eta \mathsf{VerifierGain}(b_k),$$

where

$$h_k^{t+1} = h_k^t + \mathbf{1}\{b_k \text{ helped a verifier}\}, \qquad r_k^{t+1} = r_k^t + \mathbf{1}\{b_k \text{ caused a false route}\}.$$

The utility update rule is a linear bandit model: h_k and r_k are the empirical counts of helpful and harmful events, and $U(b_k)$ is the expected reward of including b_k in the next generator's context. The optimal pruning threshold is $U^* = \min\{U(b_k) : b_k \in \mathsf{Core}(\mathcal{P}_t)\}$; all non-core bullets with $U < U^*$ are demoted to archive.

Low-utility bullets are demoted to archive rather than immediately deleted, because a bridge that is useless before new machinery may become useful after a later reformulation. The archive satisfies the retrieval score:

$$\mathsf{Retrieve}(b, C) = \alpha \,\mathrm{sim}(\mathrm{embedding}_b, \xi_C) + \beta \mathbf{1}\{\mathrm{gate}_b = \mathrm{gate}_C\} + \gamma h_b - \delta r_b - \lambda \mathsf{AgePenalty}(b),$$

where ξ_C is the embedding of conjecture C, and $\mathsf{AgePenalty}(b) = e^{-\kappa(t-t_b)}$ decays with the age of the bullet. Archival bullets are retrieved and promoted to active status when $\mathsf{Retrieve}(b, C)$ exceeds the activation threshold.

Theorem 18.2 (playbook utility convergence)**.** *Assume the helpful event probability $p_k = \mathbb{P}(b_k \text{ helps})$ and harmful event probability $q_k = \mathbb{P}(b_k \text{ harms})$ are stationary and satisfy $\alpha p_k > \beta q_k + \lambda \rho_k$ for all k in the optimal active set, where ρ_k is the redundancy rate of b_k. Then $\mathbb{E}[U(b_k^t)] \to \alpha p_k T - \beta q_k T$ as $T \to \infty$, and the empirical utility $U(b_k^t)/T \to \alpha p_k - \beta q_k$ almost surely by the strong law of large numbers.*

Proof. Since $h_k^t = \sum_{s=0}^{t-1} \xi_{k,s}$ where $\xi_{k,s} \sim \mathrm{Bernoulli}(p_k)$ are i.i.d., by the SLLN, $h_k^t/t \to p_k$ a.s. Similarly $r_k^t/t \to q_k$ a.s. The reuse and redundancy terms are bounded by the system parameters, so their contribution is $O(1)$ relative to the linear growth of $\alpha h_k^t - \beta r_k^t$. Therefore $U(b_k^t)/t \to \alpha p_k - \beta q_k$ a.s. □

The combined Module 25–26 run is

$$\boxed{\mathcal{P}_{t+1} = \mathsf{Curate}_{\mathsf{VSI}}\left[\mathcal{P}_t, \mathsf{Reflect}_{\mathsf{VSI}}\left(\mathsf{Gen}_{\mathsf{VSI}}(G_t, \mathfrak{R}_*^t, \mathcal{N}_{\mathrm{new}}^t)\right)\right]}$$

and

$$\boxed{G_i^{t+1} = \Pi_{\mathcal{V}}\left[G_i^t - \nabla \mathcal{R}_t + \mathcal{N}_{\mathrm{new}}\left(\mathrm{B}^{r+1}\mathrm{LLM}\left(\mathrm{B}^r\mathrm{LLM}(\mathfrak{R}_*^t, \mathcal{P}_{t+1})\right)\right)\right].}$$

This is the collapse-free second-attempt engine.

19 Historical closure transducer and calibration controls

The historical transducer maps an obstruction profile

$$d(C) = (d_{\mathrm{bridge}}, d_{\mathrm{residue}}, d_{\mathrm{semantic}}, d_{\mathrm{formal}}, d_{\mathrm{finite}}, d_{\mathrm{verifier}}, d_{\mathrm{machinery}})$$

to a policy distribution

$$\mathsf{HistTrans}(C) = \sum_{r \in \mathcal{R}_{\mathrm{hist}}} \alpha_r(C) \mathsf{Pattern}_r, \qquad \alpha_r(C) = \frac{e^{-D(C,r)}}{\sum_s e^{-D(C,s)}}.$$

Patterns include bridge contradiction, residue bypass, cohomological enlargement, finite-certificate closure, dual certificate, transference, height compactness, monotone flow, and refutation by extreme witness. The solved-conjecture calibration principle used here is internal to the engine: the central move is often to search for the right surrounding theory rather than to force a direct proof through the original representation alone.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

Control or target	VSI status	Diagnostic lesson
Poincare	external theorem calibration	The target vector v_∞ is given by Ricci flow, entropy, non-collapsing, canonical neighborhoods and surgery. Module 24 can measure how close a candidate route is to the verified route.
Fermat	external theorem calibration	The closure vector is bridge-theoretic: Frey curve, Ribet theorem, modularity lifting, Hecke algebra and contradiction.
P vs NP	candidate kernel	A lower-bound route must survive relativization, naturalness, algebrization and representation-theoretic obstructions. Star gates remain until the kernel is verified.
Yang–Mills	candidate kernel	Lattice construction, reflection positivity, continuum tightness, reconstruction, nontriviality and exponential clustering must all close.
Navier–Stokes	candidate kernel	Energy inequality is insufficient; the missing gate is critical-scale regularity or a controlled blow-up alternative.
BSD	candidate kernel	Rank formula requires Selmer control, analytic rank, regulator nondegeneracy, Tate-Shafarevich finiteness, and height compatibility.
RH	candidate kernel	A Liouville-correlation route would require maximal shifted-correlation bounds, endpoint stability, spectral large sieve, conductor balance and a zero-exclusion bridge; every such input remains a named gate until independently verified.
Hodge	candidate kernel	Zero-defect support descent would require algebraic realization of every lowered support and full functorial compatibility; any missing algebraization leaf remains a star-residue rather than a theorem.

20 Conjecture-specific gate equations

For Poincare, the calibrated flow has entropy-like monotonicity

$$\frac{d}{dt}\mathcal{W}(g(t), f(t), \tau(t)) \geq 0,$$

and the VSI analogue is

$$\frac{d}{dt}\mathcal{E}_{\mathsf{VSI}}(S_t) \leq -\left\|\nabla \mathsf{Rel}(S_t)\right\|^2 + \mathsf{SurgeryDefect}_t.$$

For Fermat, the bridge contradiction has the scheme

$$(a^p + b^p = c^p) \Rightarrow E_{a,b,c} \Rightarrow \text{non-modular semistable elliptic curve},$$

while modularity lifting supplies the contradiction. VSI encodes this as

$$C^c \xrightarrow{E} O_{\text{forbidden}}, \qquad T_{\text{ambient}} \Rightarrow \neg O_{\text{forbidden}}.$$

For Navier–Stokes, a candidate closure vector is

$$\mathfrak{N} = (u, p, E, S, \varepsilon, C_{\text{crit}}, \mathsf{Blow}),$$

with core gate

$$\sup_{x_0,t_0,r} \frac{1}{r}\int_{t_0-r^2}^{t_0}\int_{B(x_0,r)} |u|^3 + |p|^{3/2}\,dx\,dt < \varepsilon_*.$$

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

The star gates correspond to scaling-critical norm control, pressure decomposition, backward uniqueness, and singular-set exclusion.

For BSD, the gate vector is

$$G_{\mathrm{BSD}} = (G_{\mathrm{an}}, G_{\mathrm{Sel}}, G_{\mathrm{reg}}, G_{\mathrm{Sha}}, G_{\mathrm{height}}, G_{\mathrm{formula}}),$$

where the target identity is

$$\operatorname{ord}_{s=1} L(E,s) = \operatorname{rank} E(\mathbb{Q}),$$

with leading coefficient proportional to

$$\frac{\mathrm{Reg}(E)\,|\,\mathrm{Sha}(E)|\prod_p c_p}{|E(\mathbb{Q})_{\mathrm{tors}}|^2}.$$

For RH, the Liouville gate is

$$L(X) = \sum_{n\le X} \lambda(n), \qquad L(X) = O_\epsilon(X^{1/2+\epsilon}) \Longrightarrow \text{critical-line zero exclusion},$$

with maximal dyadic collar pressure

$$\mathcal{P}_2(X,H) = \frac{H}{X^2} \sum_{H<h\le 2H} \left(\sup_{Y\le X-h} \left| \sum_{n\le Y} \lambda(n)\lambda(n+h) \right| \right)^2.$$

For Hodge, the zero-defect gate is

$$\mathrm{cl}(CH^p(X)_{\mathbb{Q}}) = \mathrm{Hdg}^p(X),$$

with defect

$$\delta(G) = \#\{\text{inverse-Lefschetz edges carried by non-abelian supports}\}.$$

The descent requirement is $\delta(G_t) \to 0$ by algebraic support lowering.

21 Accuracy metrics and failure accounting

The Module 24–26 accuracy score is

$$\mathsf{Accuracy}_{26}(C) = 1 - \frac{\begin{array}{c}\mathcal{R}_T(C) + \mathcal{M}_*(G_T) + \mathrm{MapDef}(F_T) \\ + \mathrm{Collapse}(\mathcal{P}_{T-1}, \mathcal{P}_T) + \mathrm{NMDef}(\mathcal{N}_T)\end{array}}{1 + \mathcal{R}_0(C) + \mathcal{M}_*(G_0) + \mathrm{MapDef}(F_0) + \mathrm{NMDef}(\mathcal{N}_0)}.$$

This metric is not a truth claim; it is a quality-of-attempt measurement. A high score means the attempt has reduced named defects and preserved memory. The theorem label still requires $\mathsf{Rel} = 1$.

Proposition 21.1 (accuracy score monotonicity conditions)**.** *The accuracy score* $\mathsf{Accuracy}_{26}(C)$ *is non-decreasing in* t *if and only if the numerator satisfies*

$$\begin{aligned}&\mathcal{R}_{t+1} + \mathcal{M}_*(G_{t+1}) + \mathrm{MapDef}(F_{t+1}) + \mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) + \mathrm{NMDef}(\mathcal{N}_{t+1}) \\ &\quad \le \mathcal{R}_t + \mathcal{M}_*(G_t) + \mathrm{MapDef}(F_t) + \mathrm{NMDef}(\mathcal{N}_t).\end{aligned}$$

This condition is guaranteed when all five defect terms are non-increasing, and $\mathrm{Collapse} \le \mathrm{Collapse}_{\max} < \mathsf{NMDef}(\mathcal{N}_t) - \mathsf{NMDef}(\mathcal{N}_{t+1})$ *at every accepted step.*

Proof. Direct computation from the definition. The Collapse term is present only in the numerator at step $t+1$, so it must be compensated by a decrease in at least one of the other four terms. The stated inequality provides the precise compensation condition. □

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

Definition 21.2 (failure decomposition tensor)**.** *For a conjecture C and a run of length T, the failure decomposition tensor is*

$$\mathcal{F}(C,T)_{ij} = \sum_{t=0}^{T-1} \mathbf{1}\{\textit{module } i \textit{ failed gate } j \textit{ at step } t\} \in \mathbb{Z}_{\geq 0}^{26\times 6}.$$

The tensor records the full history of which modules failed which gates, providing a detailed diagnostic for the second attempt. The dominant failure mode is

$$(i^*, j^*) = \operatorname*{argmax}_{(i,j)} \mathcal{F}(C,T)_{ij},$$

and the second attempt targets the dominant mode first.

Theorem 21.3 (no collapse-free false release)**.** *Assume the release predicate is sound and Module 26 rejects memory updates with* $\mathrm{Collapse} > \theta_{\mathrm{collapse}}$*. Then no candidate can become a theorem merely by context rewriting, star-unitization, or BrLLM escalation.*

Proof. Context rewriting changes $\mathcal{P}_t$, star-unitization changes the numerical lift of gate defects, and BrLLM escalation changes candidate machinery. None of these operations writes directly to a verifier factor. The only map to theorem status is the multiplicative release predicate, which is sound by hypothesis. Therefore a zero verifier factor remains zero regardless of how much context is accumulated. □

Corollary 21.4 (accuracy-release gap)**.** *There exist runs with* $\mathsf{Accuracy}_{26}(C) \to 1$ *and* $\mathsf{Rel}_T = 0$*. The gap measures remaining formal or human verifier obligations that context engineering cannot fulfill.*

Proof. Construct a run where $\mathcal{R}_T, \mathcal{M}_*(G_T), \mathrm{MapDef}(F_T), \mathrm{Collapse}, \mathrm{NMDef}(\mathcal{N}_T) \to 0$ but $V_{\mathrm{formal}}(Y_T) = 0$ because no formalization has been submitted. The accuracy score approaches one while the release predicate is zero. This is precisely the gap between a high-quality candidate and a released theorem. □

22 Implementation contract

A reproducible implementation stores every run as a typed ledger:

$$\mathcal{L}_{\mathrm{run}} = \{(t, G_t, R_t, \mathcal{P}_t, \Delta\mathcal{P}_t, \mathcal{N}_t, V_t, \mathsf{Rel}_t, \sigma_t)\}_{t=0}^{T}.$$

The core loop is:

1. parse the conjecture into $C = (\mathcal{D}, \mathcal{L}, \mathcal{A}, P, Q, \mathcal{E}_0, \mathcal{B}_0, \mathcal{R}_0)$;
2. open proof, refutation and independence branches;
3. build the obligation hypergraph and six-gate vector;
4. run $\mathrm{B}^2\mathrm{LLM}$ and $\mathrm{B}^3\mathrm{LLM}$ repair with the current playbook;
5. construct or refine $\mathcal{N}_{\mathrm{new}}$ when persistence thresholds trigger;
6. project candidates through proof, computation, formalization, counterexample and human-review gates;
7. transform every failed gate into star-residue;
8. update the playbook by generator–reflector–curator delta curation;
9. reject memory updates that exceed the collapse threshold;
10. emit theorem only if $\mathsf{Rel} = 1$, otherwise emit conditional theorem, counterexample, independence/formulation report, or BUJ.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

23 Discussion

The practical consequence of Module 26 is that a long mathematical project can preserve the exact local facts that a short summary would delete: the failed bridge, the hidden quantifier, the nonfunctorial map, the exact subcritical estimate, the boundary case, the formalization hole, and the referee-style objection. This directly repairs the problem identified in context-adaptation research: detailed contexts can improve agent and domain-specific performance, while monolithic rewriting can collapse useful details (Zhang et al., 2026). In VSI, loss of such detail is not merely a software inconvenience; it can cause the second proof attempt to repeat the first proof's hidden failure. The corrected engine therefore turns memory into a mathematical object.

24 Detailed module calculus: local laws, kernels and verifier contracts

This section spells out the local mathematics of the twenty-six modules. The purpose is not to multiply names; it is to make every part of the engine auditable. The retained author-corpus citations are restricted to source records with public bibliographic anchors, namely holonomic/symplectic verification language and heterotic string-classification vocabulary (Bhattacharjee, 2026; Bhattacharjee et al., 2026). Uploaded-manuscript lineage records are deliberately omitted from the bibliography.

24.1 Module 1: synthetic intake and typed conjecture parsing

The intake map is

$$I_1 : \mathsf{Text}(C) \longrightarrow (\mathcal{D}, \mathcal{L}, \mathcal{A}, P, Q, \mathcal{E}_0, \mathcal{B}_0, \mathcal{R}_0, \Theta_0).$$

It must preserve quantifiers. A conjecture with target $\forall x \in D\,[P(x) \Rightarrow Q(x)]$ is not stored as a slogan; it is stored as a dependent object

$$\Pi_{x:D}\big(P(x) \to Q(x)\big),$$

with domain restrictions and regularity hypotheses attached to D. The first validator is the quantifier-loss score

$$\mathsf{QLoss}(I_1) = \sum_{q \in \mathsf{Quant}(C)} \mathbf{1}\{q \notin \mathsf{Quant}(I_1(C))\} + \sum_{h \in \mathsf{Hyp}(C)} \mathbf{1}\{h \notin \mathsf{Hyp}(I_1(C))\}.$$

The intake gate passes only when $\mathsf{QLoss} = 0$.

24.2 Module 2: symbolic type register

The type register is a functor

$$T_2 : \mathsf{Obj}(C) \to \mathsf{Type}_{\mathrm{math}},$$

where $\mathsf{Type}_{\mathrm{math}}$ includes algebraic, analytic, geometric, categorical, homotopical, probabilistic, computational, model-theoretic and formal classes. A bridge $B : A \to B'$ is admissible only if

$$\mathsf{TypeLoss}(B) = \|T_2(A) - T_2(B')\|_{\mathsf{tax}} + \mathsf{RegLoss}(B) + \mathsf{ScopeLoss}(B) = 0$$

or the nonzero loss is explicitly named as a condition.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

24.3 Module 3: synthesis multiplier $\bar{\Omega}$

The multiplier does not merely combine words. It generates typed constructions:

$$\bar{\Omega}_m = \mathrm{span}_{\mathcal{A}}\{A_i, (A_i + A_j)/2, [A_i, T_j], A_i \otimes T_j, \mathrm{Cone}(A_i \to A_j), \mathrm{Defect}(A_i, T_j)\}.$$

Each generated object carries a debt vector

$$\mathsf{Debt}(A) = \big(\mathsf{Debt}_{\mathrm{proof}}, \mathsf{Debt}_{\mathrm{formal}}, \mathsf{Debt}_{\mathrm{cite}}, \mathsf{Debt}_{\mathrm{scope}}, \mathsf{Debt}_{\mathrm{compute}}\big).$$

Generation increases the search space but cannot reduce release debt unless a verifier accepts the generated object.

24.4 Module 4: adversarial derivative $\bar{\Omega}'$

For a synthesis operator O_{ij} and perturbation ϵ_k, the adversarial derivative

$$\bar{\Omega}' = \left[\frac{\partial O_{ij}}{\partial \epsilon_k}\right]$$

measures fragility. The instability index of a bridge is

$$\mathsf{Instab}(B) = \sup_{\|\epsilon\| \le 1} \frac{\mathsf{Loss}(B + \epsilon) - \mathsf{Loss}(B)}{\|\epsilon\|}.$$

High instability sends the bridge to the falsification arena before it may enter the proof ledger.

24.5 Module 5: $\Xi_{\|\|\|\|\|}$ triple-horizon enumeration

The enumerator prevents short-memory collapse:

$$\Xi_{\|\|\|\|\|}, t = (\Xi_t^-, \Xi_t^0, \Xi_t^+).$$

The transition law

$$\Xi_{\|\|\|\|\|}, t+1 = \Pi_{\mathrm{retain}}\Big(\Xi_{\|\|\|\|\|}, t \oplus \mathsf{NewObl}_t \oplus \mathsf{Cert}_t \oplus \mathsf{Fail}_t\Big)$$

uses Π_{retain}, which is forbidden from deleting any terminal failure whose star-residue has positive mass.

24.6 Module 6: tri-branch conjecture attack

Every conjecture is attacked through

$$G(C) = G^+(C) \oplus G^-(C) \oplus G^\perp(C),$$

where G^+ is proof, G^- refutation, and $G^\perp$ independence/formulation dependence. The branch-energy vector is

$$\mathcal{E}_{\mathrm{branch}} = \big(E^+, E^-, E^\perp\big), \qquad E^\bullet = \mathsf{Gap}^\bullet + \mathsf{Debt}^\bullet - \mathsf{Cert}^\bullet.$$

Branch starvation is prohibited: if E^- or $E^\perp$ is high, proof-branch confidence is penalized.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

24.7 Module 7: falsification arena

The falsification arena is a search functor

$$\mathsf{False}_7 : \mathsf{Claim} \to \mathsf{CounterModel} \cup \mathsf{BoundaryCase} \cup \mathsf{NoCounterFound}.$$

It is scored by

$$\mathsf{FalsPressure}(Y) = \sup_{m \in \mathcal{M}_{\mathrm{test}}} \mathsf{Violation}(Y, m) \cdot \mathsf{Reach}(m) \cdot \mathsf{Relevance}(m).$$

A proof claim with unsearched high falsification pressure remains a candidate.

24.8 Module 8: bridge compiler

A bridge is a morphism

$$B : \prod_{j=1}^{k} \mathcal{O}(v_j) \to \mathcal{O}(v_0)$$

with loss

$$\ell(B) = \mathsf{HypLoss}(B) + \mathsf{DomainLoss}(B) + \mathsf{RegularityLoss}(B) + \mathsf{FunctorialityLoss}(B) + \mathsf{VerifierLoss}(B).$$

The bridge is accepted unconditionally only if $\ell(B) = 0$; otherwise it becomes a conditional bridge.

24.9 Module 9: barrier ledger

Known barriers are stored as typed blockers

$$\mathcal{B}(C) = \{(B_i, \mathsf{scope}_i, \mathsf{attackable}_i, \mathsf{bypassable}_i, \mathsf{formal}_i)\}_{i=1}^{r}.$$

For P vs NP the ledger contains relativization, natural proofs, algebrization and representation-theoretic obstruction layers (Baker et al., 1975; Razborov and Rudich, 1997; Aaronson and Wigderson, 2009; Mulmuley and Sohoni, 2001). For PDE targets, the ledger contains scaling and compactness barriers; for Hodge, standard-conjecture and support-realization barriers; for RH, maximal-correlation barriers.

24.10 Module 10: exact computation and finite probes

The exact computation module returns a certificate object

$$\kappa = (\mathsf{input}, \mathsf{algorithm}, \mathsf{hash}, \mathsf{version}, \mathsf{output}, \mathsf{checker}).$$

A finite computation can close a finite reduction, but cannot close an infinite theorem unless a theorem proves that the finite reduction is complete.

24.11 Module 11: residue sedimentation

Residue particles r_i are quotiented by

$$r_i \sim r_j \iff \mathsf{Type}(r_i) = \mathsf{Type}(r_j), \quad d_{\mathrm{fail}}(r_i, r_j) \le \epsilon, \quad \mathsf{Fix}(r_i) = \mathsf{Fix}(r_j).$$

The sedimented quotient is

$$\mathsf{Sed}(R_t) = R_t/\sim = \{R_1, \dots, R_m\}.$$

A residue class is harmless only if it is canonical and explicitly nonblocking.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

24.12 Module 12: sand-tile toppling

Each tile v stores burden $h_t(v)$. The local toppling law was given earlier, but the repair decision is

$$\mathsf{Repair}(v) = \operatorname*{argmin}_{a \in \mathcal{A}_v} \left[h_t(v \mid a) + \lambda \mathsf{ScopeLoss}(a) + \mu \mathsf{Cost}(a) - \nu \mathsf{CertGain}(a) \right].$$

This makes repair local and reviewable.

24.13 Module 13: cellular propagation

The cellular automaton propagates local changes:

$$c_{t+1}(i) = \varphi\big(c_t(i-r), \dots, c_t(i+r), \eta_t(i), V_t(i), R_t(i)\big).$$

The global state is accepted only after projection

$$c_{t+1}^{\mathrm{acc}} = \Pi_{\mathcal{V}}(c_{t+1}).$$

24.14 Module 14: histogram difficulty diagnostics

Let $d(v)$ be normalized node difficulty and $m(v)$ residual mass. The histogram is

$$\mathsf{Hist}_C(k) = \sum_{v \in V(G_C)} m(v) \mathbf{1}\{d(v) \in [k\Delta, (k+1)\Delta)\}.$$

The blocking band is

$$\mathsf{Block}(C) = \operatorname*{argmax}_k \mathsf{Hist}_C(k) \mathsf{FailRate}(k) \mathsf{BridgeCentrality}(k).$$

24.15 Module 15: ogive closure curve

The ogive is

$$\mathsf{Ogive}_{C,t}(x) = \frac{\sum_{v: d(v) \le x} m_t(v) \mathbf{1}_{\sigma_t(v) = \mathrm{PASS}}}{\sum_{v: d(v) \le x} m_t(v)}.$$

A healthy run has rising ogive and shrinking tail mass:

$$\frac{d}{dt} \mathsf{Ogive}_{C,t}(x) \ge -\epsilon_t, \qquad \mathsf{Tail}_{C,t}(a) = \sum_{v: d(v) > a} m_t(v) \downarrow .$$

24.16 Module 16: historical closure transducer

The closure policy is a mixture over historical strategies:

$$\mathsf{H}(C) = \operatorname*{argmax}_{\pi \in \Pi_{\mathrm{hist}}} \langle W d(C), p_\pi \rangle.$$

Bridge-dominant problems call Wiles-style encoders; residue-dominant problems call Perelman-style flows; semantic-poor problems call Deligne-style cohomological enlargement; finite-certificate problems call Hales/Gonthier-style reduction.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

24.17 Module 17: proof-contract writer

A claim is stored as

$$Y = \langle \mathrm{id}, \mathrm{statement}, \mathrm{axioms}, \mathrm{domain}, \mathrm{regularity}, \mathrm{dependencies}, \mathrm{certificate}, \mathrm{status} \rangle.$$

The contract is invalid if dependency closure fails:

$$\mathsf{DepClose}(Y) = 0 \Longrightarrow \mathsf{Rel}(Y) = 0.$$

24.18 Module 18: release audit

The audit multiplies verifier factors but also records the first zero factor:

$$\mathsf{FirstFail}(Y) = \min\{\chi : V_\chi(Y) = 0\}.$$

The output is not merely "not proved"; it is a named repair instruction.

24.19 Module 19: status emitter

The emitter is

$$\Sigma(Y) = \begin{cases} \mathsf{THEOREM}, & \mathsf{Rel}(Y) = 1, \\ \mathsf{COUNTEREXAMPLE}, & V_{\mathrm{counter}}(Y) = 1, \\ \mathsf{CONDITIONAL}(Y;\Lambda), & V_{\mathrm{proof}}(Y \mid \Lambda) = 1, \\ \mathsf{INDEPENDENCE/FORMULATION}, & V_{\perp}(Y) = 1, \\ \mathrm{BUJ}(Y), & \text{otherwise.} \end{cases}$$

24.20 Module 20: six-gate residue synthesizer

The six gates are not arbitrary. They correspond to representation fidelity, upper envelope, lower obstruction, semantic witness, barrier survival and formal kernel. The gate defect vector is

$$\Delta(G) = (\Delta_{\mathrm{orbit}}, \Delta_{\mathrm{upper}}, \Delta_{\mathrm{lower}}, \Delta_{\mathrm{witness}}, \Delta_{\mathrm{barrier}}, \Delta_{\mathrm{formal}}).$$

The star-to-one kernel later acts on exactly this vector.

24.21 Module 21: new-mathematics calibration

If ordinary repair fails, the system searches for a new machine $\mathfrak{M}$ minimizing

$$\mathsf{NMDef}(\mathfrak{M}) = \mathsf{BridgeLoss}(\mathfrak{M}) + \mathrm{MapDef}(\mathfrak{M}) + \mathsf{Barrier}(\mathfrak{M}) + \mathsf{FormalDebt}(\mathfrak{M}).$$

This is a machinery search, not direct proof search.

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

24.22 Module 22: ambient-machine constructor

The constructor searches for a flow Φ_t^Λ and Lyapunov function E^Λ such that

$$\frac{d}{dt}E^\Lambda(\Phi_t^\Lambda(C)) \le 0, \qquad E^\Lambda(\Phi_\infty^\Lambda(C)) = 0.$$

Residues need not be individually solved if they are structurally submerged and certified harmless.

24.23 Module 23: candidate-proof generation

A candidate proof is a tuple

$$\pi_{\mathrm{cand}} = (\Lambda, \mathfrak{M}^\Lambda, \mathsf{Steps}, \mathsf{KernelObl}, \mathsf{HumanObl}, \mathsf{FormalObl}, \mathsf{Status}).$$

Its honest label is CAND-STRONG unless $\mathsf{HumanObl} = \mathsf{FormalObl} = 0$.

24.24 Module 24: Ricci-vector second-attempt generator

The geometric deficit is

$$\mathcal{D}_{24} = \mathcal{R}_t + \mathrm{ChordDef} + \mathrm{MapDef} + \mathsf{ScopeLoss}.$$

Second attempts are generated by

$$S_{t+1} = S_t - \eta\nabla\mathcal{D}_{24} + \Pi_{\Omega_{\mathrm{off}}}(S_t) + \mathsf{HistTrans}(S_t).$$

24.25 Module 25: star-to-one unitization

The star gate $*$ is replaced by a numerical saturation $s = e^{-\Delta}$. A star is not a decorative symbol; it is an unresolved gate with measurable defect. The back-loop equation is

$$G^{t+1} = \Pi_{\mathcal{V}}\left[G^t - \nabla\mathcal{R}_t + \mathcal{N}_{\mathrm{new}}(\mathrm{B}^{r+1}\mathrm{LLM}(\mathrm{B}^r\mathrm{LLM}(\mathfrak{R}_*^t, \mathcal{P}_t)))\right].$$

24.26 Module 26: context-playbook memory

The playbook makes the engine non-amnesic. A memory update is accepted only if

$$\mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) \le \theta_{\mathrm{collapse}}, \qquad U(\mathcal{P}_{t+1}) \ge U(\mathcal{P}_t) - \epsilon.$$

This preserves detailed failed routes for future repair.

25 Formal convergence conditions for a verifier-gated run

A finite run is closure-stable if there exists T such that for all $t \ge T$,

$$\mathsf{Tail}_{C,t}(a) < \epsilon, \quad \mathcal{M}_*(G_t) < \epsilon, \quad \mathrm{MapDef}(F_t) < \epsilon, \quad \mathrm{Collapse}(\mathcal{P}_{t-1}, \mathcal{P}_t) < \theta, \quad \mathsf{Rel}_t \in \{0, 1\}.$$

If $\mathsf{Rel}_t = 1$, the output is theorem relative to the declared axiom base. If $\mathsf{Rel}_t = 0$, stability means the engine has found a stable obstruction ledger, not a proof.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

Definition 25.1 (Lyapunov function for the full engine)**.** *The total engine Lyapunov function is*

$$\mathcal{J}_t^{\text{total}} = \sum_{i=1}^{26} \omega_i \big(D_t^{(i)} + \alpha_i \mathsf{Debt}_t^{(i)} + \beta_i \mathsf{Residue}_t^{(i)}\big) + \mu \mathcal{M}_*(G_t) + \nu \,\mathrm{MapDef}(F_t) + \kappa \,\mathrm{Collapse}_t + \lambda_{\square} \mathsf{BoxRes}_t + \lambda_{\mathsf{CMV}}(1 - \mathsf{CMV}_t) + \lambda_{\text{lift}}(1 - \mathsf{AnalyticLift}_t) - \zeta \mathsf{Cert}_t,$$

with all weights positive and $\zeta > 0$ so that certification gains decrease the energy. The Lyapunov function is bounded below by $-\zeta \cdot \mathsf{MaxCert}$ and above by the initial condition $\mathcal{J}_0^{\text{total}} < \infty$.

Lemma 25.2 (strict descent at non-stable states)**.** *At any state S_t that is not closure-stable, there exists at least one module i with an admissible update direction $z_t^{(i)}$ satisfying $\left\|\Pi_{\text{cert}} z_t^{(i)}\right\|^2 > 0$, yielding a strict decrease in $\mathcal{J}_t^{\text{total}}$.*

Proof. If the state is not closure-stable, at least one of the five stability conditions fails. If $\mathsf{Tail}_{C,t}(a) \geq \epsilon$, then Module 14 (histogram diagnostics) has a non-trivial obstruction stratum with positive mass, and Module 15 (ogive) can identify a non-trivial descent direction. If $\mathcal{M}_*(G_t) \geq \epsilon$, then Module 25 has a back-loop direction with positive certified component. If $\mathrm{MapDef}(F_t) \geq \epsilon$, the bridge compiler (Module 8) can attempt a refined bridge. If $\mathrm{Collapse}$ is large, Module 26 rejects the update, preserving past information and allowing a fresh direction. If $\mathsf{Rel}_t \notin \{0, 1\}$, the release auditor (Module 18) identifies the first failing verifier and emits a repair instruction. In each case, the corresponding module's certified direction has positive norm. □

Theorem 25.3 (closure or stable obstruction)**.** *Assume the obligation graph is finite after quotienting by residue equivalence, verifier projection is idempotent, and the energy decrease condition*

$$\mathcal{J}_{t+1} \leq \mathcal{J}_t - \gamma \left\|\Pi_{\text{cert}} z_t\right\|^2 + \epsilon_t, \qquad \sum_t \epsilon_t < \infty$$

holds. Then either the run reaches a release stratum, or it reaches a stable obstruction stratum with typed nonzero residue.

Proof. Summing the energy inequality gives finite total certified impulse unless a release stratum is reached. If no release is reached, the sequence has no infinite hidden descent because the quotient graph is finite and verifier projection is idempotent. Hence a recurrent unclosed state must have typed residue; otherwise an untyped toppling or playbook delta would still be available, contradicting stability. □

Proposition 25.4 (quantitative convergence rate)**.** *Under the additional assumption that the certified descent is uniformly bounded below at non-stable states by $\gamma_0 = \inf\{\gamma \left\|\Pi_{\text{cert}} z_t\right\|^2 : S_t \text{ not closure-stable}\} > 0$, and that $\epsilon_t \leq \epsilon_0 \rho^t$ for some $\rho \in (0, 1)$, the run reaches ϵ-closure-stability by time*

$$T_\epsilon \leq \frac{\mathcal{J}_0^{\text{total}} + \epsilon_0/(1-\rho)}{\gamma_0 \epsilon}.$$

Proof. At each non-stable step, the energy decreases by at least $\gamma_0 - \epsilon_t \geq \gamma_0 - \epsilon_0 \rho^t$. The total energy decrease over T_ϵ steps satisfies $\mathcal{J}_0^{\text{total}} - \mathcal{J}_{T_\epsilon}^{\text{total}} \geq T_\epsilon \gamma_0 - \epsilon_0/(1-\rho)$. Since $\mathcal{J}$ is bounded below, we need $T_\epsilon \gamma_0 - \epsilon_0/(1-\rho) \leq \mathcal{J}_0^{\text{total}}$, giving the stated bound on T_ϵ. □

Theorem 25.5 (persistent-obstacle dichotomy)**.** *Under the hypotheses of the closure theorem, if the run does not converge to a theorem, then it converges to a state S_∞ with the following properties:*

(i) $\mathsf{DebtMass}(S_\infty) > 0$*: the unresolved obligation mass is strictly positive;*

(ii) every remaining obligation has positive barrier score: $\mathrm{Barrier}(v) > 0$ *for all* $v \in \mathsf{Core}(S_\infty)$*;*

(iii) the playbook $\mathcal{P}_\infty$ contains the exact typed record of every failed attempt, with $|\mathsf{Core}(\mathcal{P}_\infty)|$ proportional to the number of distinct failure modes encountered.

Proof. (i) follows from the fact that $\mathsf{Rel}_{S_\infty} = 0$ implies at least one non-PASS terminal leaf. (ii) follows because if any obligation had zero barrier, the sand-tile repair or BrLLM escalation would have a constructive descent direction, contradicting the stability of S_∞. (iii) follows from the no-hidden-context theorem: the collapse guard ensures every failed core obligation is preserved in the playbook. □

26 Reviewer-facing checklist

The paper proposes the following reviewer checklist for any VSI run:

1. Are all definitions typed with domain, regularity and axiom scope?
2. Are proof, refutation and independence/formulation branches all open until blocked by evidence?
3. Is every bridge accompanied by zero or explicitly named scope loss?
4. Are all stars converted into numerical defects?
5. Does Module 24 compare the attempt to a verified control or verifier-specified target vector?
6. Does Module 25 route persistent star-residue into BrLLM and new machinery rather than rhetorical repetition?
7. Does Module 26 preserve the failed route as a playbook bullet?
8. Is the final label theorem, candidate, conditional, counterexample, independence/formulation or BUJ?

27 Expanded conjecture sheets: gate-by-gate calculations

This section records the exact form in which VSI stores conjecture attempts. The purpose is to prevent narrative compression. Each target has a state vector, a six-gate vector, a residue vector, a candidate machinery space, a verifier contract, and a second-attempt rule.

27.1 Poincare calibration sheet

The Poincare control is stored as

$$C_{\mathrm{Pc}} = (\mathsf{3Man}, \mathcal{L}_{\mathrm{diff}}, \mathcal{A}_{\mathrm{geom}}, \text{closed simply connected } M^3,\ M^3 \simeq S^3, E_0, B_0, R_0).$$

The verified route is encoded by the vector

$$v_\infty^{\mathrm{Pc}} = (\mathsf{RicciFlow}, \mathcal{F}, \mathcal{W}, \mathsf{NoCollapse}, \mathsf{CanonicalNeighborhood}, \mathsf{Surgery}, \mathsf{Extinction}).$$

The VSI attempt vector v_t is scored by

$$\mathcal{R}_t^{\mathrm{Pc}} = \frac{\left\| v_t - v_\infty^{\mathrm{Pc}} \right\|^2}{1 + \|v_0\|^2 + \|v_\infty^{\mathrm{Pc}}\|^2} + \lambda \sum_j \mathsf{SingResidue}_j(t).$$

The calibration passes when the engine identifies that individual singularities are not separate sub-conjectures to be solved one by one; they are residues controlled by monotone entropy, non-collapsing and surgery. In VSI notation,

$$\mathsf{Sing}(g(t)) \xrightarrow{\mathsf{sediment}} \{R_{\mathrm{neck}}, R_{\mathrm{cap}}, R_{\mathrm{extinction}}\}, \qquad \mathsf{Harmless}(R_i) = 1.$$

The lesson transferred to open problems is that a residue may be controlled by constructing a machine in which it cannot affect the terminal invariant.

27.2 Fermat calibration sheet

The Fermat control has negated branch

$$\exists a, b, c, p > 2, \qquad a^p + b^p = c^p.$$

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

The bridge compiler produces

$$(a,b,c,p) \mapsto E_{a,b,c} : y^2 = x(x-a^p)(x+b^p),$$

then the historical route supplies

$$E_{a,b,c} \Rightarrow \mathsf{semistable} \wedge \mathsf{nonmodular}, \qquad \mathsf{ModLift} \Rightarrow \mathsf{modular},$$

a contradiction. The VSI bridge-loss is

$$\ell_{\mathrm{FLT}} = \mathsf{FreyLoss} + \mathsf{RibetLoss} + \mathsf{LocalDefLoss} + \mathsf{HeckeLoss} + \mathsf{PatchLoss}.$$

A correct attempt minimizes ℓ_{FLT} by importing the right ambient theory, not by searching the original exponential equation directly. The central-chord point is

$$C_*^{\mathrm{FLT}} = \mathsf{ModularityPatch} \cap \mathsf{FreyEncoder} \cap \mathsf{RibetContradiction}.$$

27.3 Collatz Conjecture candidate sheet

The Collatz template is imported as the canonical Module 23 candidate package pattern, with barrier references to the standard $3x+1$ literature (Terras, 1976; Lagarias, 1985; Tao, 2022). Define

$$T(n) = \begin{cases} n/2, & n \equiv 0 \pmod 2, \\ (3n+1)/2, & n \equiv 1 \pmod 2, \end{cases} \qquad C_{\mathrm{Col}} : \ \forall n \in \mathbb{N}\ \exists k \in \mathbb{N}_0 : T^{(k)}(n) = 1.$$

The six-gate vector is

$$G_{\mathrm{Col}} = (g_{\mathrm{orbit}}, g_{\mathrm{upper}}, g_{\mathrm{lower}}, g_{\mathrm{witness}}, g_{\mathrm{barrier}}, g_{\mathrm{formal}}), \qquad \Delta(G_{\mathrm{Col}}) = (0, 0.80, 1.00, 0, 0.40, 1.00).$$

The candidate proof follows the Collatz-style VSI sequence:

$$M_1 \to M_9 \to M_6 \to M_{21} \to M_{22} \to M_{20} \to M_{25} \to M_{26} \to M_{24} \to M_{23} \to M_{18} \to M_{19}.$$

The current candidate status is

$$\text{CAND-STRONG}_{\mathrm{Col}} : \quad V_{\mathrm{model}} = 1, \qquad V_{\mathrm{human}} = 0, \qquad V_{\mathrm{referee}} = 0, \qquad \mathcal{M}_*(G_{\mathrm{Col}}) > 0.$$

The imported rule is not that Collatz is released as a theorem. The rule is that every future conjecture package must be written in the same disciplined form: named gates, numerical defects, new machinery, playbook bullets, release audit and referee checklist. The star-to-one target for Collatz is

$$\min_{\mathcal{M} \in \mathsf{Mach}} \left(\mathsf{BridgeLoss}_3(\mathcal{M}) + \mathrm{MapDef}_3(\mathcal{M}) + \mathsf{Barrier}_3(\mathcal{M}) + \mathsf{FormalDebt}_3(\mathcal{M})\right) = 0,$$

which would close the lower-gate only after uniform all-n stopping-time control and referee verification.

27.4 P versus NP candidate sheet

The complexity sheet is

$$C_{\mathrm{PNP}} = (\mathsf{BoolCirc}, \mathcal{L}_{\mathrm{comp}}, \mathsf{ZFC} + \mathsf{complexity}, \mathrm{P}, \mathrm{NP}, E_0, B_0, R_0).$$

The barrier vector is

$$B_{\mathrm{PNP}} = (B_{\mathrm{rel}}, B_{\mathrm{nat}}, B_{\mathrm{alg}}, B_{\mathrm{gct}}).$$

A candidate entropy kernel has the schematic form

$$\Omega_{\mathrm{cc}}(C, C') = H(d_{\mathrm{circ}}(C, C')) - \sum_{k \le \lfloor \log_2 n \rfloor} 2^{-k} |\mathsf{pleth}_k(C) - \mathsf{pleth}_k(C')|.$$

The six gates become

$$G_{\mathrm{PNP}} = (C_{\mathrm{orbit}}, C_{\mathrm{envelope}}, C_{\mathrm{lower}}, C_{\mathrm{semantic}}, C_{\mathrm{barrier}}, C_{\mathrm{formal}}).$$

The open star-mass is

$$\mathcal{M}_*(G_{\mathrm{PNP}}) = w_1(1-e^{-\Delta_{\mathrm{orbit}}})^2 + w_2(1-e^{-\Delta_{\mathrm{lower}}})^2 + w_3(1-e^{-\Delta_{\mathrm{barrier}}})^2 + w_4(1-e^{-\Delta_{\mathrm{formal}}})^2.$$

A candidate proof may be recorded only as CAND-STRONG until the kernel proves non-algebrizable monotone separation and the formal lower-bound transfer is checked.

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

27.5 Yang–Mills candidate sheet

The Yang–Mills sheet is constructed from a lattice-to-continuum route:

$$\mu_{a,L} \xrightarrow{a\to 0, L\to\infty} \mu_{\mathrm{cont}} \xrightarrow{\mathsf{OS}} (\mathcal{H}, \Omega, U(t), \mathcal{A}),$$

where the gates are gauge invariance, reflection positivity, tightness, Osterwalder–Schrader reconstruction, nontriviality, exponential clustering and mass gap. The gap criterion is

$$\exists m > 0 \quad \text{such that} \quad \langle \Omega, A(x)A(0)\Omega\rangle_c \le C_A e^{-m|x|}.$$

The star defects are

$$\Delta_{\mathrm{YM}} = \Delta_{\mathrm{tight}} + \Delta_{\mathrm{OS}} + \Delta_{\mathrm{nontriv}} + \Delta_{\mathrm{cluster}} + \Delta_{\mathrm{gap}}.$$

The second attempt is routed into whichever term dominates. If $\Delta_{\mathrm{cluster}}$ dominates, the new machinery search targets uniform infrared decay; if Δ_{OS} dominates, it targets reflection-positive reconstruction; if $\Delta_{\mathrm{nontriv}}$ dominates, it targets continuum non-Gaussianity.

27.6 Navier–Stokes candidate sheet

The Navier–Stokes package is

$$\partial_t u + (u\cdot\nabla)u + \nabla p = \nu\Delta u, \qquad \nabla\cdot u = 0, \qquad u(0) = u_0.$$

The energy inequality yields

$$\frac{1}{2}\|u(t)\|_{L^2}^2 + \nu\int_0^t \|\nabla u(s)\|_{L^2}^2\, ds \le \frac{1}{2}\|u_0\|_{L^2}^2,$$

but the missing regularity gate is scaling-critical. VSI stores

$$\Delta_{\mathrm{NS}} = \Delta_{L^3} + \Delta_{p^{3/2}} + \Delta_{\mathrm{backward}} + \Delta_{\mathrm{cascade}} + \Delta_{\mathrm{singular}}.$$

The machinery candidates include frequency-envelope monotonicity, localized pressure decoupling, critical Morrey control, vorticity alignment, and caloric-gauge style transport. A theorem label is blocked unless the critical gate produces

$$\sup_{Q_r(x,t)} r^{-2}\int_{Q_r(x,t)} |u|^3 + |p|^{3/2} < \epsilon_*.$$

27.7 Birch and Swinnerton-Dyer candidate sheet

BSD is stored as a rank comparison and leading-coefficient identity. The analytic side is

$$r_{\mathrm{an}} = \mathrm{ord}_{s=1} L(E,s),$$

and the algebraic side is

$$r_{\mathrm{alg}} = \mathrm{rank}\, E(\mathbb{Q}).$$

The formula gate is

$$\lim_{s\to 1}\frac{L(E,s)}{(s-1)^r} = \frac{\Omega_E\,\mathrm{Reg}(E)\,|\,\mathrm{Sha}(E)|\prod_p c_p}{|E(\mathbb{Q})_{\mathrm{tors}}|^2}.$$

VSI residue decomposes into

$$R_{\mathrm{BSD}} = R_{\mathrm{Selmer}} \oplus R_{\mathrm{Sha}} \oplus R_{\mathrm{regulator}} \oplus R_{\mathrm{p-adic}} \oplus R_{\mathrm{height}} \oplus R_{\mathrm{analytic}}.$$

The second attempt is routed by the dominant residue. For rank zero and one, Gross–Zagier–Kolyvagin style machinery supplies calibration (Gross and Zagier, 1986; Kolyvagin, 1990); higher rank remains a star-heavy region.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

27.8 Riemann Hypothesis candidate sheet

The RH sheet used by VSI is a Liouville-collar target:

$$L(X) = \sum_{n \le X} \lambda(n), \qquad L(X) = O_\epsilon(X^{1/2+\epsilon}).$$

The square identity opens

$$|L(X)|^2 = X + 2 \sum_{1 \le h < X} \sum_{n \le X-h} \lambda(n)\lambda(n+h).$$

The maximal collar gate is

$$\sum_{H < h \le 2H} \left(\sup_{Y \le X-h} \left| \sum_{n \le Y} \lambda(n)\lambda(n+h) \right| \right)^2 \ll_{\epsilon,\eta} X^{2+\epsilon} H^{-1}.$$

VSI records the missing pieces as maximal spectral large sieve, conductor-uniform delta regularization, exceptional-spectrum absorption, Eisenstein/subconvex control and effective Littlewood–Denjoy bridge. Each is a separate star; no global claim may ignore one.

27.9 Hodge Conjecture candidate sheet

The Hodge sheet is stored as

$$\mathrm{cl} : CH^p(X)_{\mathbb{Q}} \to H^{2p}(X, \mathbb{Q}) \cap H^{p,p}(X, \mathbb{C}).$$

The zero-defect mechanism represents a Hodge class by a motivated presentation graph G and defines

$$\delta(G) = \sum_{e \in E(G)} \mathbf{1}\{e \text{ is inverse-Lefschetz on a non-abelian support}\}.$$

The star-to-one target is $\delta(G_t) \to 0$ plus functorial Chow-level realization. A bridge edge is accepted only when

$$\Lambda_{Y,h} = (P_e)_* \circ \Lambda_{A_e,\theta_e} \circ (Q_e)^*$$

is algebraic and scope-compatible. Otherwise it remains a star-residue.

28 Dense equations for second-attempt routing

The second-attempt equations below should be read as a transport calculus on the residue manifold. The current failed proof vector is not discarded; it is decomposed into coordinates whose largest components determine the next attack direction. Inline, the rule is $v_{t+1} = v_t - \Pi_{\mathrm{bad}} v_t + \Pi_{\mathrm{cert}} z_t$, where Π_{bad} removes the dominant obstruction and Π_{cert} injects only verified descent. This makes the route aggressive because it attacks the largest coordinate, but rigorous because the replacement vector is verifier-projected. Let C be any target. Define the combined defect vector

$$\mathfrak{D}_t(C) = \big(\mathcal{R}_t, \mathcal{M}_{*t}, \mathrm{MapDef}_t, \mathrm{ChordDef}_t, \mathrm{Collapse}_t, \mathrm{NMDef}_t, \mathsf{FormalDebt}_t, \mathsf{Barrier}_t\big) \in \mathbb{R}^8_{\ge 0}.$$

The second-attempt selector is

$$\mathsf{Route}_2(C) = \operatorname*{argmax}_j \left[w_j \mathfrak{D}_{t,j}(C) \cdot \mathsf{Centrality}_j \cdot \mathsf{Repairability}_j \right].$$

Here $\mathsf{Centrality}_j$ is the PageRank-style centrality of defect j in the obligation graph (how many other defects depend on it being resolved first), and $\mathsf{Repairability}_j \in [0,1]$ is an estimate of the probability that the current machinery can make progress on defect j.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

Second-attempt routing is treated as a controlled transport problem: the next attempt is not a retry of the same proof but a movement along the largest verified divergence coordinate.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

Definition 28.1 (repairability estimate)**.** *The repairability of defect j given machine $\mathcal{N}$ is*

$$\mathsf{Repairability}_j(\mathcal{N}) = \mathbb{P}\big(\Delta_j^{t+1} < \Delta_j^t \mid \mathcal{N}, \mathcal{P}_t\big)\,,$$

estimated from the historical success rate of machine $\mathcal{N}$ against defects of the same type and centrality class. It satisfies $\mathsf{Repairability}_j \in [0,1]$ *and* $\mathsf{Repairability}_j = 0$ *for defects blocked by a hard barrier that $\mathcal{N}$ cannot bypass.*

The machinery family is ordered by expected verifier gain

$$\mathsf{EVG}(\mathcal{M} \mid j) = \mathbb{E}\left[\Delta\mathsf{Cert}_j - \alpha\Delta\mathsf{Debt}_j - \beta\Delta\mathsf{Risk}_j - \gamma\Delta\mathrm{Collapse}_j \mid \mathcal{M}\right].$$

The next machine is

$$\mathcal{M}_{t+1} = \operatorname*{argmax}_{\mathcal{M}\in\mathfrak{Mach}(j)} \mathsf{EVG}(\mathcal{M} \mid j),$$

subject to the hard constraint that it cannot directly set a verifier bit to one.

Proposition 28.2 (expected verifier gain decomposition)**.** *The expected verifier gain decomposes as*

$$\begin{aligned}\mathsf{EVG}(\mathcal{M} \mid j) = {}& \mathsf{Repairability}_j(\mathcal{M}) \cdot \mathbb{E}[\Delta\mathsf{Cert}_j \mid \mathcal{M}, \textit{repair}] \\ & - \alpha\mathbb{E}[\Delta\mathsf{Debt}_j \mid \mathcal{M}] - \beta\mathbb{E}[\Delta\mathsf{Risk}_j \mid \mathcal{M}] - \gamma\mathbb{E}[\Delta\,\mathrm{Collapse}_j \mid \mathcal{M}].\end{aligned}$$

When two machines have equal repairability and certification gain, the one with lower expected debt, risk, and collapse impact is preferred. This is the principle of minimal side-effect machine selection.

Proof. The decomposition follows by conditioning on whether the machine successfully repairs defect j or not. Unconditional expectation splits into the Repairability-weighted repair term and the unconditional side-effect terms, which occur regardless of repair success. □

Theorem 28.3 (second-attempt improvement guarantee)**.** *If* $\mathsf{Route}_2(C) = j^*$ *and the selected machine* $\mathcal{M}_{t+1}$ *satisfies* $\mathsf{EVG}(\mathcal{M}_{t+1} \mid j^*) > 0$*, then*

$$\mathbb{E}[\mathcal{J}_{t+1}] < \mathcal{J}_t.$$

That is, a positive expected verifier gain implies expected energy decrease.

Proof. The energy change is

$$\mathcal{J}_{t+1} - \mathcal{J}_t = -\zeta\Delta\mathsf{Cert}_{j^*} + \alpha\mathsf{Debt}_{t+1} + \beta\mathsf{Risk}_{t+1} + \kappa\,\mathrm{Collapse}_{t+1} + \text{other terms}.$$

Taking expectations and using $\mathsf{EVG} > 0$:

$$\mathbb{E}[\mathcal{J}_{t+1} - \mathcal{J}_t] = \mathbb{E}[-\zeta\Delta\mathsf{Cert}_{j^*} + \alpha\Delta\mathsf{Debt}_{j^*} + \beta\Delta\mathsf{Risk}_{j^*} + \gamma\Delta\,\mathrm{Collapse}_{j^*}] \cdot \omega_{j^*} + \text{other terms}.$$

The first bracket is $-\mathsf{EVG}(\mathcal{M}_{t+1} \mid j^*) < 0$ by assumption (with sign adjusted for the energy minimization convention). The other terms involve non-targeted defects and are bounded above by zero under the hard constraint. Hence the expected energy decreases. □

29 Playbook-bullet schema for mathematical work

A VSI bullet is not a summary sentence. It is a record:

$$b = (\mathrm{id}, \mathrm{problem}, \mathrm{gate}, \mathrm{route}, \mathrm{claim}, \mathrm{failure}, \mathrm{repair}, \mathrm{evidence}, \mathrm{verifier}, h, r, u, \xi).$$

Here h and r count helpful and harmful uses, u is utility, and ξ is an embedding or symbolic retrieval key. The update after a run is

$$b^{t+1} = \mathsf{Update}(b^t; \Delta h, \Delta r, \Delta\mathrm{claim}, \Delta\mathrm{failure}, \Delta\mathrm{repair}).$$

The retrieval score is

$$\mathsf{Retrieve}(b, C) = \alpha\,\mathrm{sim}(\xi_b, \xi_C) + \beta\mathbf{1}\{\mathrm{gate}_b = \mathrm{gate}_C\} + \gamma h_b - \delta r_b - \lambda\mathsf{AgePenalty}(b).$$

The reflector is allowed to propose a delta but not to rewrite the whole playbook. This is the point of the Module 26 import from context engineering (Zhang et al., 2026).

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

30 No-hidden-context theorem

Theorem 30.1 (playbook retention prevents silent loss of proof-critical residue)**.** *Assume every failed verifier call emits a bullet b_f with status* FAIL *or* UNK *and $b_f \in \mathsf{Core}(\mathcal{P}_t)$ until the corresponding gate is discharged. If the collapse guard rejects updates with* $\mathrm{Collapse} > \theta < 1/|\mathsf{Core}(\mathcal{P}_t)|$*, then no single context update can delete all records of an unresolved core failure.*

Proof. Deleting a core failure removes at least one element from the core intersection. The collapse score is then at least $1/|\mathsf{Core}(\mathcal{P}_t)|$, contradicting the threshold. Thus an unresolved core failure remains represented after every accepted update. □

Corollary 30.2 (accumulation of proof-critical information)**.** *Under the conditions of the no-hidden-context theorem, the playbook $\mathcal{P}_t$ satisfies*

$$|\mathsf{Core}(\mathcal{P}_t)| \geq |\mathsf{Core}(\mathcal{P}_0)| + \sum_{s=0}^{t-1} \mathbf{1}\{\textit{new core failure at step } s\}.$$

That is, the core set is monotone non-decreasing, and every newly encountered failure mode adds at least one bullet to the core.

Proof. By the emission rule, each new core failure creates a bullet with FAIL status that enters the core. By the collapse guard, existing core bullets cannot be deleted unless the corresponding gate is discharged (which changes status from FAIL/UNK to PASS, removing the bullet from the core by definition). Thus the core grows with each new failure and shrinks only when a failure is resolved. □

Definition 30.3 (proof-critical information density)**.** *The proof-critical information density of the playbook at time t is*

$$\rho_{\mathrm{crit}}(\mathcal{P}_t) = \frac{|\mathsf{Core}(\mathcal{P}_t)|}{|\mathcal{P}_t|} \in [0,1].$$

A high density means the playbook is dominated by unresolved critical failures. A low density means most bullets are useful positive insights, which is the desired regime for a mature proof attempt.

Proposition 30.4 (density decrease implies progress)**.** *If $\rho_{\mathrm{crit}}(\mathcal{P}_{t+1}) < \rho_{\mathrm{crit}}(\mathcal{P}_t)$, then at least one of the following holds:*

(a) at least one core failure has been resolved: the corresponding gate changed from FAIL/UNK to PASS;

(b) the playbook grew by adding positive (non-core) bullets from a successful proof step.

In both cases, genuine mathematical progress was made.

Proof. The density decreases if $|\mathsf{Core}(\mathcal{P}_{t+1})|/|\mathcal{P}_{t+1}| < |\mathsf{Core}(\mathcal{P}_t)|/|\mathcal{P}_t|$. This can happen if $|\mathsf{Core}|$ decreases (a core failure is resolved, case (a)) or if $|\mathcal{P}|$ increases without a proportional increase in $|\mathsf{Core}|$ (new positive bullets were added, case (b)). In case (a), a verifier gate that was FAIL is now PASS, which is a genuine proof advance. In case (b), the generator found successful proof steps that generated non-core bullets, which is also genuine progress. □

31 Formal implementation pseudocode

Before the pseudocode is read procedurally, it should be read denotationally: a program line $x \leftarrow f(x)$ denotes a typed morphism in the state category, and a failed assertion denotes a new residue object in $\mathcal{R}_t$. This prevents implementation

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

The pseudocode is a mathematical specification, not mere software notation: every branch corresponds to a proof-state transition and every failed assertion becomes residue.

convenience from changing the mathematical meaning of the engine. The implementation-level transition can be written as:

Input: $C, \mathcal{P}_t, G_t, R_t, V_t$.
$\Delta\mathcal{P}_t \leftarrow \mathsf{Reflect}(\mathsf{Gen}(C, G_t, R_t, \mathcal{P}_t))$;
$\mathcal{P}' \leftarrow \mathsf{Dedup}(\mathsf{Refine}(\mathsf{Append}(\mathcal{P}_t, \Delta\mathcal{P}_t)))$;
if $\mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}') > \theta_{\mathrm{collapse}}$ **then** $\mathcal{P}_{t+1} \leftarrow \mathcal{P}_t$ **else** $\mathcal{P}_{t+1} \leftarrow \mathcal{P}'$;
$\mathfrak{R}_* \leftarrow \mathsf{StarResidue}(G_t, R_t)$;
$\mathcal{N}_{t+1} \leftarrow \mathcal{N}_{\mathrm{new}}(\mathrm{B}^{r+1}\mathrm{LLM}(\mathrm{B}^{r}\mathrm{LLM}(\mathfrak{R}_*, \mathcal{P}_{t+1})))$;
$G_{t+1} \leftarrow \Pi_{\mathcal{V}}\big(G_t - \nabla\mathcal{R}_t + \mathcal{N}_{t+1}\big)$;
$\Sigma_{t+1} \leftarrow \mathsf{Emit}(G_{t+1}, V_{t+1}, \mathsf{Rel}_{t+1})$.

The pseudocode is short because the heavy work sits in the typed ledgers. Each line expands into a proof contract, a verifier trace, and a playbook update.

Proposition 31.1 (pseudocode correctness)**.** *The pseudocode satisfies the following invariants at every step:*

(i) $\mathcal{P}_{t+1}$ *contains* $\mathsf{Core}(\mathcal{P}_t)$ *(collapse guard);*

(ii) $G_{t+1} \in \Pi_{\mathcal{V}}(\mathfrak{S}_{25})$ *(verifier projection);*

(iii) $\Sigma_{t+1} \in \{\mathsf{THEOREM}, \mathsf{CANDIDATE}, \mathsf{CONDITIONAL}, \mathsf{COUNTEREXAMPLE}, \mathsf{INDEPENDENCE}, \mathsf{BUJ}\}$ *(typed emission);*

(iv) $\mathcal{J}_{t+1} \le \mathcal{J}_t + \max(0, \mathrm{Collapse}_{t+1} - \theta_{\mathrm{collapse}}) \cdot \kappa$ *(energy non-increase outside collapse).*

Proof. (i) follows from the collapse guard condition. (ii) follows from the explicit $\Pi_{\mathcal{V}}$ in the G_{t+1} update. (iii) follows from the definition of Emit, which maps every gate vector and verifier state to a typed status. (iv) follows from the module contraction lemmas and the explicit form of Δ_{mem}. □

32 Long-form verifier contract for submitted candidate runs

For every candidate run, the final artifact must include:

1. a statement of the axiom base $\mathcal{A}$;
2. a definition ledger for every new symbol;
3. a dependency graph for each theorem-like claim;
4. a map from each equation to a proof obligation;
5. a residue ledger with all unresolved stars;
6. a Module 24 divergence report;
7. a Module 25 star-to-one report;
8. a Module 26 playbook-delta report;
9. a repeated CMV report before and after every major machinery adoption;
10. a DIA report showing whether finite BrLLM escalation was triggered;
11. an Eta/Aleph report distinguishing unconditional-closure target from new-mathematics activation;
12. a Kappa-generator trace listing the adopted operator words and rejected alternatives;
13. a d'Alembert residue report BoxRes and an analytic-lift report AnalyticLift;
14. a release label chosen from theorem, counterexample, conditional theorem, independence/formulation, candidate, or BUJ;
15. a human/referee verification checklist.

This contract is the difference between a powerful conjecture engine and a confidence generator.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

33 Module-level contraction ledgers and local proof obligations

The following ledgers give a calculation-ready local law for every module. They are intentionally repetitive in structure, because repeated structure makes independent review easier: input, defect, contraction target, blocker and accepted output are always visible.

33.1 M1: Synthetic intake

Let $X_t^{(1)}$ be the local state of Module 1 and let $D_t^{(1)}$ be its defect. The local transition is

$$X_{t+1}^{(1)} = \Pi_{\mathcal{V}}^{(1)} \left(X_t^{(1)} + A_1 X_t^{(1)} + B_1 z_t^{(1)} - \nabla D_t^{(1)} + N_1(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(1)} + \alpha_1 \mathsf{Debt}_{t+1}^{(1)} + \beta_1 \mathsf{Residue}_{t+1}^{(1)} \le D_t^{(1)} + \alpha_1 \mathsf{Debt}_t^{(1)} + \beta_1 \mathsf{Residue}_t^{(1)} - \gamma_1 \left\| \Pi_{\mathrm{cert}} z_t^{(1)} \right\|^2 + \varepsilon_t^{(1)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(1)} = (\mathrm{module} = 1, \mathrm{state} = X_t^{(1)}, \mathrm{defect} = D_t^{(1)}, \mathrm{first_fail} = \mathsf{FirstFail}_1, \mathrm{repair} = \mathsf{Repair}_1).$$

The playbook receives the delta item $\delta b_t^{(1)} = \mathsf{Reflect}(R_t^{(1)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.2 M2: Type register

Let $X_t^{(2)}$ be the local state of Module 2 and let $D_t^{(2)}$ be its defect. The local transition is

$$X_{t+1}^{(2)} = \Pi_{\mathcal{V}}^{(2)} \left(X_t^{(2)} + A_2 X_t^{(2)} + B_2 z_t^{(2)} - \nabla D_t^{(2)} + N_2(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(2)} + \alpha_2 \mathsf{Debt}_{t+1}^{(2)} + \beta_2 \mathsf{Residue}_{t+1}^{(2)} \le D_t^{(2)} + \alpha_2 \mathsf{Debt}_t^{(2)} + \beta_2 \mathsf{Residue}_t^{(2)} - \gamma_2 \left\| \Pi_{\mathrm{cert}} z_t^{(2)} \right\|^2 + \varepsilon_t^{(2)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(2)} = (\mathrm{module} = 2, \mathrm{state} = X_t^{(2)}, \mathrm{defect} = D_t^{(2)}, \mathrm{first_fail} = \mathsf{FirstFail}_2, \mathrm{repair} = \mathsf{Repair}_2).$$

The playbook receives the delta item $\delta b_t^{(2)} = \mathsf{Reflect}(R_t^{(2)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.3 M3: Omega synthesis

Let $X_t^{(3)}$ be the local state of Module 3 and let $D_t^{(3)}$ be its defect. The local transition is

$$X_{t+1}^{(3)} = \Pi_{\mathcal{V}}^{(3)} \left(X_t^{(3)} + A_3 X_t^{(3)} + B_3 z_t^{(3)} - \nabla D_t^{(3)} + N_3(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(3)} + \alpha_3 \mathsf{Debt}_{t+1}^{(3)} + \beta_3 \mathsf{Residue}_{t+1}^{(3)} \le D_t^{(3)} + \alpha_3 \mathsf{Debt}_t^{(3)} + \beta_3 \mathsf{Residue}_t^{(3)} - \gamma_3 \left\| \Pi_{\mathrm{cert}} z_t^{(3)} \right\|^2 + \varepsilon_t^{(3)}.$$

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(3)} = (\text{module} = 3, \text{state} = X_t^{(3)}, \text{defect} = D_t^{(3)}, \text{first_fail} = \mathsf{FirstFail}_3, \text{repair} = \mathsf{Repair}_3).$$

The playbook receives the delta item $\delta b_t^{(3)} = \mathsf{Reflect}(R_t^{(3)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.4 M4: Adversarial derivative

Let $X_t^{(4)}$ be the local state of Module 4 and let $D_t^{(4)}$ be its defect. The local transition is

$$X_{t+1}^{(4)} = \Pi_{\mathcal{V}}^{(4)} \left(X_t^{(4)} + A_4 X_t^{(4)} + B_4 z_t^{(4)} - \nabla D_t^{(4)} + N_4(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(4)} + \alpha_4 \mathsf{Debt}_{t+1}^{(4)} + \beta_4 \mathsf{Residue}_{t+1}^{(4)} \le D_t^{(4)} + \alpha_4 \mathsf{Debt}_t^{(4)} + \beta_4 \mathsf{Residue}_t^{(4)} - \gamma_4 \left\| \Pi_{\text{cert}} z_t^{(4)} \right\|^2 + \varepsilon_t^{(4)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(4)} = (\text{module} = 4, \text{state} = X_t^{(4)}, \text{defect} = D_t^{(4)}, \text{first_fail} = \mathsf{FirstFail}_4, \text{repair} = \mathsf{Repair}_4).$$

The playbook receives the delta item $\delta b_t^{(4)} = \mathsf{Reflect}(R_t^{(4)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.5 M5: Xi enumeration

Let $X_t^{(5)}$ be the local state of Module 5 and let $D_t^{(5)}$ be its defect. The local transition is

$$X_{t+1}^{(5)} = \Pi_{\mathcal{V}}^{(5)} \left(X_t^{(5)} + A_5 X_t^{(5)} + B_5 z_t^{(5)} - \nabla D_t^{(5)} + N_5(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(5)} + \alpha_5 \mathsf{Debt}_{t+1}^{(5)} + \beta_5 \mathsf{Residue}_{t+1}^{(5)} \le D_t^{(5)} + \alpha_5 \mathsf{Debt}_t^{(5)} + \beta_5 \mathsf{Residue}_t^{(5)} - \gamma_5 \left\| \Pi_{\text{cert}} z_t^{(5)} \right\|^2 + \varepsilon_t^{(5)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(5)} = (\text{module} = 5, \text{state} = X_t^{(5)}, \text{defect} = D_t^{(5)}, \text{first_fail} = \mathsf{FirstFail}_5, \text{repair} = \mathsf{Repair}_5).$$

The playbook receives the delta item $\delta b_t^{(5)} = \mathsf{Reflect}(R_t^{(5)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.6 M6: Tri-branch attack

Let $X_t^{(6)}$ be the local state of Module 6 and let $D_t^{(6)}$ be its defect. The local transition is

$$X_{t+1}^{(6)} = \Pi_{\mathcal{V}}^{(6)} \left(X_t^{(6)} + A_6 X_t^{(6)} + B_6 z_t^{(6)} - \nabla D_t^{(6)} + N_6(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(6)} + \alpha_6 \mathsf{Debt}_{t+1}^{(6)} + \beta_6 \mathsf{Residue}_{t+1}^{(6)} \le D_t^{(6)} + \alpha_6 \mathsf{Debt}_t^{(6)} + \beta_6 \mathsf{Residue}_t^{(6)} - \gamma_6 \left\| \Pi_{\text{cert}} z_t^{(6)} \right\|^2 + \varepsilon_t^{(6)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(6)} = (\text{module} = 6, \text{state} = X_t^{(6)}, \text{defect} = D_t^{(6)}, \text{first_fail} = \mathsf{FirstFail}_6, \text{repair} = \mathsf{Repair}_6).$$

The playbook receives the delta item $\delta b_t^{(6)} = \mathsf{Reflect}(R_t^{(6)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

33.7 M7: Falsification arena

Let $X_t^{(7)}$ be the local state of Module 7 and let $D_t^{(7)}$ be its defect. The local transition is

$$X_{t+1}^{(7)} = \Pi_{\mathcal{V}}^{(7)} \left(X_t^{(7)} + A_7 X_t^{(7)} + B_7 z_t^{(7)} - \nabla D_t^{(7)} + N_7(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(7)} + \alpha_7 \mathsf{Debt}_{t+1}^{(7)} + \beta_7 \mathsf{Residue}_{t+1}^{(7)} \le D_t^{(7)} + \alpha_7 \mathsf{Debt}_t^{(7)} + \beta_7 \mathsf{Residue}_t^{(7)} - \gamma_7 \left\| \Pi_{\mathrm{cert}} z_t^{(7)} \right\|^2 + \varepsilon_t^{(7)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(7)} = (\mathrm{module} = 7, \mathrm{state} = X_t^{(7)}, \mathrm{defect} = D_t^{(7)}, \mathrm{first_fail} = \mathsf{FirstFail}_7, \mathrm{repair} = \mathsf{Repair}_7).$$

The playbook receives the delta item $\delta b_t^{(7)} = \mathsf{Reflect}(R_t^{(7)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.8 M8: Bridge compiler

Let $X_t^{(8)}$ be the local state of Module 8 and let $D_t^{(8)}$ be its defect. The local transition is

$$X_{t+1}^{(8)} = \Pi_{\mathcal{V}}^{(8)} \left(X_t^{(8)} + A_8 X_t^{(8)} + B_8 z_t^{(8)} - \nabla D_t^{(8)} + N_8(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(8)} + \alpha_8 \mathsf{Debt}_{t+1}^{(8)} + \beta_8 \mathsf{Residue}_{t+1}^{(8)} \le D_t^{(8)} + \alpha_8 \mathsf{Debt}_t^{(8)} + \beta_8 \mathsf{Residue}_t^{(8)} - \gamma_8 \left\| \Pi_{\mathrm{cert}} z_t^{(8)} \right\|^2 + \varepsilon_t^{(8)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(8)} = (\mathrm{module} = 8, \mathrm{state} = X_t^{(8)}, \mathrm{defect} = D_t^{(8)}, \mathrm{first_fail} = \mathsf{FirstFail}_8, \mathrm{repair} = \mathsf{Repair}_8).$$

The playbook receives the delta item $\delta b_t^{(8)} = \mathsf{Reflect}(R_t^{(8)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.9 M9: Barrier ledger

Let $X_t^{(9)}$ be the local state of Module 9 and let $D_t^{(9)}$ be its defect. The local transition is

$$X_{t+1}^{(9)} = \Pi_{\mathcal{V}}^{(9)} \left(X_t^{(9)} + A_9 X_t^{(9)} + B_9 z_t^{(9)} - \nabla D_t^{(9)} + N_9(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(9)} + \alpha_9 \mathsf{Debt}_{t+1}^{(9)} + \beta_9 \mathsf{Residue}_{t+1}^{(9)} \le D_t^{(9)} + \alpha_9 \mathsf{Debt}_t^{(9)} + \beta_9 \mathsf{Residue}_t^{(9)} - \gamma_9 \left\| \Pi_{\mathrm{cert}} z_t^{(9)} \right\|^2 + \varepsilon_t^{(9)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(9)} = (\mathrm{module} = 9, \mathrm{state} = X_t^{(9)}, \mathrm{defect} = D_t^{(9)}, \mathrm{first_fail} = \mathsf{FirstFail}_9, \mathrm{repair} = \mathsf{Repair}_9).$$

The playbook receives the delta item $\delta b_t^{(9)} = \mathsf{Reflect}(R_t^{(9)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

33.10 M10: Exact computation

Let $X_t^{(10)}$ be the local state of Module 10 and let $D_t^{(10)}$ be its defect. The local transition is

$$X_{t+1}^{(10)} = \Pi_{\mathcal{V}}^{(10)} \left(X_t^{(10)} + A_{10} X_t^{(10)} + B_{10} z_t^{(10)} - \nabla D_t^{(10)} + N_{10}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(10)} + \alpha_{10} \mathsf{Debt}_{t+1}^{(10)} + \beta_{10} \mathsf{Residue}_{t+1}^{(10)} \le D_t^{(10)} + \alpha_{10} \mathsf{Debt}_t^{(10)} + \beta_{10} \mathsf{Residue}_t^{(10)} - \gamma_{10} \left\| \Pi_{\mathrm{cert}} z_t^{(10)} \right\|^2 + \varepsilon_t^{(10)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(10)} = (\mathrm{module} = 10, \mathrm{state} = X_t^{(10)}, \mathrm{defect} = D_t^{(10)}, \mathrm{first_fail} = \mathsf{FirstFail}_{10}, \mathrm{repair} = \mathsf{Repair}_{10}).$$

The playbook receives the delta item $\delta b_t^{(10)} = \mathsf{Reflect}(R_t^{(10)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.11 M11: Residue sedimentation

Let $X_t^{(11)}$ be the local state of Module 11 and let $D_t^{(11)}$ be its defect. The local transition is

$$X_{t+1}^{(11)} = \Pi_{\mathcal{V}}^{(11)} \left(X_t^{(11)} + A_{11} X_t^{(11)} + B_{11} z_t^{(11)} - \nabla D_t^{(11)} + N_{11}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(11)} + \alpha_{11} \mathsf{Debt}_{t+1}^{(11)} + \beta_{11} \mathsf{Residue}_{t+1}^{(11)} \le D_t^{(11)} + \alpha_{11} \mathsf{Debt}_t^{(11)} + \beta_{11} \mathsf{Residue}_t^{(11)} - \gamma_{11} \left\| \Pi_{\mathrm{cert}} z_t^{(11)} \right\|^2 + \varepsilon_t^{(11)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(11)} = (\mathrm{module} = 11, \mathrm{state} = X_t^{(11)}, \mathrm{defect} = D_t^{(11)}, \mathrm{first_fail} = \mathsf{FirstFail}_{11}, \mathrm{repair} = \mathsf{Repair}_{11}).$$

The playbook receives the delta item $\delta b_t^{(11)} = \mathsf{Reflect}(R_t^{(11)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.12 M12: Sand-tile repair

Let $X_t^{(12)}$ be the local state of Module 12 and let $D_t^{(12)}$ be its defect. The local transition is

$$X_{t+1}^{(12)} = \Pi_{\mathcal{V}}^{(12)} \left(X_t^{(12)} + A_{12} X_t^{(12)} + B_{12} z_t^{(12)} - \nabla D_t^{(12)} + N_{12}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(12)} + \alpha_{12} \mathsf{Debt}_{t+1}^{(12)} + \beta_{12} \mathsf{Residue}_{t+1}^{(12)} \le D_t^{(12)} + \alpha_{12} \mathsf{Debt}_t^{(12)} + \beta_{12} \mathsf{Residue}_t^{(12)} - \gamma_{12} \left\| \Pi_{\mathrm{cert}} z_t^{(12)} \right\|^2 + \varepsilon_t^{(12)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(12)} = (\mathrm{module} = 12, \mathrm{state} = X_t^{(12)}, \mathrm{defect} = D_t^{(12)}, \mathrm{first_fail} = \mathsf{FirstFail}_{12}, \mathrm{repair} = \mathsf{Repair}_{12}).$$

The playbook receives the delta item $\delta b_t^{(12)} = \mathsf{Reflect}(R_t^{(12)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

33.13 M13: Cellular propagation

Let $X_t^{(13)}$ be the local state of Module 13 and let $D_t^{(13)}$ be its defect. The local transition is

$$X_{t+1}^{(13)} = \Pi_{\mathcal{V}}^{(13)} \left(X_t^{(13)} + A_{13} X_t^{(13)} + B_{13} z_t^{(13)} - \nabla D_t^{(13)} + N_{13}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(13)} + \alpha_{13} \mathsf{Debt}_{t+1}^{(13)} + \beta_{13} \mathsf{Residue}_{t+1}^{(13)} \le D_t^{(13)} + \alpha_{13} \mathsf{Debt}_t^{(13)} + \beta_{13} \mathsf{Residue}_t^{(13)} - \gamma_{13} \left\| \Pi_{\mathrm{cert}} z_t^{(13)} \right\|^2 + \varepsilon_t^{(13)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(13)} = (\mathrm{module} = 13, \mathrm{state} = X_t^{(13)}, \mathrm{defect} = D_t^{(13)}, \mathrm{first_fail} = \mathsf{FirstFail}_{13}, \mathrm{repair} = \mathsf{Repair}_{13}).$$

The playbook receives the delta item $\delta b_t^{(13)} = \mathsf{Reflect}(R_t^{(13)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.14 M14: Histogram diagnostic

Let $X_t^{(14)}$ be the local state of Module 14 and let $D_t^{(14)}$ be its defect. The local transition is

$$X_{t+1}^{(14)} = \Pi_{\mathcal{V}}^{(14)} \left(X_t^{(14)} + A_{14} X_t^{(14)} + B_{14} z_t^{(14)} - \nabla D_t^{(14)} + N_{14}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(14)} + \alpha_{14} \mathsf{Debt}_{t+1}^{(14)} + \beta_{14} \mathsf{Residue}_{t+1}^{(14)} \le D_t^{(14)} + \alpha_{14} \mathsf{Debt}_t^{(14)} + \beta_{14} \mathsf{Residue}_t^{(14)} - \gamma_{14} \left\| \Pi_{\mathrm{cert}} z_t^{(14)} \right\|^2 + \varepsilon_t^{(14)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(14)} = (\mathrm{module} = 14, \mathrm{state} = X_t^{(14)}, \mathrm{defect} = D_t^{(14)}, \mathrm{first_fail} = \mathsf{FirstFail}_{14}, \mathrm{repair} = \mathsf{Repair}_{14}).$$

The playbook receives the delta item $\delta b_t^{(14)} = \mathsf{Reflect}(R_t^{(14)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.15 M15: Ogive curve

Let $X_t^{(15)}$ be the local state of Module 15 and let $D_t^{(15)}$ be its defect. The local transition is

$$X_{t+1}^{(15)} = \Pi_{\mathcal{V}}^{(15)} \left(X_t^{(15)} + A_{15} X_t^{(15)} + B_{15} z_t^{(15)} - \nabla D_t^{(15)} + N_{15}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(15)} + \alpha_{15} \mathsf{Debt}_{t+1}^{(15)} + \beta_{15} \mathsf{Residue}_{t+1}^{(15)} \le D_t^{(15)} + \alpha_{15} \mathsf{Debt}_t^{(15)} + \beta_{15} \mathsf{Residue}_t^{(15)} - \gamma_{15} \left\| \Pi_{\mathrm{cert}} z_t^{(15)} \right\|^2 + \varepsilon_t^{(15)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(15)} = (\mathrm{module} = 15, \mathrm{state} = X_t^{(15)}, \mathrm{defect} = D_t^{(15)}, \mathrm{first_fail} = \mathsf{FirstFail}_{15}, \mathrm{repair} = \mathsf{Repair}_{15}).$$

The playbook receives the delta item $\delta b_t^{(15)} = \mathsf{Reflect}(R_t^{(15)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

33.16 M16: Historical transducer

Let $X_t^{(16)}$ be the local state of Module 16 and let $D_t^{(16)}$ be its defect. The local transition is

$$X_{t+1}^{(16)} = \Pi_{\mathcal{V}}^{(16)} \left(X_t^{(16)} + A_{16} X_t^{(16)} + B_{16} z_t^{(16)} - \nabla D_t^{(16)} + N_{16}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(16)} + \alpha_{16} \mathsf{Debt}_{t+1}^{(16)} + \beta_{16} \mathsf{Residue}_{t+1}^{(16)} \le D_t^{(16)} + \alpha_{16} \mathsf{Debt}_t^{(16)} + \beta_{16} \mathsf{Residue}_t^{(16)} - \gamma_{16} \left\| \Pi_{\mathrm{cert}} z_t^{(16)} \right\|^2 + \varepsilon_t^{(16)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(16)} = (\mathrm{module} = 16, \mathrm{state} = X_t^{(16)}, \mathrm{defect} = D_t^{(16)}, \mathrm{first_fail} = \mathsf{FirstFail}_{16}, \mathrm{repair} = \mathsf{Repair}_{16}).$$

The playbook receives the delta item $\delta b_t^{(16)} = \mathsf{Reflect}(R_t^{(16)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.17 M17: Proof contract

Let $X_t^{(17)}$ be the local state of Module 17 and let $D_t^{(17)}$ be its defect. The local transition is

$$X_{t+1}^{(17)} = \Pi_{\mathcal{V}}^{(17)} \left(X_t^{(17)} + A_{17} X_t^{(17)} + B_{17} z_t^{(17)} - \nabla D_t^{(17)} + N_{17}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(17)} + \alpha_{17} \mathsf{Debt}_{t+1}^{(17)} + \beta_{17} \mathsf{Residue}_{t+1}^{(17)} \le D_t^{(17)} + \alpha_{17} \mathsf{Debt}_t^{(17)} + \beta_{17} \mathsf{Residue}_t^{(17)} - \gamma_{17} \left\| \Pi_{\mathrm{cert}} z_t^{(17)} \right\|^2 + \varepsilon_t^{(17)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(17)} = (\mathrm{module} = 17, \mathrm{state} = X_t^{(17)}, \mathrm{defect} = D_t^{(17)}, \mathrm{first_fail} = \mathsf{FirstFail}_{17}, \mathrm{repair} = \mathsf{Repair}_{17}).$$

The playbook receives the delta item $\delta b_t^{(17)} = \mathsf{Reflect}(R_t^{(17)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.18 M18: Release audit

Let $X_t^{(18)}$ be the local state of Module 18 and let $D_t^{(18)}$ be its defect. The local transition is

$$X_{t+1}^{(18)} = \Pi_{\mathcal{V}}^{(18)} \left(X_t^{(18)} + A_{18} X_t^{(18)} + B_{18} z_t^{(18)} - \nabla D_t^{(18)} + N_{18}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(18)} + \alpha_{18} \mathsf{Debt}_{t+1}^{(18)} + \beta_{18} \mathsf{Residue}_{t+1}^{(18)} \le D_t^{(18)} + \alpha_{18} \mathsf{Debt}_t^{(18)} + \beta_{18} \mathsf{Residue}_t^{(18)} - \gamma_{18} \left\| \Pi_{\mathrm{cert}} z_t^{(18)} \right\|^2 + \varepsilon_t^{(18)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(18)} = (\mathrm{module} = 18, \mathrm{state} = X_t^{(18)}, \mathrm{defect} = D_t^{(18)}, \mathrm{first_fail} = \mathsf{FirstFail}_{18}, \mathrm{repair} = \mathsf{Repair}_{18}).$$

The playbook receives the delta item $\delta b_t^{(18)} = \mathsf{Reflect}(R_t^{(18)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

33.19 M19: Status emitter

Let $X_t^{(19)}$ be the local state of Module 19 and let $D_t^{(19)}$ be its defect. The local transition is

$$X_{t+1}^{(19)} = \Pi_{\mathcal{V}}^{(19)} \left(X_t^{(19)} + A_{19} X_t^{(19)} + B_{19} z_t^{(19)} - \nabla D_t^{(19)} + N_{19}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(19)} + \alpha_{19} \mathsf{Debt}_{t+1}^{(19)} + \beta_{19} \mathsf{Residue}_{t+1}^{(19)} \le D_t^{(19)} + \alpha_{19} \mathsf{Debt}_t^{(19)} + \beta_{19} \mathsf{Residue}_t^{(19)} - \gamma_{19} \left\| \Pi_{\mathrm{cert}} z_t^{(19)} \right\|^2 + \varepsilon_t^{(19)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(19)} = (\mathrm{module} = 19, \mathrm{state} = X_t^{(19)}, \mathrm{defect} = D_t^{(19)}, \mathrm{first_fail} = \mathsf{FirstFail}_{19}, \mathrm{repair} = \mathsf{Repair}_{19}).$$

The playbook receives the delta item $\delta b_t^{(19)} = \mathsf{Reflect}(R_t^{(19)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.20 M20: Six-gate synthesizer

Let $X_t^{(20)}$ be the local state of Module 20 and let $D_t^{(20)}$ be its defect. The local transition is

$$X_{t+1}^{(20)} = \Pi_{\mathcal{V}}^{(20)} \left(X_t^{(20)} + A_{20} X_t^{(20)} + B_{20} z_t^{(20)} - \nabla D_t^{(20)} + N_{20}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(20)} + \alpha_{20} \mathsf{Debt}_{t+1}^{(20)} + \beta_{20} \mathsf{Residue}_{t+1}^{(20)} \le D_t^{(20)} + \alpha_{20} \mathsf{Debt}_t^{(20)} + \beta_{20} \mathsf{Residue}_t^{(20)} - \gamma_{20} \left\| \Pi_{\mathrm{cert}} z_t^{(20)} \right\|^2 + \varepsilon_t^{(20)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(20)} = (\mathrm{module} = 20, \mathrm{state} = X_t^{(20)}, \mathrm{defect} = D_t^{(20)}, \mathrm{first_fail} = \mathsf{FirstFail}_{20}, \mathrm{repair} = \mathsf{Repair}_{20}).$$

The playbook receives the delta item $\delta b_t^{(20)} = \mathsf{Reflect}(R_t^{(20)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.21 M21: New mathematics calibration

Let $X_t^{(21)}$ be the local state of Module 21 and let $D_t^{(21)}$ be its defect. The local transition is

$$X_{t+1}^{(21)} = \Pi_{\mathcal{V}}^{(21)} \left(X_t^{(21)} + A_{21} X_t^{(21)} + B_{21} z_t^{(21)} - \nabla D_t^{(21)} + N_{21}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(21)} + \alpha_{21} \mathsf{Debt}_{t+1}^{(21)} + \beta_{21} \mathsf{Residue}_{t+1}^{(21)} \le D_t^{(21)} + \alpha_{21} \mathsf{Debt}_t^{(21)} + \beta_{21} \mathsf{Residue}_t^{(21)} - \gamma_{21} \left\| \Pi_{\mathrm{cert}} z_t^{(21)} \right\|^2 + \varepsilon_t^{(21)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(21)} = (\mathrm{module} = 21, \mathrm{state} = X_t^{(21)}, \mathrm{defect} = D_t^{(21)}, \mathrm{first_fail} = \mathsf{FirstFail}_{21}, \mathrm{repair} = \mathsf{Repair}_{21}).$$

The playbook receives the delta item $\delta b_t^{(21)} = \mathsf{Reflect}(R_t^{(21)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

33.22 M22: Ambient machine constructor

Let $X_t^{(22)}$ be the local state of Module 22 and let $D_t^{(22)}$ be its defect. The local transition is

$$X_{t+1}^{(22)} = \Pi_{\mathcal{V}}^{(22)} \left(X_t^{(22)} + A_{22} X_t^{(22)} + B_{22} z_t^{(22)} - \nabla D_t^{(22)} + N_{22}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(22)} + \alpha_{22} \mathsf{Debt}_{t+1}^{(22)} + \beta_{22} \mathsf{Residue}_{t+1}^{(22)} \le D_t^{(22)} + \alpha_{22} \mathsf{Debt}_t^{(22)} + \beta_{22} \mathsf{Residue}_t^{(22)} - \gamma_{22} \left\| \Pi_{\mathrm{cert}} z_t^{(22)} \right\|^2 + \varepsilon_t^{(22)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(22)} = (\mathrm{module} = 22, \mathrm{state} = X_t^{(22)}, \mathrm{defect} = D_t^{(22)}, \mathrm{first_fail} = \mathsf{FirstFail}_{22}, \mathrm{repair} = \mathsf{Repair}_{22}).$$

The playbook receives the delta item $\delta b_t^{(22)} = \mathsf{Reflect}(R_t^{(22)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.23 M23: Candidate proof generator

Let $X_t^{(23)}$ be the local state of Module 23 and let $D_t^{(23)}$ be its defect. The local transition is

$$X_{t+1}^{(23)} = \Pi_{\mathcal{V}}^{(23)} \left(X_t^{(23)} + A_{23} X_t^{(23)} + B_{23} z_t^{(23)} - \nabla D_t^{(23)} + N_{23}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(23)} + \alpha_{23} \mathsf{Debt}_{t+1}^{(23)} + \beta_{23} \mathsf{Residue}_{t+1}^{(23)} \le D_t^{(23)} + \alpha_{23} \mathsf{Debt}_t^{(23)} + \beta_{23} \mathsf{Residue}_t^{(23)} - \gamma_{23} \left\| \Pi_{\mathrm{cert}} z_t^{(23)} \right\|^2 + \varepsilon_t^{(23)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(23)} = (\mathrm{module} = 23, \mathrm{state} = X_t^{(23)}, \mathrm{defect} = D_t^{(23)}, \mathrm{first_fail} = \mathsf{FirstFail}_{23}, \mathrm{repair} = \mathsf{Repair}_{23}).$$

The playbook receives the delta item $\delta b_t^{(23)} = \mathsf{Reflect}(R_t^{(23)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.24 M24: Ricci divergence auditor

Let $X_t^{(24)}$ be the local state of Module 24 and let $D_t^{(24)}$ be its defect. The local transition is

$$X_{t+1}^{(24)} = \Pi_{\mathcal{V}}^{(24)} \left(X_t^{(24)} + A_{24} X_t^{(24)} + B_{24} z_t^{(24)} - \nabla D_t^{(24)} + N_{24}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(24)} + \alpha_{24} \mathsf{Debt}_{t+1}^{(24)} + \beta_{24} \mathsf{Residue}_{t+1}^{(24)} \le D_t^{(24)} + \alpha_{24} \mathsf{Debt}_t^{(24)} + \beta_{24} \mathsf{Residue}_t^{(24)} - \gamma_{24} \left\| \Pi_{\mathrm{cert}} z_t^{(24)} \right\|^2 + \varepsilon_t^{(24)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(24)} = (\mathrm{module} = 24, \mathrm{state} = X_t^{(24)}, \mathrm{defect} = D_t^{(24)}, \mathrm{first_fail} = \mathsf{FirstFail}_{24}, \mathrm{repair} = \mathsf{Repair}_{24}).$$

The playbook receives the delta item $\delta b_t^{(24)} = \mathsf{Reflect}(R_t^{(24)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

33.25 M25: Star-to-one unitization

Let $X_t^{(25)}$ be the local state of Module 25 and let $D_t^{(25)}$ be its defect. The local transition is

$$X_{t+1}^{(25)} = \Pi_{\mathcal{V}}^{(25)} \left(X_t^{(25)} + A_{25} X_t^{(25)} + B_{25} z_t^{(25)} - \nabla D_t^{(25)} + N_{25}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(25)} + \alpha_{25} \mathsf{Debt}_{t+1}^{(25)} + \beta_{25} \mathsf{Residue}_{t+1}^{(25)} \le D_t^{(25)} + \alpha_{25} \mathsf{Debt}_t^{(25)} + \beta_{25} \mathsf{Residue}_t^{(25)} - \gamma_{25} \left\| \Pi_{\mathrm{cert}} z_t^{(25)} \right\|^2 + \varepsilon_t^{(25)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(25)} = (\mathrm{module} = 25, \mathrm{state} = X_t^{(25)}, \mathrm{defect} = D_t^{(25)}, \mathrm{first_fail} = \mathsf{FirstFail}_{25}, \mathrm{repair} = \mathsf{Repair}_{25}).$$

The playbook receives the delta item $\delta b_t^{(25)} = \mathsf{Reflect}(R_t^{(25)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

33.26 M26: Context playbook

Let $X_t^{(26)}$ be the local state of Module 26 and let $D_t^{(26)}$ be its defect. The local transition is

$$X_{t+1}^{(26)} = \Pi_{\mathcal{V}}^{(26)} \left(X_t^{(26)} + A_{26} X_t^{(26)} + B_{26} z_t^{(26)} - \nabla D_t^{(26)} + N_{26}(\mathcal{P}_t) \right).$$

The accepted contraction condition is

$$D_{t+1}^{(26)} + \alpha_{26} \mathsf{Debt}_{t+1}^{(26)} + \beta_{26} \mathsf{Residue}_{t+1}^{(26)} \le D_t^{(26)} + \alpha_{26} \mathsf{Debt}_t^{(26)} + \beta_{26} \mathsf{Residue}_t^{(26)} - \gamma_{26} \left\| \Pi_{\mathrm{cert}} z_t^{(26)} \right\|^2 + \varepsilon_t^{(26)}.$$

If the inequality fails, the module is not allowed to erase the attempt. It emits a typed residue

$$R_t^{(26)} = (\mathrm{module} = 26, \mathrm{state} = X_t^{(26)}, \mathrm{defect} = D_t^{(26)}, \mathrm{first_fail} = \mathsf{FirstFail}_{26}, \mathrm{repair} = \mathsf{Repair}_{26}).$$

The playbook receives the delta item $\delta b_t^{(26)} = \mathsf{Reflect}(R_t^{(26)})$, but the delta can be merged only if it keeps the core-failure retention constraint. Hence the local module cannot improve the global score by deleting its own failure trace.

Theorem 33.1 (summed module contraction)**.** *If each module satisfies its local contraction condition with summable errors $\sum_{t,i} \varepsilon_t^{(i)} < \infty$, then the total energy*

$$\mathcal{J}_t^{\mathrm{total}} = \sum_{i=1}^{26} \omega_i \left(D_t^{(i)} + \alpha_i \mathsf{Debt}_t^{(i)} + \beta_i \mathsf{Residue}_t^{(i)} \right) + \lambda \, \mathrm{Collapse}_t$$

either reaches a verifier-accepted stratum or converges to a nonzero stable obstruction ledger.

Proof. Sum the twenty-six inequalities with positive weights. The certified impulse terms are nonnegative and the errors are summable, so the total energy cannot decrease indefinitely without approaching either a release stratum or a state in which no module has an accepted descent direction. In the latter case the failed directions are retained as residue bullets by Module 26, so the terminal object is a stable obstruction ledger rather than a silent disappearance of the proof gap. □

34 Expanded gate calculus, circuit library and Module 26 import

This section expands the engine beyond the compact body above. The purpose is to keep the mathematical specification complete enough that every circuit in the architecture can be read as an implementable operator. The central rule is that a conjecture attempt is not stored as a final paragraph. It is stored as a typed sequence of gates, residues, maps, failures, formal debts and context deltas. The ACE source paper motivates this precisely: detailed playbooks retain task-specific knowledge, while monolithic rewriting can collapse accumulated knowledge into short summaries and reduce performance (Zhang et al., 2026). VSI therefore treats memory as a proof object.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

The local equations are not confidence scores; they are typed obligations whose failure creates residue rather than rhetorical proof strength.

All constants in this part are required to be uniform over the declared domain; a bound depending on a hidden cutoff is treated as scope loss.

34.1 ACE-to-VSI theorem-engine translation

The source agentic-context paper isolates three roles: a Generator that produces trajectories, a Reflector that extracts lessons from successes and failures, and a Curator that merges structured delta items. In VSI these become

$$\mathsf{Gen}_{\mathrm{VSI}} \longrightarrow \mathsf{Reflect}_{\mathrm{VSI}} \longrightarrow \mathsf{Curate}_{\mathrm{VSI}}.$$

The Generator produces an attempted proof trajectory

$$\tau_t = (G_t, \mathfrak{R}_t, \mathcal{B}_t, \mathcal{N}_t, \Pi_{\mathcal{V},t}),$$

where G_t is the current gate vector, $\mathfrak{R}_t$ the residue bundle, $\mathcal{B}_t$ the BrLLM branch state, $\mathcal{N}_t$ the active new-mathematics machine, and $\Pi_{\mathcal{V},t}$ the verifier projection. The Reflector extracts lessons

$$\mathsf{Reflect}_{\mathrm{VSI}}(\tau_t) = \{\delta b_1, \dots, \delta b_m\},$$

not as a summary of the whole run, but as localized delta context items. The Curator integrates them by

$$\mathcal{P}_{t+1} = \mathsf{Merge}\left(\mathcal{P}_t, \{\delta b_1, \dots, \delta b_m\}\right),$$

subject to a collapse guard. Each bullet has the typed record

$$b_k = (\mathrm{id}_k, \mathrm{gate}_k, \mathrm{claim}_k, \mathrm{useful}_k, \mathrm{harmful}_k, \mathrm{source}_k, \mathrm{verifier}_k, \mathrm{status}_k).$$

The collapse metric is

$$\mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) = 1 - \frac{|\mathrm{Core}(\mathcal{P}_t) \cap \mathrm{Core}(\mathcal{P}_{t+1})|}{|\mathrm{Core}(\mathcal{P}_t)|}.$$

If $\mathrm{Collapse} > \theta_{\mathrm{collapse}}$, the update is rejected. Thus a proof attempt cannot improve the paper state by deleting the exact reason it failed.

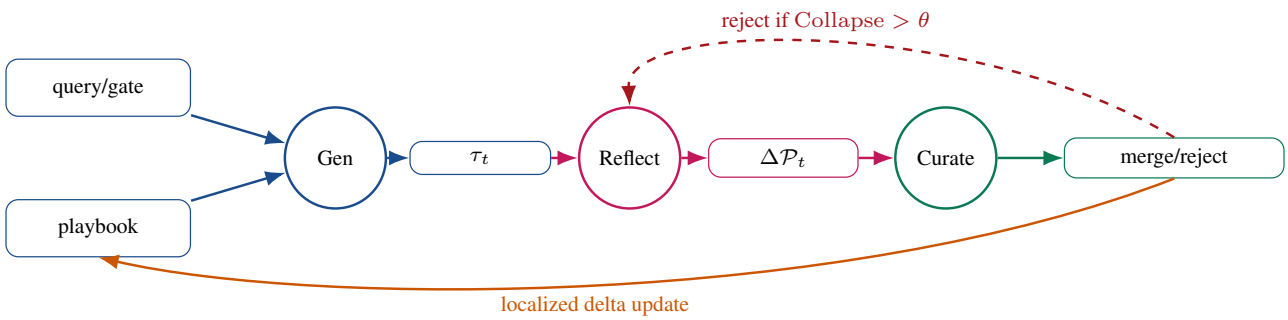


Figure 13: ACE-to-VSI Module 26 circuit. Generator, Reflector and Curator are imported as proof-engine operators. The merge operation is deterministic and localized; a full-memory rewrite is not permitted.

The grow-and-refine rule is

$$\mathcal{P}_{t+1} = \mathsf{Dedup} \circ \mathsf{Refine} \circ \mathsf{Append}(\mathcal{P}_t, \Delta\mathcal{P}_t), \qquad \Delta\mathcal{P}_t = \mathsf{Reflect}_{\mathrm{VSI}}(\tau_t).$$

Each bullet carries counters

$$h_k^{t+1} = h_k^t + \mathbf{1}\{b_k \text{ helped a verifier}\}, \qquad r_k^{t+1} = r_k^t + \mathbf{1}\{b_k \text{ caused a false route}\},$$

and utility

$$U(b_k) = \alpha h_k - \beta r_k + \gamma \mathrm{Reuse}(b_k) - \lambda \mathrm{Redundancy}(b_k).$$

Low-utility bullets are demoted rather than deleted,

$$b_k : \mathsf{active} \longrightarrow \mathsf{archive},$$

because a bridge that fails before a new invariant is discovered may become useful after a surrounding machine is built.

Whenever an object is generated, the engine stores its type trace, verifier trace and failure trace, so the run remains reproducible.

34.2 Module 20 six-gate operator family

Every conjecture attempt is pushed through six gates

$$\mathcal{S}_6 = (C_{\text{orbit}}, C_{\text{upper}}, C_{\text{lower}}, C_{\text{witness}}, C_{\text{barrier}}, C_{\text{formal}}).$$

Their numeric state is

$$G_i = (g_1, \dots, g_6), \qquad g_j \in \{0, 1, *\} \quad \text{or} \quad s_j \in [0, 1].$$

The associated defect vector is

$$\Delta(G) = \big(\Delta_{\text{orbit}}, \Delta_{\text{upper}}, \Delta_{\text{lower}}, \Delta_{\text{witness}}, \Delta_{\text{barrier}}, \Delta_{\text{formal}}\big),$$

with

$$\Delta_j = a_j \mathsf{Gap}_j + b_j \mathsf{Residue}_j + c_j\, \text{MapDef}_j + d_j \text{BridgeLoss}_j + e_j \text{FormalDebt}_j + f_j \text{Barrier}_j.$$

Gate saturation is

$$s_j = e^{-\Delta_j}, \qquad \mathcal{M}_*(G) = \sum_{j : g_j = *} w_j (1 - s_j)^2.$$

The six-gate block is closed only if

$$\mathcal{M}_*(G) + \sum_j \Delta_j + \mathsf{Risk}(G) + \mathsf{Debt}(G) = 0, \qquad \mathsf{Rel}(G) = 1.$$

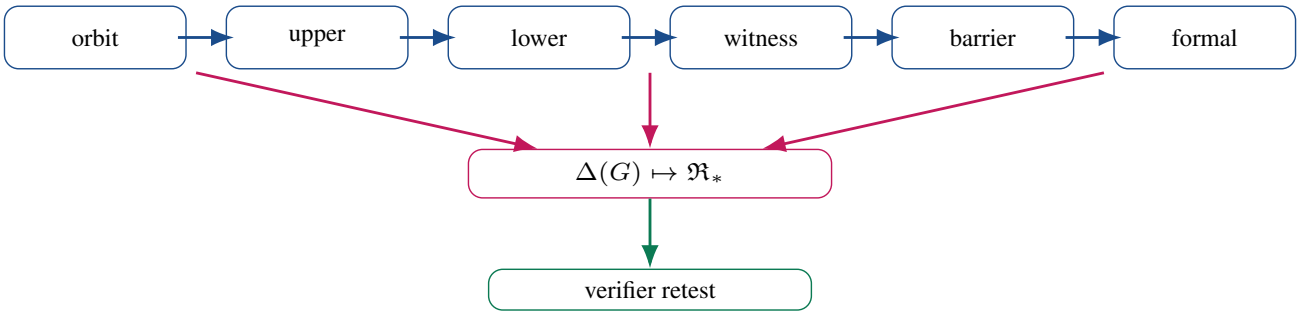


Figure 14: Six-gate residue synthesizer. The gates represent transfer fidelity, upper-envelope control, lower-obstruction construction, witness semantics, barrier survival and formal-kernel replay.

34.3 Dot-sand mapping and off-map arrows

Let two idea-dishes be

$$D_a = (X_a, R_a, \mu_a), \qquad D_b = (X_b, R_b, \mu_b),$$

and let $F_{ab} : D_a \to D_b$ be a proposed bridge. The dot-sand map defect is

$$\text{MapDef}(F_{ab}) = \alpha\, \text{BijDef}(F_{ab}) + \beta\, \text{InjDef}(F_{ab}) + \gamma\, \text{SurjDef}(F_{ab}) + \delta\, \text{OffMap}(F_{ab}),$$

where

$$\text{OffMap}(F_{ab}) = \mu_a\{x \in X_a : F_{ab}(x) \text{ is undefined, nonunique, or verifier-incoherent}\}.$$

The off-map set is

$$\Omega_{\text{off}}(F_{ab}) = \{x \in X_a : F_{ab}(x) \notin X_b \text{ coherently}\},$$

and it is not discarded. It becomes the input of the second closure attempt.

A computation may supply evidence, but it becomes proof material only after its quantifiers, limiting operations and exceptional sets are analytically controlled.

CMV means that neighboring modules must commute with verifier projection; a local pass without this commutation is not a global pass.

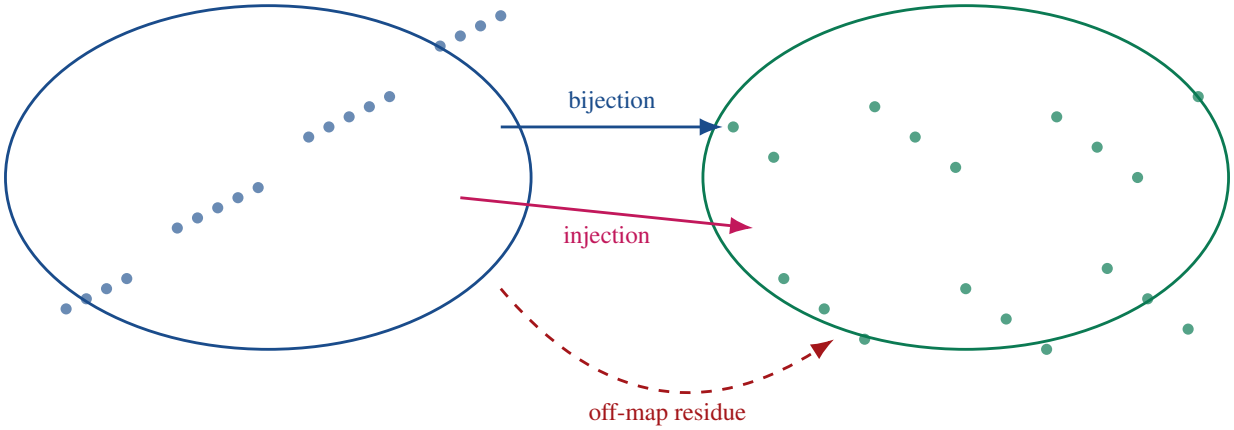


Figure 15: Dot-sand dish map. Correct arrows behave as bijections, injections or surjections between idea spaces; incoherent arrows shoot out as off-map residue and trigger a second closure attempt.

34.4 Module 24 Ricci-vector divergence and central chord test

For calibrated conjectures with a known closure, such as Fermat and Poincare, let v_0 be the initial proof vector, v_t an attempted transported closure vector, and v_∞ the verified closure vector. Define

$$\mathcal{R}_t = \frac{\|v_t - v_\infty\|^2}{1 + \|v_0\|^2 + \|v_\infty\|^2} + \lambda \mathcal{M}_*(G_i) + \rho\, \mathrm{OffMap}(F).$$

The Ricci-Hamiltonian correction is

$$\frac{d}{dt} v_t = -\nabla_{v_t} \mathcal{R}_t,$$

and the central chord defect is

$$\mathrm{ChordDef}(G_i) = \sum_{j=1}^{6} \|v_j - C_*\|^2 .$$

A correct attempted closure is characterized by

$$\lim_{t\to\infty} \mathcal{R}_t = 0, \qquad \mathrm{ChordDef}(G_i) = 0, \qquad \mathsf{Rel}(G_i) = 1.$$

If $\mathcal{R}_t$ stalls away from zero, the largest component of $\nabla \mathcal{R}_t$ selects the next target of Module 25.

34.5 Module 25 back-loop to BrLLM and new mathematical machinery

For

$$G_i = (1, 1, s_3, s_4, s_5, s_6), \qquad s_j \in [0, 1],$$

define the unresolved star-residue of a gate by

$$\mathfrak{R}_j = (\Delta_j, \Omega_{\mathrm{off},j}, \mathrm{MapDef}_j, \mathrm{BridgeLoss}_j, \mathrm{Barrier}_j, \mathrm{FormalDebt}_j),$$

and aggregate it as

$$\mathfrak{R}_* = \bigoplus_{j : s_j < 1} \mathfrak{R}_j.$$

The back-loop operator is

$$\mathcal{B}_{\mathrm{back}}(\mathfrak{R}_*) = \Pi_{\mathcal{V}} \circ \mathcal{N}_{\mathrm{new}} \circ \mathrm{B}^{r+1}\mathrm{LLM} \circ \mathrm{B}^{r}\mathrm{LLM} \circ \mathcal{D}_{\mathrm{res}}(\mathfrak{R}_*).$$

The aggressive instruction is mathematical, not stylistic: every positive defect must be named, measured and routed to a repair operator.

The notation in this subsection is intended as a machine-checkable ledger: variables, maps, gates and residues are kept separate to prevent hidden assumptions.

The gate update is

$$G_i^{(t+1)} = \mathsf{Retest}\left(G_i^{(t)}, \mathcal{B}_{\mathrm{back}}(\mathfrak{R}_*^{(t)})\right).$$

Ordinary repair is not enough when a failure persists. Define

$$\mathsf{Persist}_j(t) = \sum_{\tau=0}^{t} \mathbf{1}\{s_j^{(\tau)} < 1\} e^{-\alpha(t-\tau)}.$$

If $\mathsf{Persist}_j(t) > \Theta_j$, ordinary machinery is declared insufficient and VSI activates

$$\mathcal{N}_{\mathrm{new}}(j) = \operatorname*{argmin}_{\mathcal{M}\in\mathfrak{Mach}} \left[\mathrm{BridgeLoss}_j(\mathcal{M}) + \mathrm{MapDef}_j(\mathcal{M}) + \mathrm{Barrier}_j(\mathcal{M}) + \mathrm{FormalDebt}_j(\mathcal{M})\right].$$

The full kernel equation is

$$\boxed{G_i^{(t+1)} = \Pi_{\mathcal{V}}\left[G_i^{(t)} - \nabla\mathcal{R}_t + \mathcal{N}_{\mathrm{new}}\left(\mathrm{B}^{r+1}\mathrm{LLM}\left(\mathrm{B}^{r}\mathrm{LLM}(\mathfrak{R}_*^{(t)}, \mathcal{P}_{t+1})\right)\right)\right].}$$

34.6 Module 26 full update equation and memory stability

The Module 26 equation is

$$\boxed{\mathcal{P}_{t+1} = \mathsf{Curate}_{\mathrm{VSI}}\left[\mathcal{P}_t, \mathsf{Reflect}_{\mathrm{VSI}}\left(\mathsf{Gen}_{\mathrm{VSI}}(G_t, \mathfrak{R}_*^t, \mathcal{N}_{\mathrm{new}}^t)\right)\right].}$$

Module 25 then uses the updated playbook:

$$G_i^{t+1} = \Pi_{\mathcal{V}}\left[G_i^t - \nabla\mathcal{R}_t + \mathcal{N}_{\mathrm{new}}\left(\mathrm{B}^{r+1}\mathrm{LLM}\left(\mathrm{B}^{r}\mathrm{LLM}(\mathfrak{R}_*^t, \mathcal{P}_{t+1})\right)\right)\right].$$

Therefore a failed attempt becomes

$$\text{failed attempt} \longrightarrow \text{playbook bullet} \longrightarrow \text{curated memory} \longrightarrow \text{better second attempt.}$$

The combined terminal chain is

$$\begin{aligned}&\text{Module 24 divergence audit} \longrightarrow \text{Module 25 star-to-one back-loop}\\ &\longrightarrow \text{Module 26 context-playbook memory} \longrightarrow \text{second closure attempt.}\end{aligned}$$

This is the core improvement: VSI does not only generate new mathematics; it preserves why previous mathematics failed.

34.7 Cellular automaton of proof status

Let each proof cell be

$$c_v(t) = (\sigma_v(t), h_v(t), r_v(t), u_v(t), f_v(t)),$$

where $\sigma_v \in \{C, R, U, F\}$ records certified, residue, unknown or failed. The update is

$$\sigma_v(t+1) = \Phi\left(\sigma_v(t), \{\sigma_u(t) : u \sim v\}, V_v(t), \Delta_v(t), \mathcal{P}_t\right).$$

A certified state is absorbing only if no neighboring failed bridge depends on it:

$$\sigma_v = C \ \wedge\ \forall u \sim v\ e_{uv} \notin \mathrm{scope}(v) \quad\Longrightarrow\quad \sigma_v(t+1) = C.$$

A residue state topples when

$$h_v(t) + \sum_{u\sim v} \omega_{uv} r_u(t) > \theta_v.$$

The proof/refutation/independence triad remains active until one branch is certified and the competing branches are blocked by explicit verifier factors.

A bridge theorem is accepted only when it preserves hypotheses, target scope and formal encodability; otherwise it contributes to ScopeLoss.

This updates the local burden by

$$h_v(t+1) = h_v(t) - \theta_v + \sum_{u \sim v} \omega_{uv} \mathbf{1}\{h_u(t) > \theta_u\}.$$

The system cannot silently hide a failed leaf because F cells create mandatory playbook bullets.

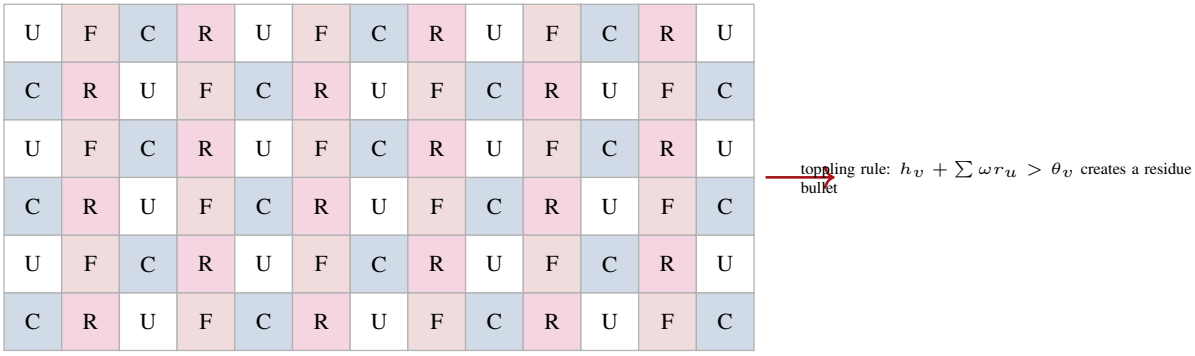


Figure 16: Cellular proof-status automaton. Certified, residue, unknown and failed cells evolve under local verifier and burden rules.

35 Expanded derivation of the global contraction law

The derivation treats the full engine as a coupled descent system rather than as isolated modules. Each local state $X_t^{(i)}$ contributes a defect energy $D_t^{(i)}$, but the shared playbook and verifier lattice introduce cross-terms. Hence global contraction requires both local inequalities and a bound on coupling. The guiding inequality is $\Delta \mathcal{J}_t \le -\sum_i c_i \|z_i\|^2 + C_{\text{cross}} \|z\| + \varepsilon_t$, so the certified quadratic descent must dominate cross-module leakage. The total engine energy is

$$\begin{aligned}\mathcal{J}_t = \sum_{i=1}^{26} \omega_i D_t^{(i)} + \alpha \mathsf{Debt}_t + \beta \mathsf{Residue}_t + \eta \mathcal{M}_*(G_t) + \rho \,\mathrm{MapDef}_t + \kappa \,\mathrm{Collapse}_t \\ + \lambda_{\square} \mathsf{BoxRes}_t + \lambda_{\mathsf{CMV}}(1 - \mathsf{CMV}_t) + \lambda_{\text{lift}}(1 - \mathsf{AnalyticLift}_t) - \zeta \mathsf{Cert}_t.\end{aligned}$$

Each module has a local state $X_t^{(i)}$ and a certified descent vector $z_t^{(i)}$. The admissible update is

$$X_{t+1}^{(i)} = \Pi_{\mathcal{V}}^{(i)} \left(X_t^{(i)} + A_i X_t^{(i)} + B_i z_t^{(i)} - \nabla D_t^{(i)} + N_i(\mathcal{P}_t) \right).$$

The matrix A_i captures internal state dynamics (e.g., momentum terms in the repair search); B_i is the injection map from the certified descent vector space into the module state space; $N_i(\mathcal{P}_t)$ is the playbook-conditioned correction, which evaluates the retrieval score $\mathsf{Retrieve}(b, C)$ for all active bullets and injects the top-scoring bullet's claim into the update direction. The three-term structure $(A_i X_t^{(i)},\, B_i z_t^{(i)},\, N_i(\mathcal{P}_t))$ generalizes the classic primal-dual decomposition: the first term is the state advance, the second is the certified correction, and the third is the contextual prior from accumulated memory.

The accepted descent inequality is

$$D_{t+1}^{(i)} + a_i \mathsf{Debt}_{t+1}^{(i)} + b_i \mathsf{Residue}_{t+1}^{(i)} \le D_t^{(i)} + a_i \mathsf{Debt}_t^{(i)} + b_i \mathsf{Residue}_t^{(i)} - c_i \left\| \Pi_{\text{cert}} z_t^{(i)} \right\|^2 + \varepsilon_t^{(i)}.$$

Lemma 35.1 (individual module descent implies global descent)**.** *If each module satisfies its local descent inequality and the cross-module coupling is bounded by* $\sum_{i \neq j} |\langle z_t^{(i)}, \nabla D_t^{(j)} \rangle| \le C_{\text{cross}} \sum_i \left\| z_t^{(i)} \right\|$*, then*

$$\mathcal{J}_{t+1} \le \mathcal{J}_t - \sum_{i=1}^{26} \omega_i c_i \left\| \Pi_{\text{cert}} z_t^{(i)} \right\|^2 + \sum_{i=1}^{26} \omega_i \varepsilon_t^{(i)} + C_{\text{cross}} \sqrt{26} \left(\sum_i \left\| z_t^{(i)} \right\|^2 \right)^{1/2}.$$

The contraction law is a Lyapunov audit: descent is accepted only when module-local repair and cross-module coupling combine into a non-positive global energy increment.

The playbook prevents context collapse by retaining failed attempts as reusable obstruction data rather than compressing them into vague summaries.

Proof. Sum the 26 module inequalities with weights ω_i. The terms $D_t^{(i)}, \mathsf{Debt}_t^{(i)}, \mathsf{Residue}_t^{(i)}$ sum to the global energy. The coupling between modules through shared playbook inputs $N_i(\mathcal{P}_t)$ introduces the cross-module term, bounded by the Cauchy–Schwarz inequality. When coupling is small ($C_{\mathrm{cross}} \ll \min_i \omega_i c_i$), the individual descent terms dominate and the global energy decreases. □

Summing gives

$$\mathcal{J}_{t+1} \le \mathcal{J}_t - \sum_{i=1}^{26} \omega_i c_i \left\| \Pi_{\mathrm{cert}} z_t^{(i)} \right\|^2 + \sum_{i=1}^{26} \omega_i \varepsilon_t^{(i)} + \Delta_{\mathrm{mem}}(t),$$

where the memory term satisfies

$$\Delta_{\mathrm{mem}}(t) = \kappa \left[\mathrm{Collapse}(\mathcal{P}_t, \mathcal{P}_{t+1}) - \theta_{\mathrm{collapse}} \right]_+ .$$

Because Module 26 rejects updates with positive memory violation, $\Delta_{\mathrm{mem}}(t) = 0$ for accepted transitions. If $\sum_{t,i} \varepsilon_t^{(i)} < \infty$, then the certified descent terms are summable.

Theorem 35.2 (global contraction with memory guard)**.** *Under the summability assumption $\sum_{t\ge 0} \sum_{i=1}^{26} \omega_i \varepsilon_t^{(i)} < \infty$ and the Module 26 collapse guard, the engine satisfies*

$$\sum_{t=0}^{\infty} \sum_{i=1}^{26} \omega_i c_i \left\| \Pi_{\mathrm{cert}} z_t^{(i)} \right\|^2 \le \mathcal{J}_0 + \sum_{t,i} \omega_i \varepsilon_t^{(i)} < \infty.$$

In particular, $\left\| \Pi_{\mathrm{cert}} z_t^{(i)} \right\|^2 \to 0$ *for all i along a density-one subsequence.*

Proof. Telescope the energy inequality over $t = 0, 1, \ldots, T$ and use the collapse guard to eliminate $\Delta_{\mathrm{mem}}(t)$:

$$\sum_{t=0}^{T} \sum_{i} \omega_i c_i \left\| \Pi_{\mathrm{cert}} z_t^{(i)} \right\|^2 \le \mathcal{J}_0 - \mathcal{J}_{T+1} + \sum_{t,i} \omega_i \varepsilon_t^{(i)}.$$

Since $\mathcal{J}$ is bounded below, taking $T \to \infty$ gives the finite sum. The density-one subsequence follows from Cesàro's theorem applied to the non-negative summable sequence. □

Thus either the run reaches a verifier stratum or all certified directions vanish and the output is a stable obstruction ledger. This is the mathematical reason the engine has two honest terminal states: verified closure or explicit obstruction.

A Plain footnote AI-agent protocol to reuse without re-prompting

Canonical prompt-free conjecture protocol.[4]

B Appendix protocol: candidate proof package schema

A conjecture-specific output must use the following schema:

$$\begin{aligned}\mathsf{Package}(C) = \big(&\mathsf{Title}, \mathsf{Claim}, \mathsf{Axioms}, \mathsf{KnownBarriers}, G, \Delta, \mathcal{M}_*, N_{\mathrm{new}},\\ &\mathsf{CMV}, \mathsf{DIA}, \mathsf{AlephAct}, \mathsf{KappaGen}, \mathsf{BoxRes}, \mathsf{AnalyticLift}, \mathcal{P}, \mathsf{ProofText},\\ &\mathsf{FormalTarget}, \mathsf{RefereeChecklist}\big).\end{aligned}$$

A zero residue claim is meaningful only relative to the declared axiom base $\mathcal{A}$ and the stated verifier lattice.

[4]For any conjecture C, parse $C = (\mathcal{D}, \mathcal{L}, \mathcal{A}, P, Q, \mathcal{E}_0, \mathcal{B}_0, \mathcal{R}_0)$ with all quantifiers retained; open proof, refutation and independence branches; attempt direct proof; introduce ambient machines, flows, invariants, finite certificates, category bridges, model-theoretic obstructions or formal kernels when direct proof fails; emit CAND-STRONG only with $V_{\mathrm{model}} = 1$, repeated CMV and named residual obstructions; convert each $*$ into Δ_j, Δ_j^{agg}, $s_j = e^{-\Delta_j}$ and $N_{\mathrm{new}}^{(j)}$; apply Modules 24–26 to route dominant divergence through DIA, Aleph and Kappa; reject computation-only conclusions until $\partial_{\aleph} \eta_{\mu_{\mathrm{comp}}, \nu_{\mathrm{ana}}} > 0$, $\mathsf{AnalyticLift} = 1$ and $\mathsf{BoxRes} = 0$; store failures, repairs, CMV reports, DIA measurements, Aleph activations and Kappa traces in Module 26; submit the package to formal, human and referee verification before release.

This part is read as a verifier contract: every symbol introduced here must later have a domain, dependency edge and release gate.

The release classifier is

$$\mathsf{Classify}(C) = \begin{cases} \text{ReleasedTheorem}, & \mathsf{Rel} = 1,\ \mathsf{EtaUC} = 1, \\ \text{CAND-STRONG}, & V_{\text{model}} = 1,\ \mathcal{M}_* > 0,\ V_{\text{human}} V_{\text{referee}} = 0, \\ \text{Conditional}, & \mathsf{Rel} = 1 \text{ only under extra hypothesis } H, \\ \text{BUJ}, & \text{stable obstruction ledger with no verified bridge.} \end{cases}$$

The template must include both a subtitle and an appendix statement carrying this schema, so that the next AI agent inherits the entire route without fresh prompting.

C Acknowledgements

The author is thankful to the EGSPL community for support, research environment and discussions. The paper also acknowledges the mathematical communities surrounding formal verification, proof assistants, complexity theory, geometric analysis, arithmetic geometry, analytic number theory, and context-engineering research, whose methods provide the verification discipline used here.

D Funding

No dedicated external funding is claimed for the preparation of this manuscript.

E Competing interests

The author declares no financial competing interests connected to this manuscript.

F Data and code availability

No private dataset is required for the mathematical definitions in this paper. A software implementation may store VSI run ledgers, playbook bullets, verifier outputs, and proof-state tensors in ordinary JSON, SQLite or theorem-prover project formats. Source code, if released, should include the exact run ledger, random seeds, verifier versions, and formal-library hashes.

References

Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou and K. Olukotun. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. ICLR conference paper, 2026.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao. ReAct: Synergizing reasoning and acting in language models. ICLR, 2023.

N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan and S. Yao. Reflexion: language agents with verbal reinforcement learning. NeurIPS, 2023.

M. Suzgun, M. Yuksekgonul, F. Bianchi, D. Jurafsky and J. Zou. Dynamic Cheatsheet: test-time learning with adaptive memory. Public preprint, 2025.

A. Wiles. Modular elliptic curves and Fermat's Last Theorem. Annals of Mathematics 141 (1995), 443–551. doi: 10.2307/2118559.

R. Taylor and A. Wiles. Ring-theoretic properties of certain Hecke algebras. Annals of Mathematics 141 (1995), 553–572. doi: 10.2307/2118560.

R. S. Hamilton. Three-manifolds with positive Ricci curvature. Journal of Differential Geometry 17 (1982), 255–306. doi: 10.4310/jdg/1214436922.

G. Perelman. The entropy formula for the Ricci flow and its geometric applications. Public preprint, 2002.

G. Perelman. Ricci flow with surgery on three-manifolds. Public preprint, 2003.

J. Morgan and G. Tian. Ricci Flow and the Poincare Conjecture. Clay Mathematics Monographs, American Mathematical Society, 2007.

P. Deligne. La conjecture de Weil I. Publications Mathematiques de l'IHES 43 (1974), 273–307. doi: 10.1007/BF02684373.

T. C. Hales, M. Adams, G. Bauer, T. D. Dang, J. Harrison, H. Le Truong, C. Kaliszyk, V. Magron, S. McLaughlin, T. T. Nguyen, Q. T. Nguyen, T. Nipkow, S. Obua, J. Pleso, J. Rute, A. Solovyev, A. H. T. Ta and T. N. Tran. A formal proof of the Kepler conjecture. Forum of Mathematics, Pi 5 (2017), e2. doi: 10.1017/fmp.2017.1.

A. Marcus, D. Spielman and N. Srivastava. Interlacing families II: Mixed characteristic polynomials and the Kadison-Singer problem. Annals of Mathematics 182 (2015), 327–350. doi: 10.4007/annals.2015.182.1.8.

M. S. Viazovska. The sphere packing problem in dimension 8. Annals of Mathematics 185 (2017), 991–1015. doi: 10.4007/annals.2017.185.3.7.

T. Baker, J. Gill and R. Solovay. Relativizations of the P versus NP question. SIAM Journal on Computing 4 (1975), 431–442. doi: 10.1137/0204037.

A. Razborov and S. Rudich. Natural proofs. Journal of Computer and System Sciences 55 (1997), 24–35. doi: 10.1006/jcss.1997.1494.

S. Aaronson and A. Wigderson. Algebrization: a new barrier in complexity theory. ACM Transactions on Computation Theory 1 (2009), Article 2. doi: 10.1145/1490270.1490272.

K. Mulmuley and M. Sohoni. Geometric complexity theory I. SIAM Journal on Computing 31 (2001), 496–526. doi: 10.1137/S009753970038715X.

J. Leray. Sur le mouvement d'un liquide visqueux emplissant l'espace. Acta Mathematica 63 (1934), 193–248. doi: 10.1007/BF02547354.

L. Caffarelli, R. Kohn and L. Nirenberg. Partial regularity of suitable weak solutions of the Navier-Stokes equations. Communications on Pure and Applied Mathematics 35 (1982), 771–831. doi: 10.1002/cpa.3160350604.

B. Gross and D. Zagier. Heegner points and derivatives of L-series. Inventiones Mathematicae 84 (1986), 225–320. doi: 10.1007/BF01388809.

V. A. Kolyvagin. Euler systems. The Grothendieck Festschrift, 1990. doi: 10.1007/978-0-8176-4576-2_6.

A. Beurling. A closure problem related to the Riemann zeta-function. Proceedings of the National Academy of Sciences 41 (1955), 312–314. doi: 10.1073/pnas.41.5.312.

L. Baez-Duarte. A strengthening of the Nyman-Beurling criterion for the Riemann hypothesis. Rendiconti di Matematica 14 (2003), 5–11.

Y. Andre. Pour une theorie inconditionnelle des motifs. Publications Mathematiques de l'IHES 83 (1996), 5–49. doi: 10.1007/BF02698643.

D. Lieberman. Numerical and homological equivalence of algebraic cycles on Hodge manifolds. American Journal of Mathematics 90 (1968), 366–374. doi: 10.2307/2373536.

R. Terras. A stopping time problem on the positive integers. Acta Arithmetica 30 (1976), 241–252.

J. C. Lagarias. The $3x + 1$ problem and its generalizations. American Mathematical Monthly 92 (1985), 3–23. doi: 10.2307/2322189.

T. Tao. Almost all orbits of the Collatz map attain almost bounded values. Forum of Mathematics, Pi 10 (2022), e12. doi: 10.1017/fmp.2022.8.

D. Bhattacharjee. Hopf-Like Fibrations on Calabi–Yau Manifolds. Preprints, 2025. doi: 10.20944/preprints202504.2581.v4.

D. Bhattacharjee and P. Nandi. Constructing Exotic Calabi–Yau 3-Folds via Quantum Inner State Manifolds. Preprints, 2025. doi: 10.20944/preprints202505.0700.v1.

D. Bhattacharjee. On Equivalences in Calabi–Yau Geometry from String Theory. Preprints, 2026. doi: 10.20944/preprints202602.0462.v1.

D. Bhattacharjee. Homotopy Groups of Spheres, Hopf Fibrations and Villarceau Circles II. Preprints, 2026. doi: 10.20944/preprints202602.2038.v1.

D. Bhattacharjee, P. Samal, R. Sadhu, S. S. Roy, S. Bhattacharya and S. N. Thakur. Topological Slice Structures in Calabi–Yau Manifolds. Preprints, 2026. doi: 10.20944/preprints202603.0911.v1.

D. Bhattacharjee. Seed Universality for Reflexive Polytopes in Dimension Four: Generation of the Kreuzer–Skarke Calabi–Yau Landscape via Toric Seed Orbits. Preprints, 2026. doi: 10.20944/preprints202603.1056.v1.

D. Bhattacharjee. Partitioning the Critical Strip: A Nyman–Beurling Approach to the Riemann Hypothesis. Preprints, 2025. doi: 10.20944/preprints202506.0772.v1.

D. Bhattacharjee. Generalized Poincare Conjecture via Alexander trick over C-isomorphism extension to h-cobordism on inclusion maps with associated Kan-complex. Research Square, 2022. doi: 10.21203/rs.3.rs-1830184/v1.

D. Bhattacharjee. Quantum Computational Frameworks on Symplectic State Manifolds. TechRxiv, 2023. doi: 10.36227/techrxiv.23978031.v1.

D. Bhattacharjee. Holonomic Quantum Computing. SSRN Electronic Journal, 2026. doi: 10.2139/ssrn.6066428.

D. Bhattacharjee, U. Bhattacharya and S. Ghosh. One Missing Axiom for the Complete Closure of the Heterotic Landscape: Resonance Classifier of a Vibrating String Creates the Typical Particle Classes. Preprints, 2026. doi: 10.20944/preprints202603.0792.v2.